

Jerarquías Cache en Procesadores Multicore

Indice

- Multiprocesadores de memoria compartida
- Procesadores multicore: Jerarquía cache
- Coherencia cache
- Optimización del uso de la jerarquía cache multicore

Indice

- **Multiprocesadores de memoria compartida**
- Procesadores multicore: Jerarquía cache
- Coherencia cache
- Optimización del uso de la jerarquía cache multicore

Introducción

- Multiprocesador de memoria compartida

- Extiende el sistema de memoria para soportar múltiples procesadores
- La organización es atractiva para:
 - » Multiprogramación de alta productividad (*throughput*) (servidores multiprocesador)
 - » Computación paralela (facilidad de programación)

- Comunicación

- La **comunicación** es implícita: operaciones de lectura/escritura en variables globales

- Sincronización

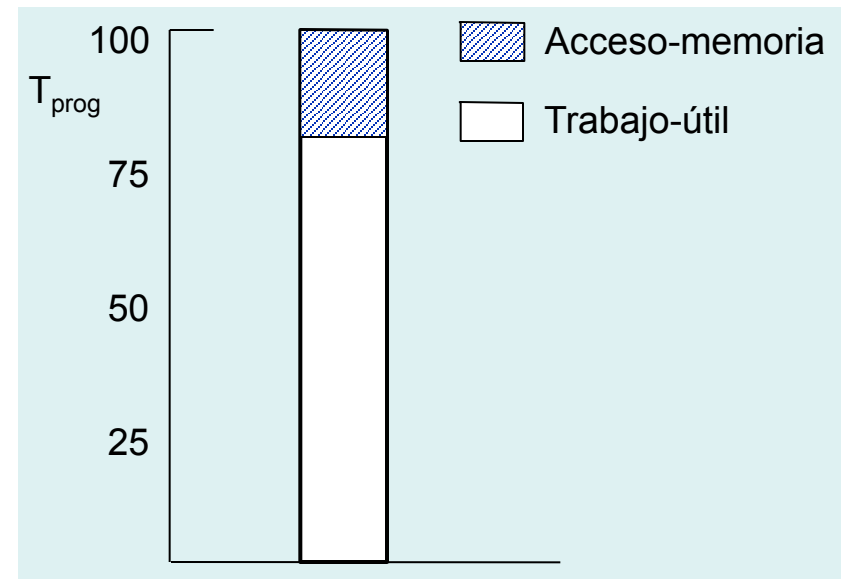
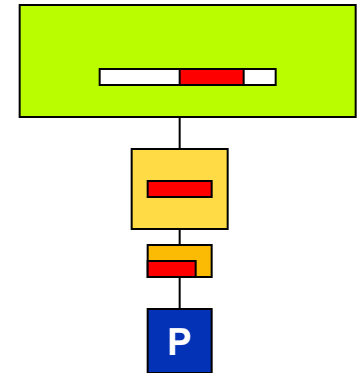
- Los accesos a memoria global desde diferentes procesadores no están ordenados
- La **sincronización** permite establecer ordenamientos parciales

- Latencia y ancho de banda

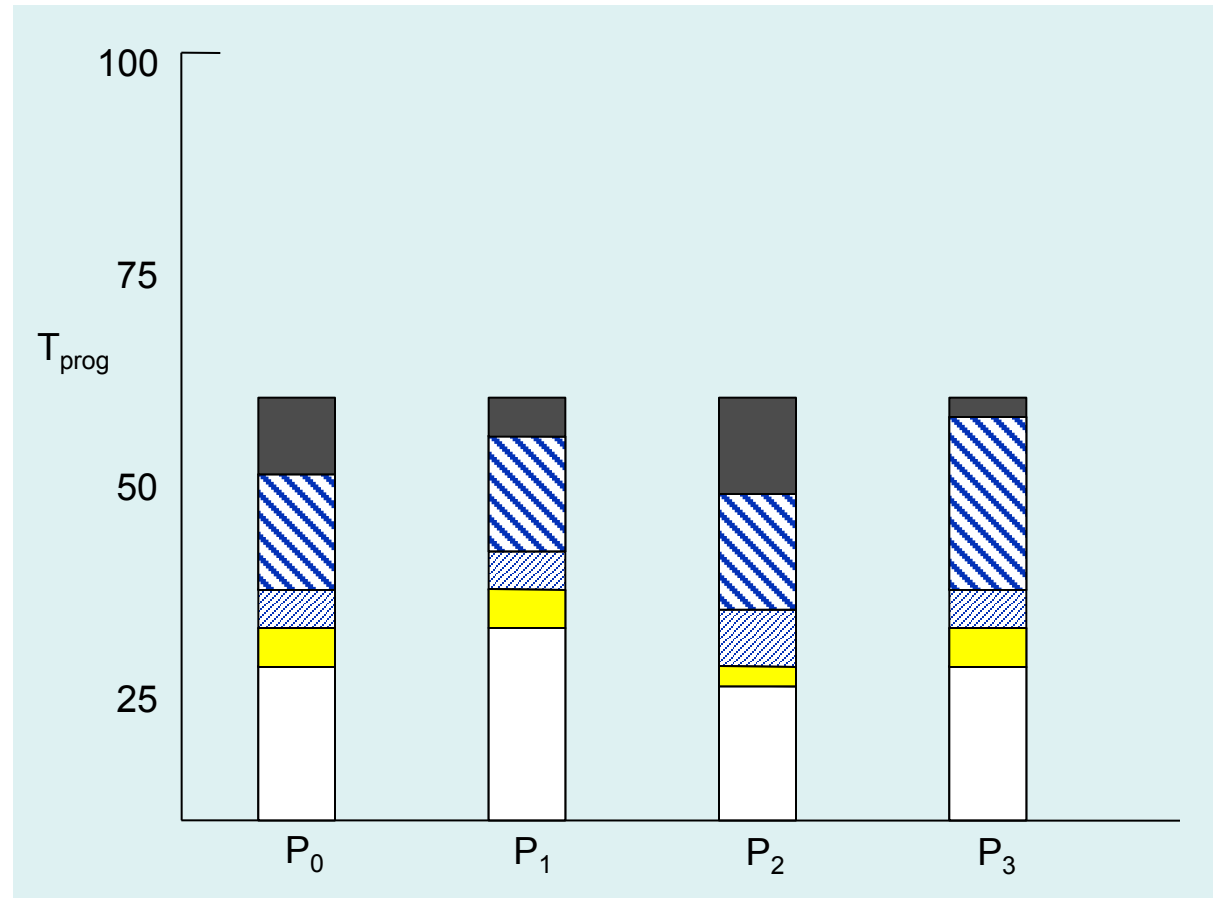
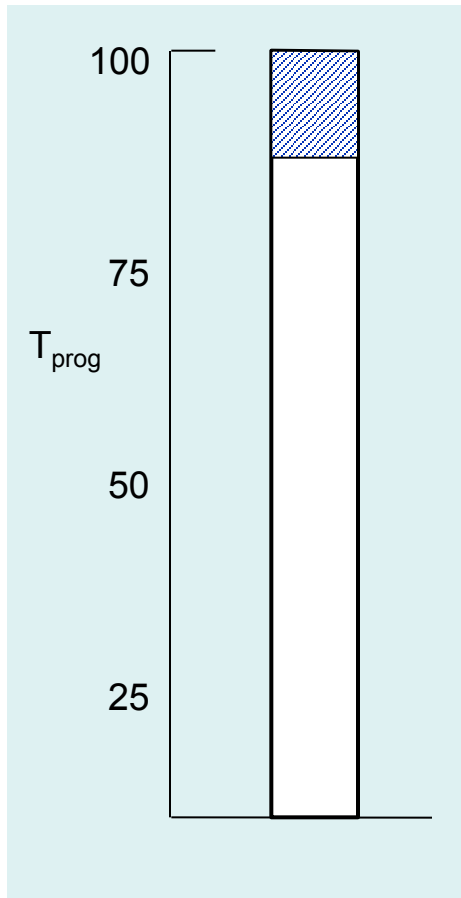
- La comunicación entre procesos/tareas se realiza mediante la jerarquía de memoria
- **Es importante eliminar el tráfico de memoria innecesario**

Rendimiento: Uniprocador

- El rendimiento depende de forma importante de la jerarquía de memoria
 - La gestión del tráfico de memoria está realizada en hardware
- Tiempo de ejecución de un programa
 - $T_{\text{prog}} = \text{Trabajo-útil} + \text{Acceso-memoria}$
- El tiempo de acceso a los datos puede reducirse:
 - Optimizando la máquina
 - » Caches más grandes
 - » Caches más rápidas
 - Optimizando el programa
 - » Localidad espacial y temporal



Rendimiento: Multiprocesador



 Acceso-memoria

 Sincronización

 Trabajo-extra-paralelo

 Trabajo-útil

 Acceso-memoria-remota

Rendimiento: Multiprocesador

- Speed-up (factor de mejora del rendimiento)

$$S(n) = T(1)/T(n)$$

$$\text{Ej: } S(4) = 100 / 60 = 1,66$$

- Eficiencia del sistema

$$E(n) = S(n) / n = T(1) / n \cdot T(n)$$

$$\text{Ej: } E(4) = 1,66 / 4 = 0,41$$

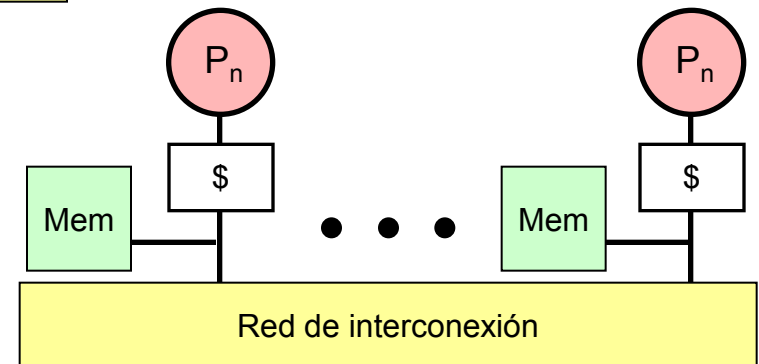
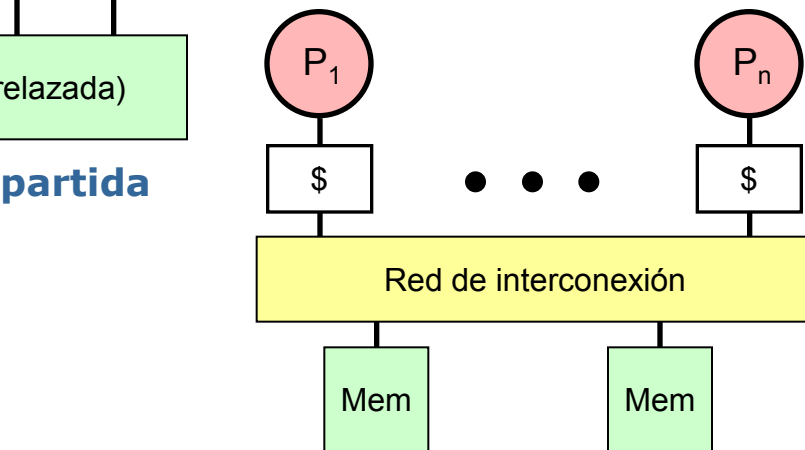
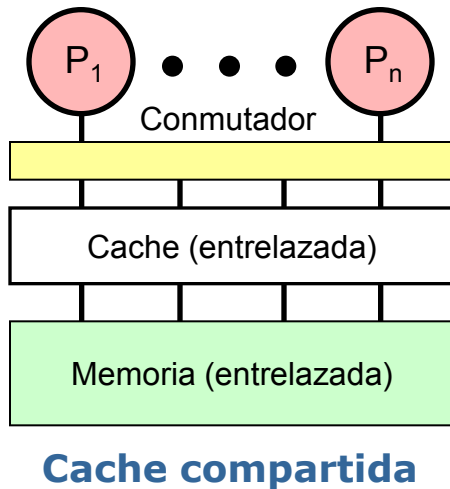
La eficiencia es una comparación del grado de speed-up conseguido frente al valor máximo. Dado que $1 \leq S(n) \leq n$, tenemos $1/n \leq E(n) \leq 1$.

Organización del Sistema de Memoria



Escalabilidad

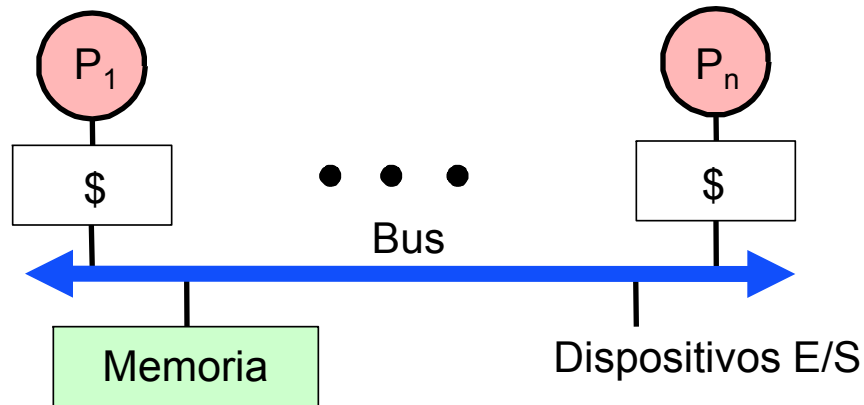
(Aumento del rendimiento proporcional al aumento en tamaño)



Memoria distribuida (NUMA)

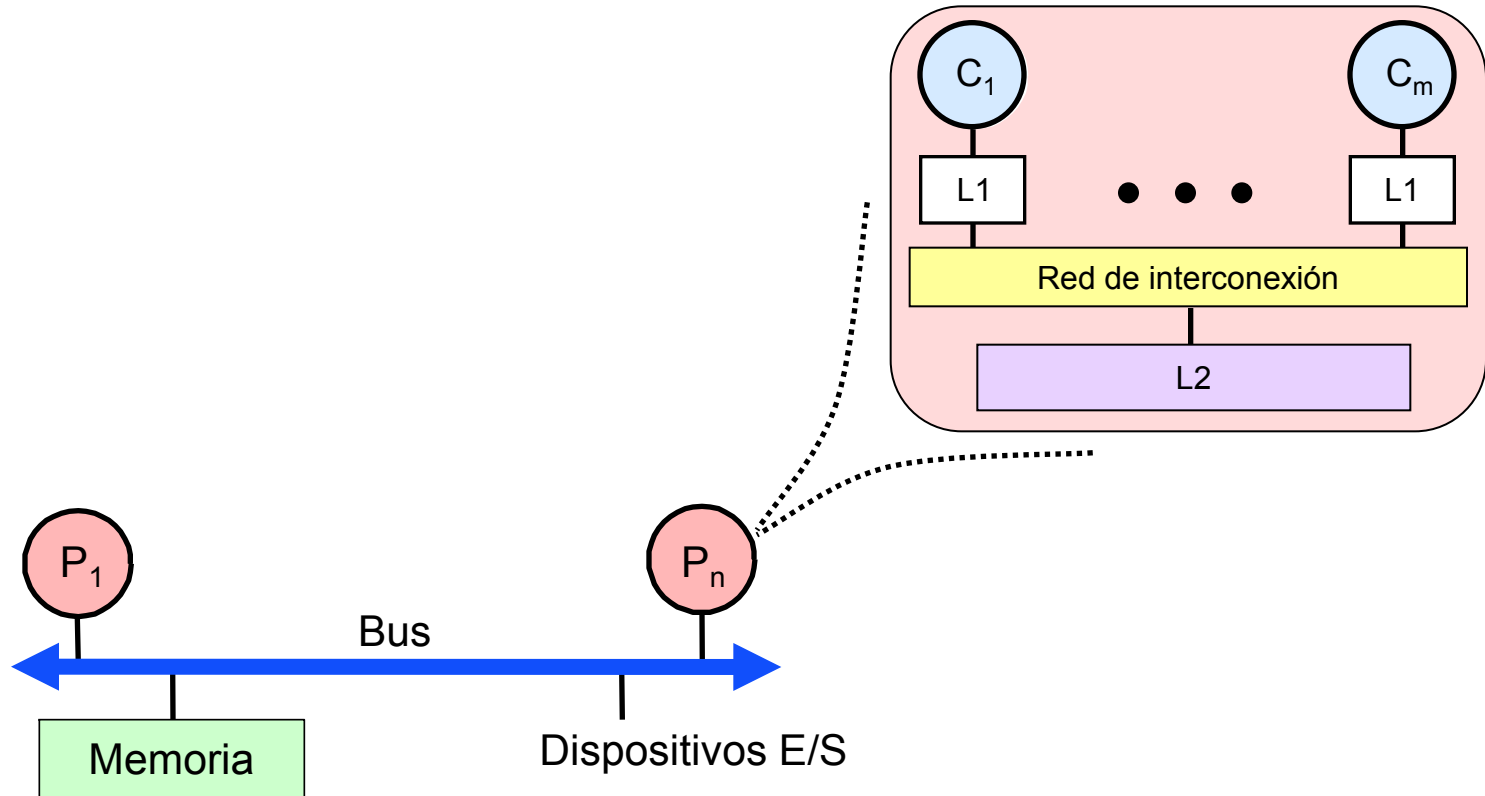
Multiprocesador con Bus Común (SMP)

- Organización muy habitual
 - Común en el mercado de los servidores
- Organización atractiva para:
 - Servidor multiprogramado
 - » Productividad (*Troughput*)
 - Máquina paralela
 - » Modelo de programación cómodo



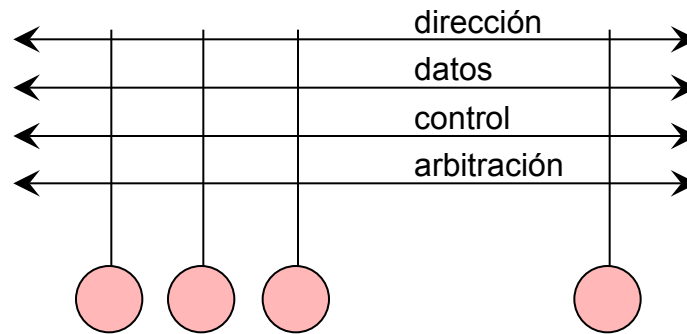
SMP Multicore

- SMP con procesadores multicore



SMP Multicore: Red / Bus

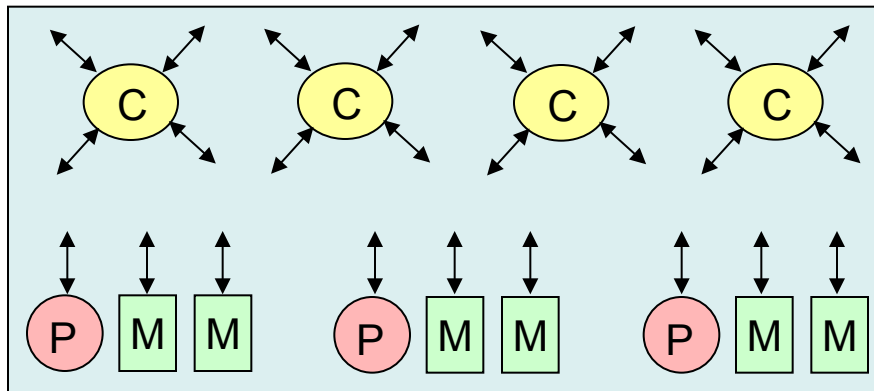
- Bus compartido de interconexión
 - Comunicación visible a todos los nodos
 - Las transacciones de bus pueden estar segmentadas



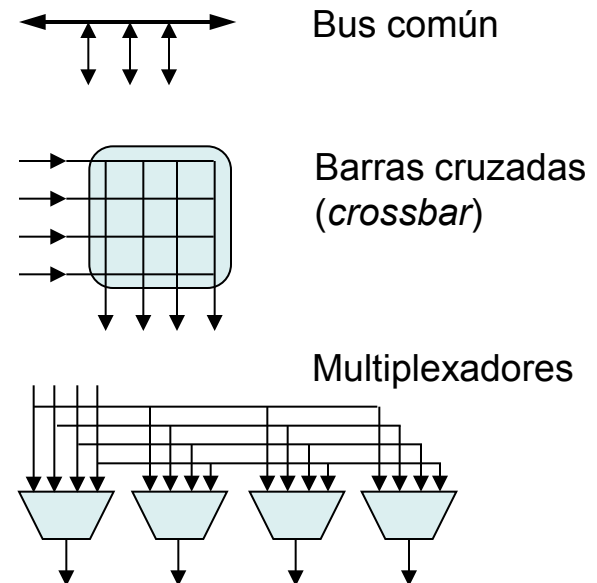
Escalabilidad en el Ancho de Banda

- Límite fundamental del ancho de bandas

- El número de cables físicos constituye un límite fundamental
- Para aumentar el ancho de banda necesitamos, por tanto, un gran número de cables físicos
- Estos cables físicos deben organizarse y conectarse entre sí, utilizando para ello **conmutadores**



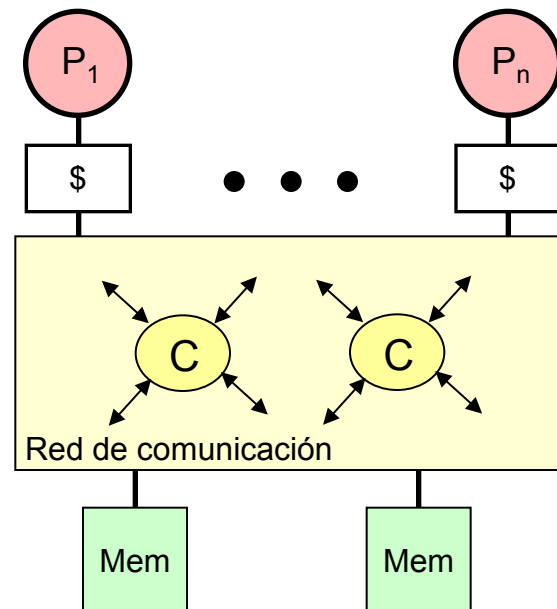
C: conmutadores
P: procesadores
M: memorias



Organización UMA

• Escalabilidad

- La red de interconexión escalable permite soportar un número **considerable** de procesadores
- Sin embargo, la red debe soportar el tráfico de memoria para resolver **todos los fallos cache**
- La red no debe ser muy profunda para no aumentar demasiado la latencia

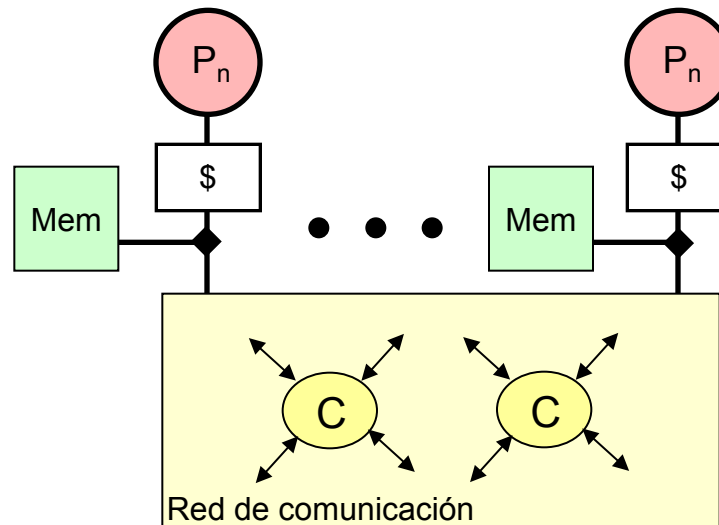


Todos los fallos cache circulan por la red

Organización NUMA

- Escalabilidad

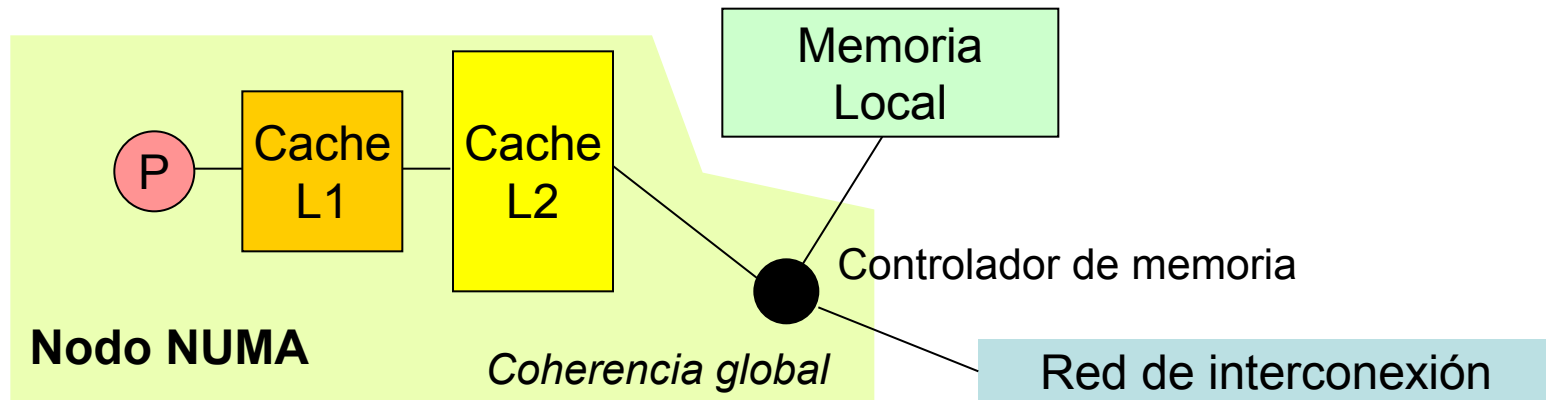
- La red de interconexión escalable permite soportar un número **bastante elevado** de procesadores
- Esta vez, la red debe soportar el tráfico de memoria para resolver **sólo los fallos cache que no correspondan a la memoria local**
- La red no debe ser muy profunda para no aumentar demasiado la latencia



Todos los fallos cache que no se resuelven en la memoria local circulan por la red

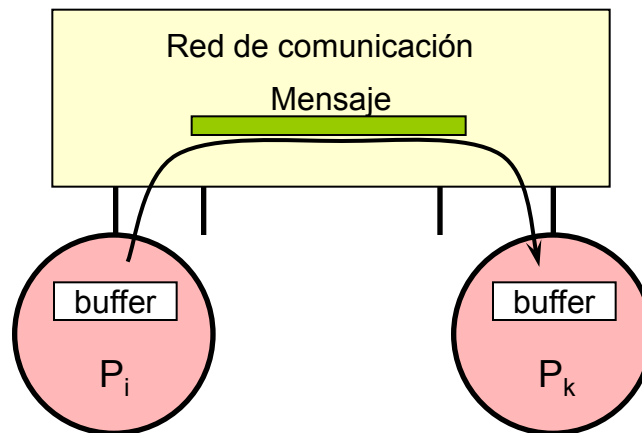
Operación de Comunicación

- En una arquitectura de memoria compartida, el controlador de memoria inicia y recibe las transacciones de red
- Una operación de memoria consta de una o varias transacciones de red para realizarse
 - La granularidad de la comunicación es un bloque cache

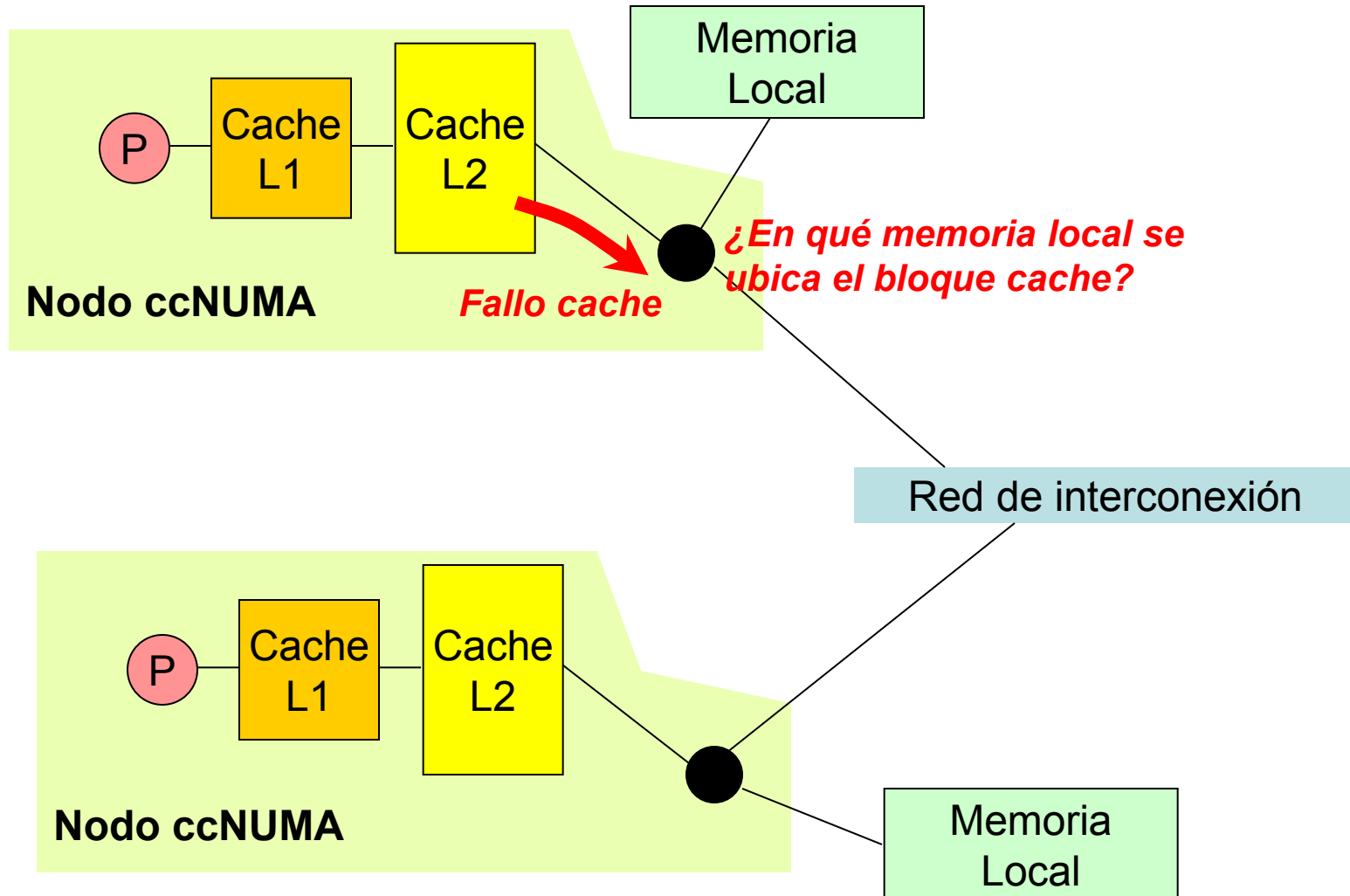


Transacción de Red

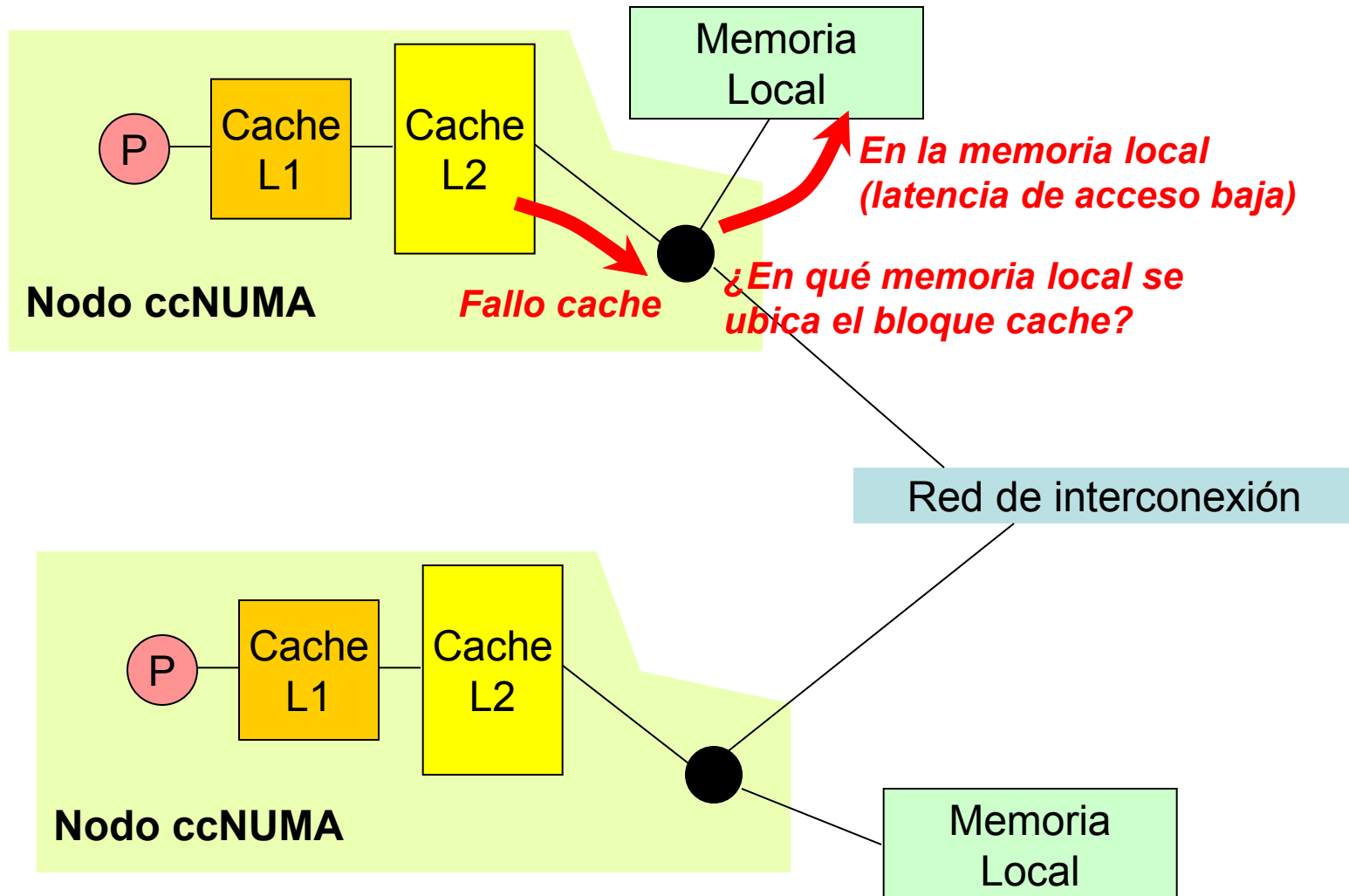
- La primitiva de comunicación básica es la transacción de red
 - La **transacción de red** es una transferencia unidireccional de información desde un nodo origen a un nodo destino que:
 - » Causa una acción en el destino
 - » No es visible inmediatamente desde el nodo origen
- La transacción de red es similar a la transacción de bus, pero carece de la visibilidad global



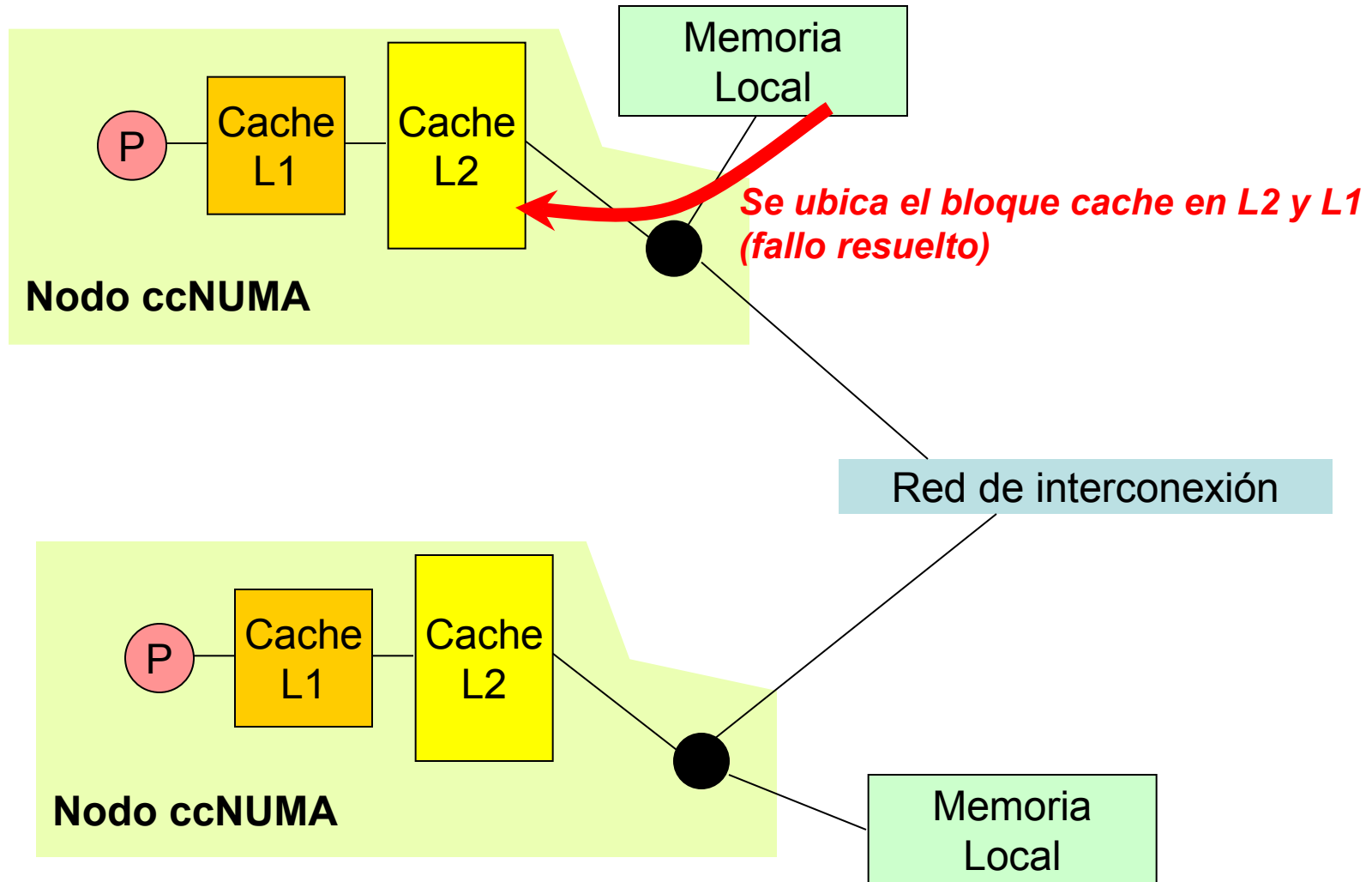
NUMA: Acceso a Memoria



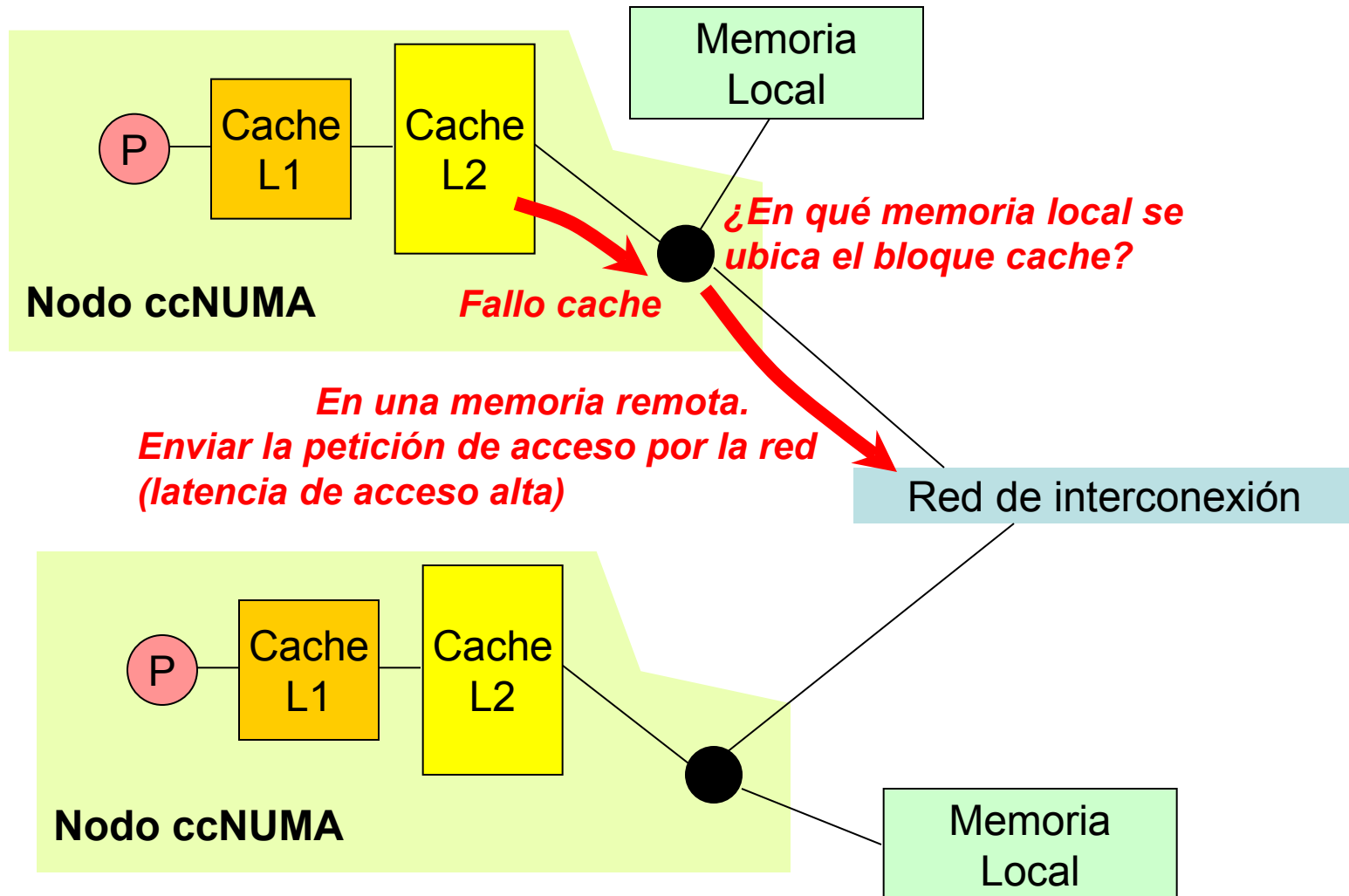
NUMA: Acceso a Memoria



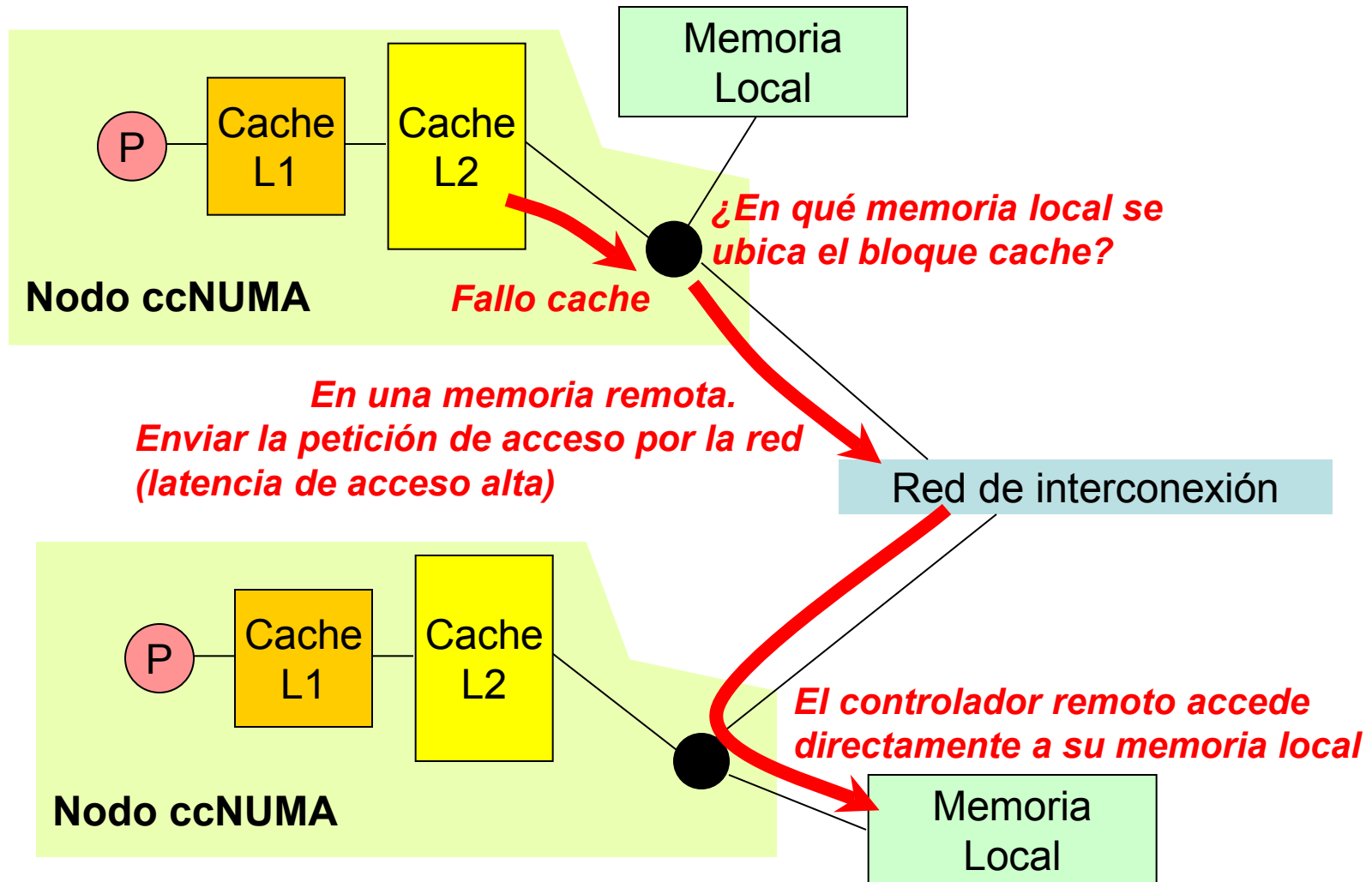
NUMA: Acceso a Memoria



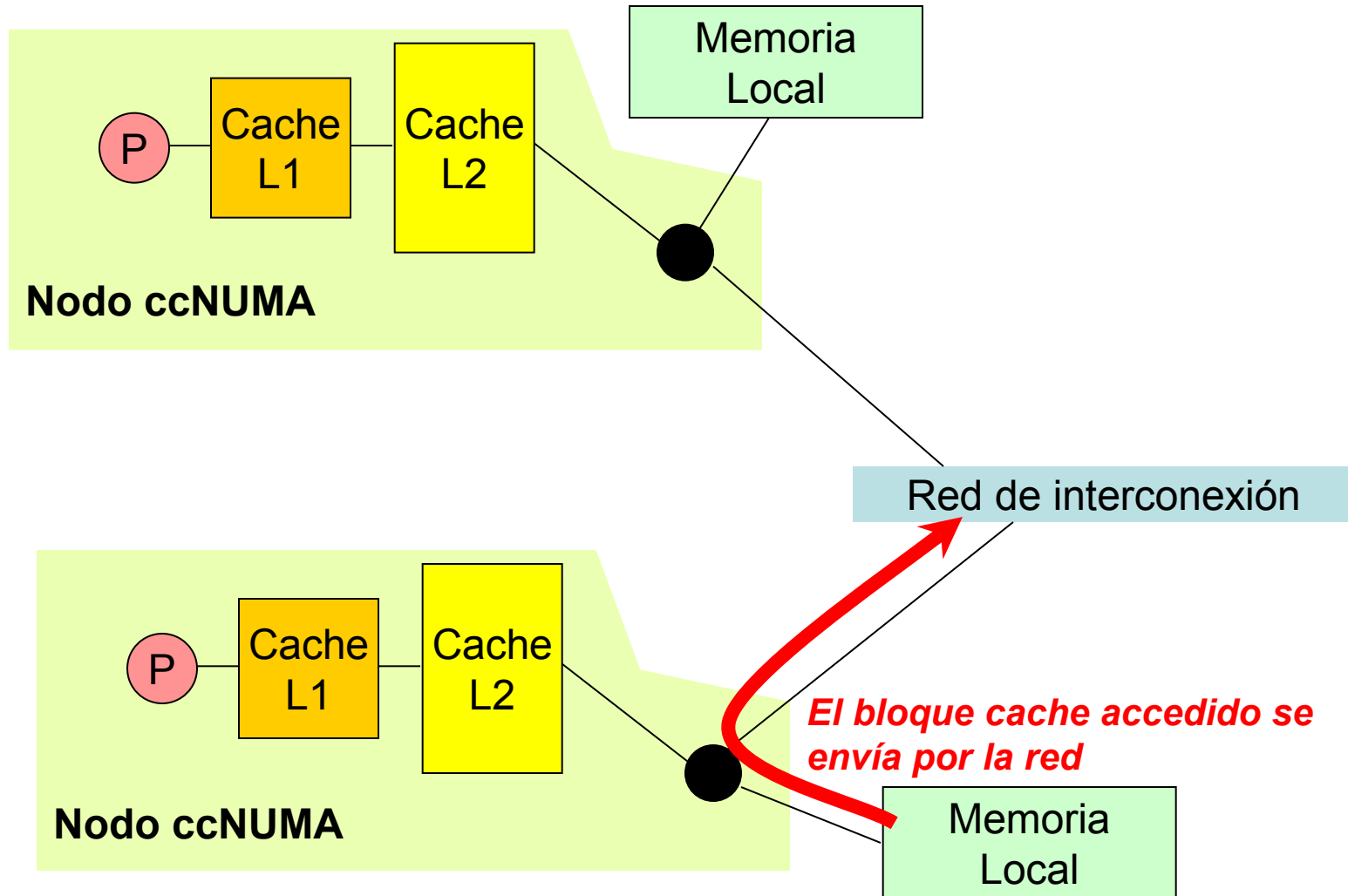
NUMA: Acceso a Memoria



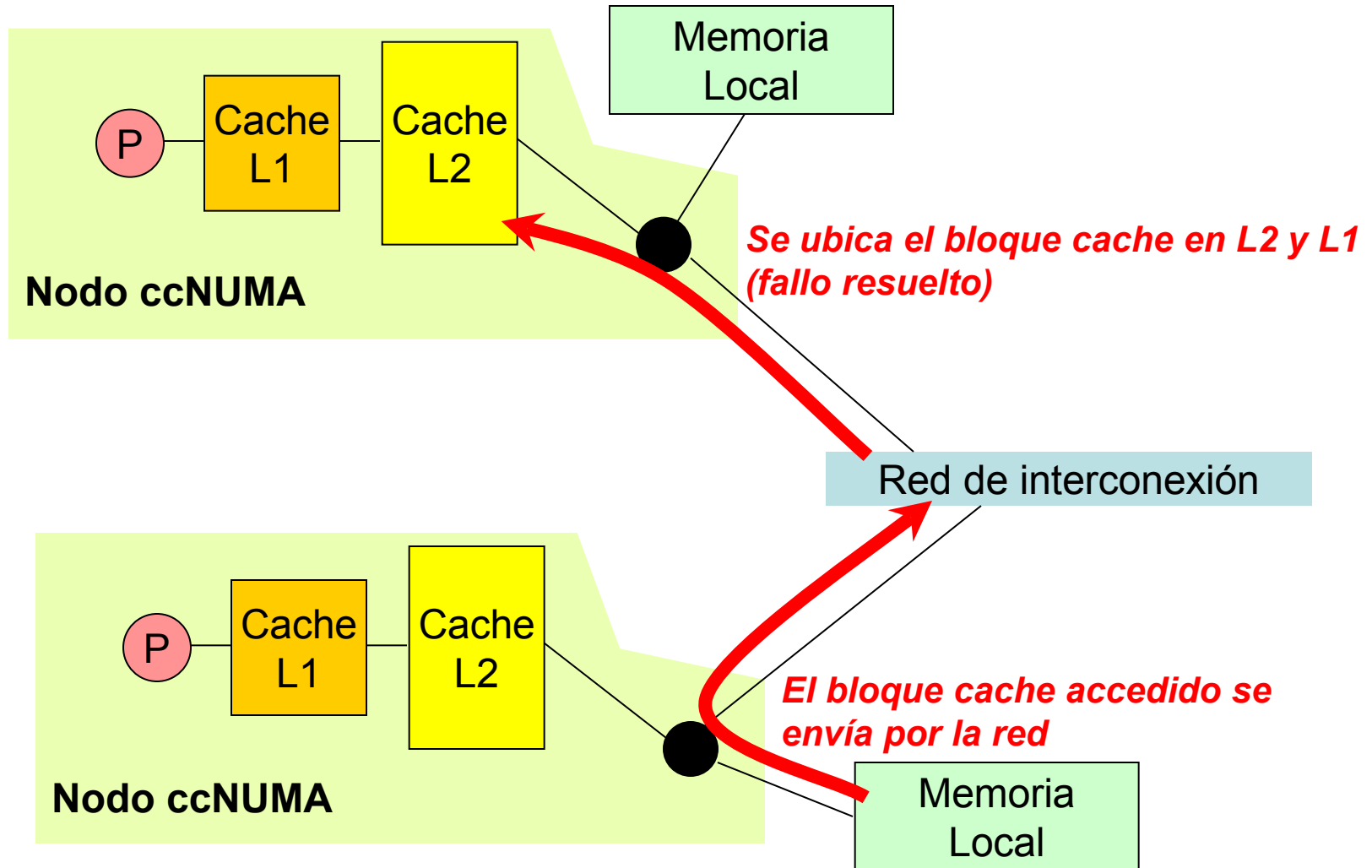
NUMA: Acceso a Memoria



NUMA: Acceso a Memoria



NUMA: Acceso a Memoria



Indice

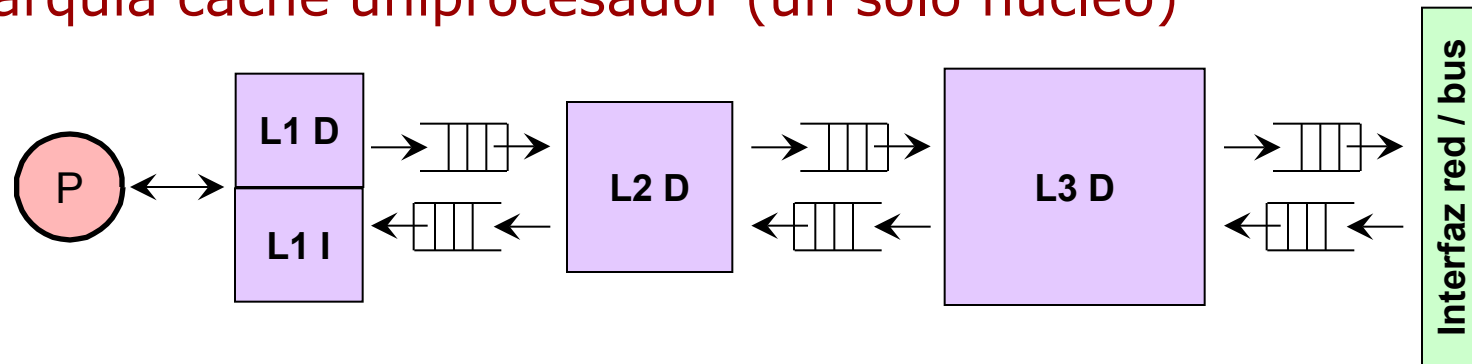
- Introducción
- **Procesadores multicore: Jerarquía cache**
- Coherencia cache
- Optimización del uso de la jerarquía cache multicore

Procesador Multicore: Jerarquía Cache

- **Cache no bloqueante**

- Los fallos cache se resuelven si bloquear la cache
- Pueden procesar múltiples aciertos y fallos concurrentemente
- La resolución de fallos se solapan con otros fallos y con computación

- **Jerarquía cache uniprocador (un solo núcleo)**



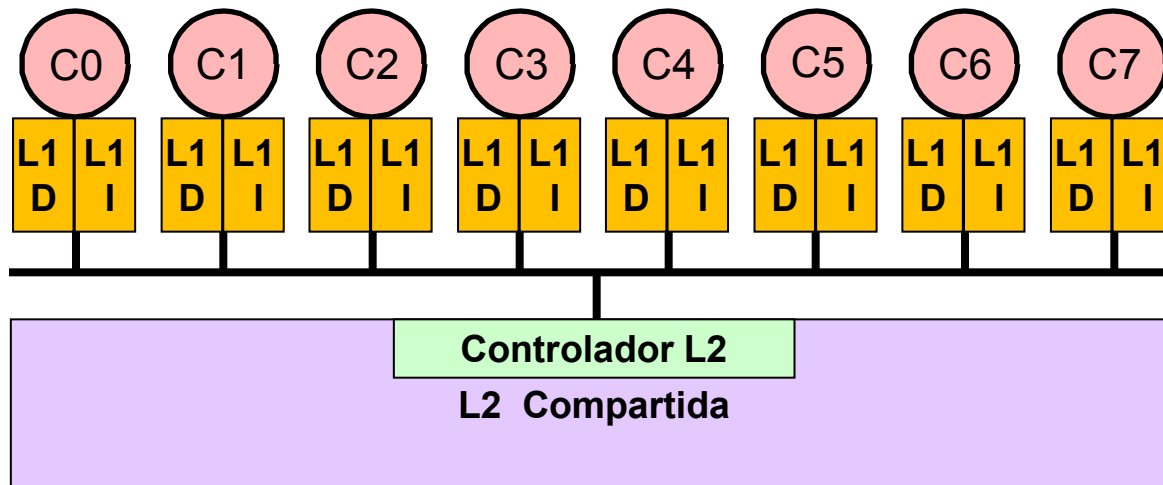
- **Jerarquía cache multicore: Alternativas de diseño**

- Cache compartida o privada
- Cache compartida centralizada o distribuida
- Cache de acceso uniforme y no uniforme

Jerarquía Cache Compartida

- **Cache compartida**

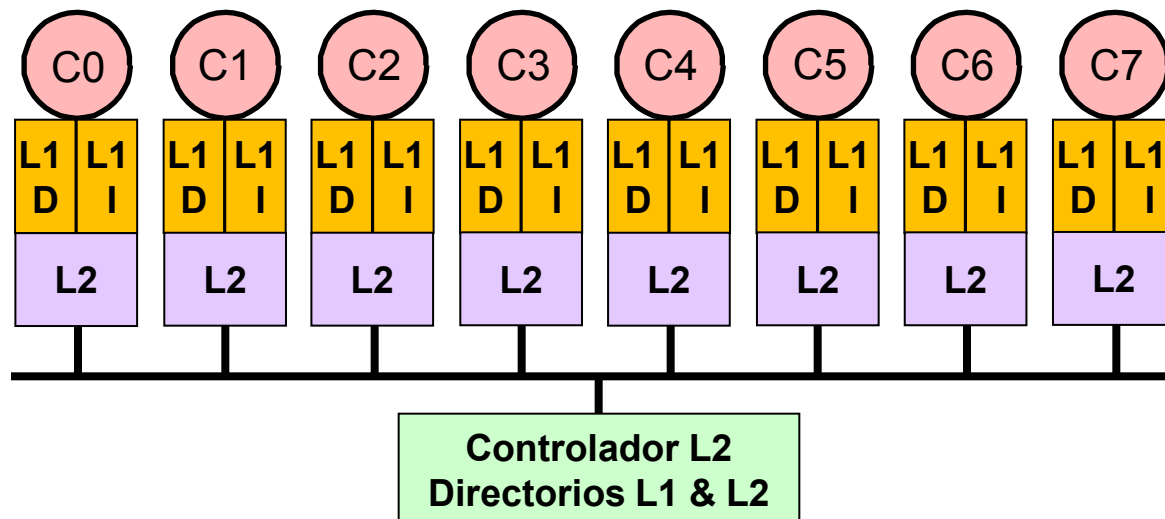
- Cache L1 es privada para minimizar la latencia de acceso a los datos
- Cache L2 es compartida entre los núcleos
- **Ventajas:**
 - » No se duplican los datos compartidos en la L2
 - » El espacio L2 se asigna a los núcleos dinámicamente (según demanda)
- **Desventajas:**
 - » Interferencia entre los datos accedidos por diferentes núcleos
 - » Caches compartidas grandes tienen latencia alta



Jerarquía Cache Privada

- **Cache privada**

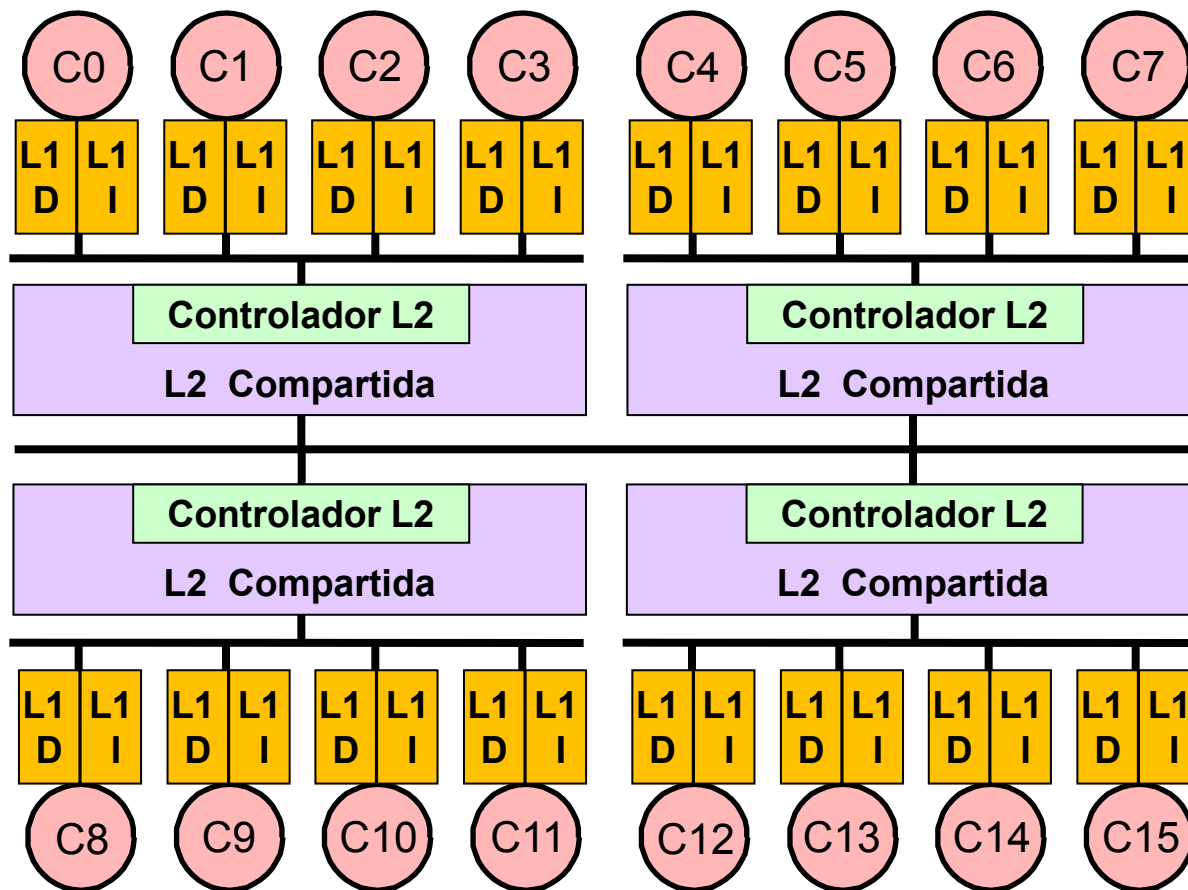
- Caches L1 y L2 son privadas
- **Ventajas:**
 - » No hay interferencia entre los datos accedidos por diferentes núcleos
 - » Las caches L2 privadas tiene menos latencia
- **Desventajas:**
 - » Se duplican los datos compartidos en la L2



Jerarquía Cache Compartida/Privada

- Cache combinada, privada y compartida

- Cuando el número de núcleos es amplio, una solución parcialmente compartida puede ser la mejor opción

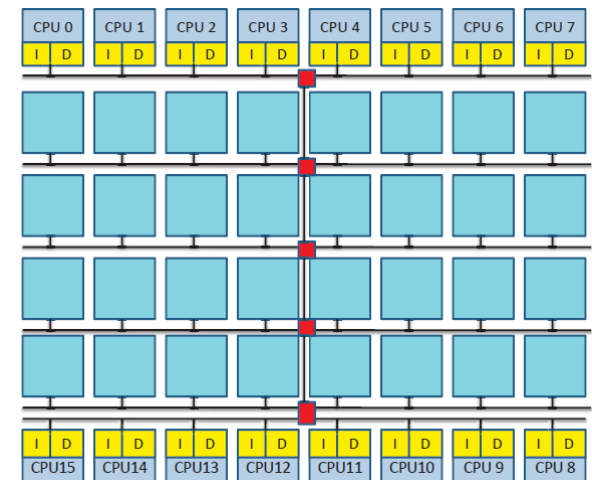
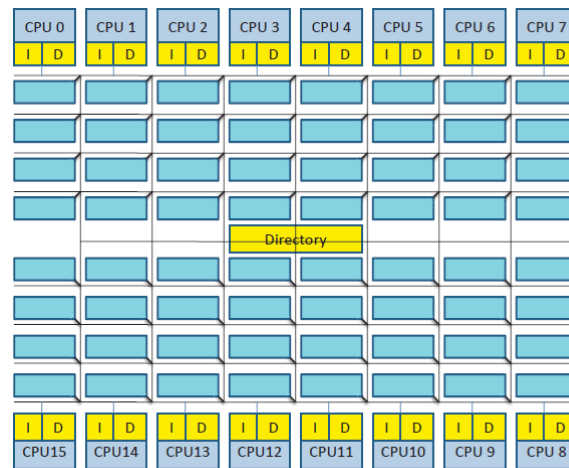
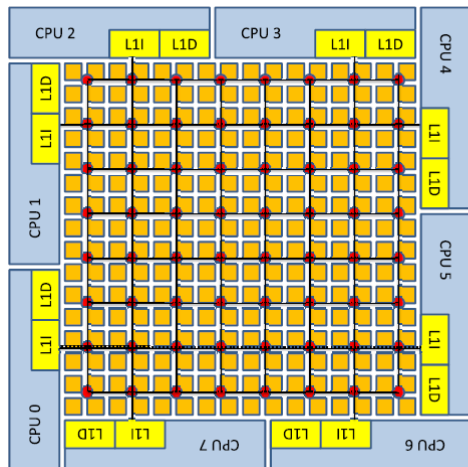


Jerarquía Cache Compartida Central

- **Cache compartida centralizada**

- Si la cache compartida es grande, se divide en bancos interconectados en red, pero ubicados de forma compacta en el chip
- El directorio para localizar los datos puede estar centralizado o distribuido en controladores asociados a cada banco
- La cache compartida constituye una estructura centralizada en el chip:
 - » Se simplifica el movimiento de datos entre bancos
 - » Se simplifica la conexión de la cache compartida con el controlador de memoria

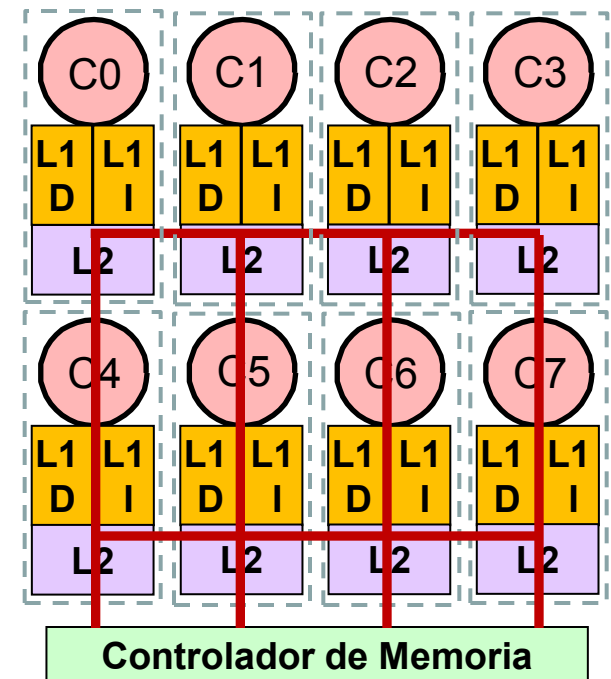
Caso: Primeros juegos en dispositivos móviles



Jerarquía Cache Compartida Distribuida

- Cache compartida distribuida

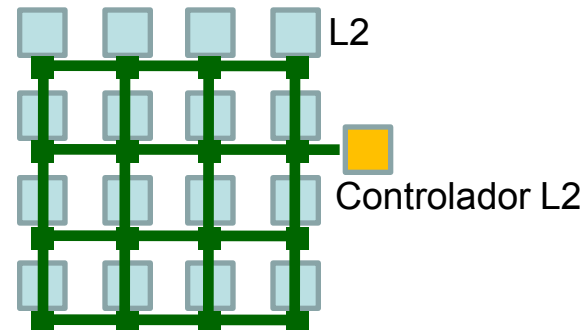
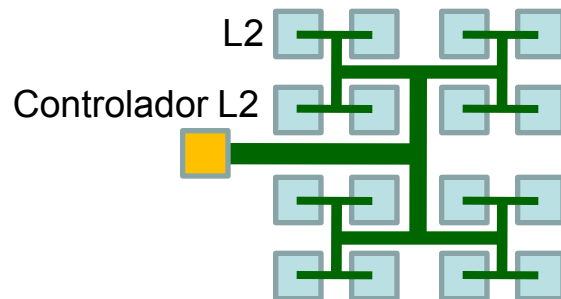
- Los bancos de la cache compartida se distribuyen físicamente en el chip
- Cada banco se ubica próximo físicamente a un núcleo
- El conjunto *núcleo – cache L1 – banco cache L2* se denomina **tile** (segmento)
- Facilita el diseño escalable (replicación de segmentos) y la verificación
- Sin embargo, la latencia de acceso a los datos compartidos es mayor
- Tiene un mejor comportamiento térmico y escalabilidad que la solución centralizada



Jerarquía Cache Uniforme

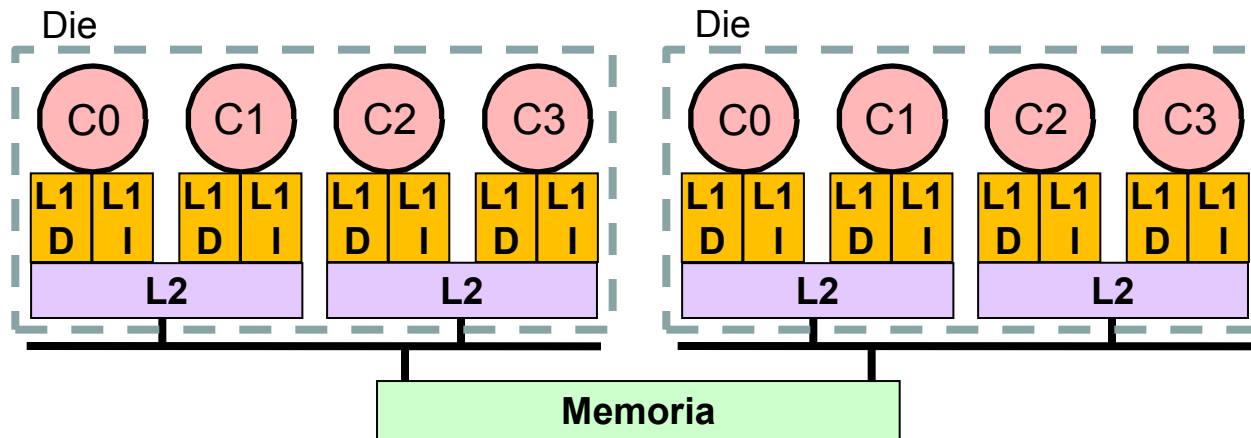
- Cache de acceso uniforme y no uniforme

- UCA (*Uniform Cache Access*) , NUCA (*Non-Uniform Cache Access*)
- Las cache multi-banco pueden diseñarse como UCA o NUCA, en función de la red de interconexión
- Ejemplo: Cache UCA multi-banco con red en árbol tipo H
- Ejemplo: Cache NUCA multi-banco con red en malla
- Variantes de cache NUCA:
 - » S-NUCA (*Static NUCA*): Todas las vías de un conjunto se asignan al mismo banco cache
 - » D-NUCA (*Dynamic NUCA*): Las vías de un conjunto se distribuyen entre diferentes bancos, y se permite el movimiento de bloques cache entre esos bancos



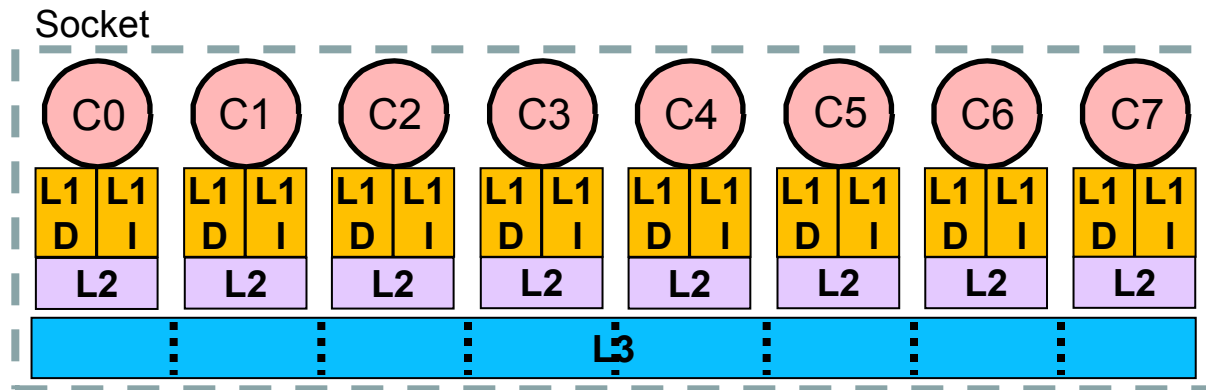
Jerarquía Cache: Ejemplo 1

- Quad-core Intel Xeon Core X5355 (Clovertown)
 - L1D y L1I: 32KB
 - L2: 4MB, compartida entre 2 núcleos
 - Memoria: Hasta 16GB



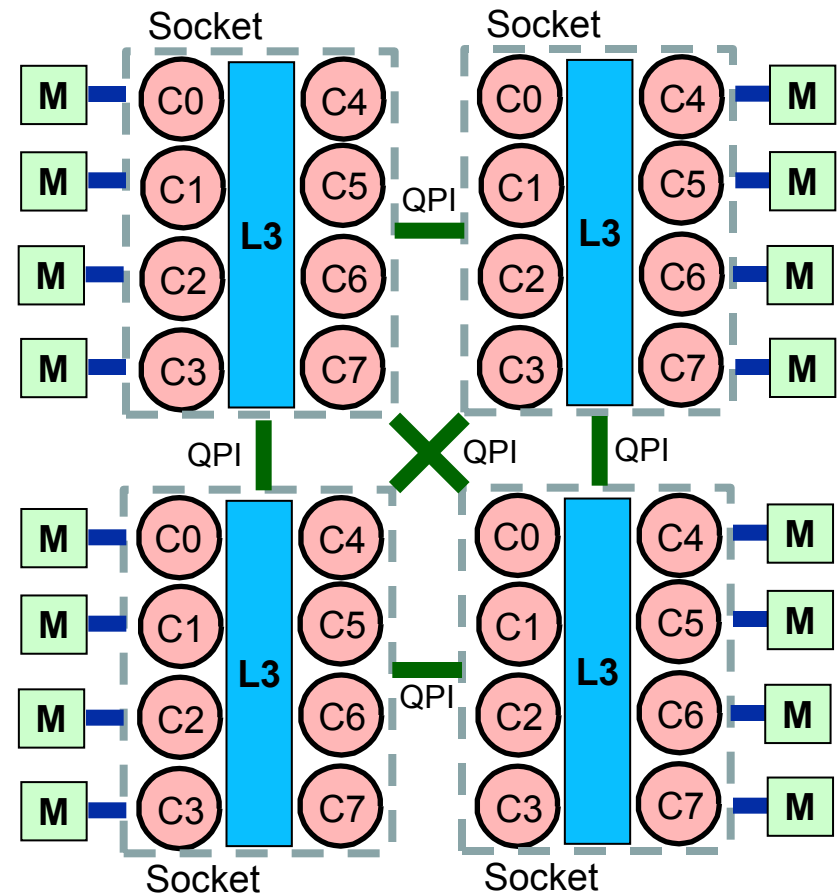
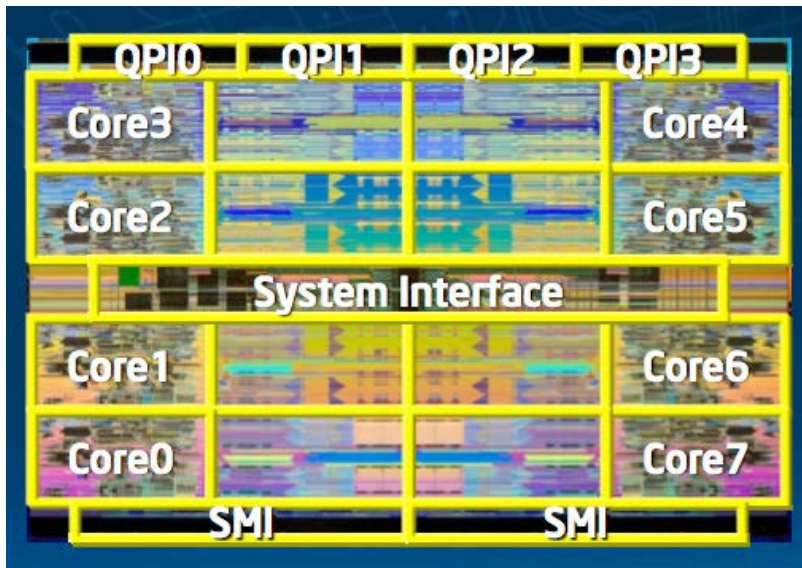
Jerarquía Cache: Ejemplo 2

- Eight-core Intel Xeon Nehalem X7550 (Beckton)
 - L1D y L1I: 32KB
 - L2: 256KB, privada a cada núcleo
 - L3: 18MB, compartida entre todos los núcleos, multi-banco, UCA



Jerarquía Cache: Ejemplo 2

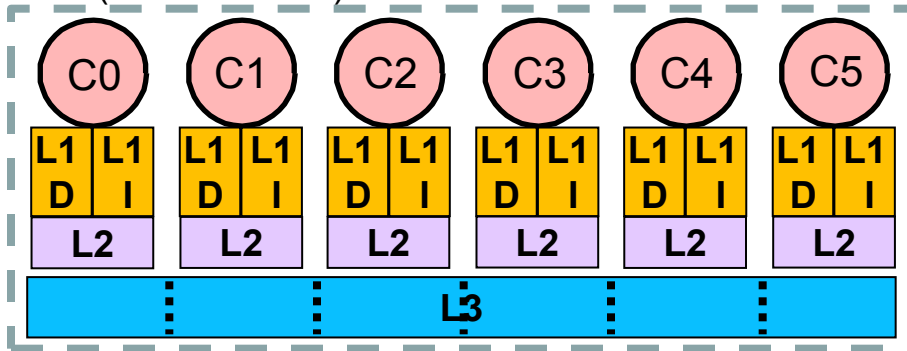
- Eight-core Intel Xeon Nehalem X7550 (Beckton)



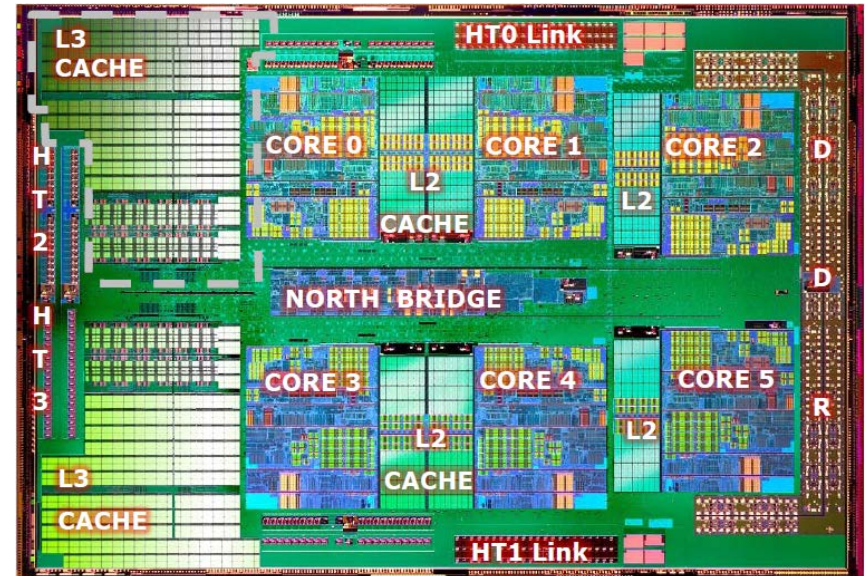
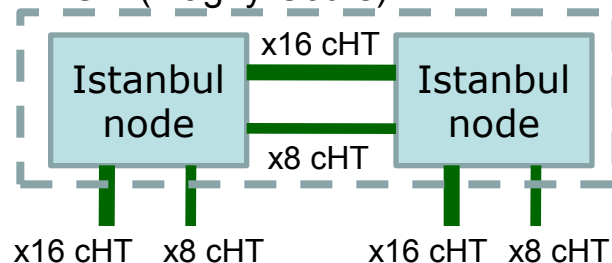
Jerarquía Cache: Ejemplo 3

- Twelve-core AMD Opteron 6172 (Magny-Cours)
 - L1D y L1I: 64KB
 - L2: 512KB, privada a cada núcleo
 - L3: 12MB, compartida entre todos los núcleos (6+6), multi-banco, UCA

Die (Istanbul node)

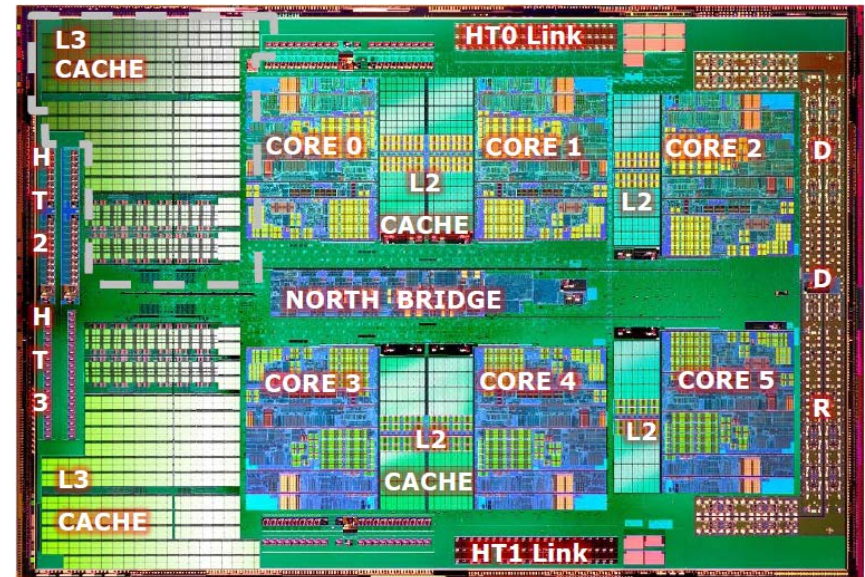
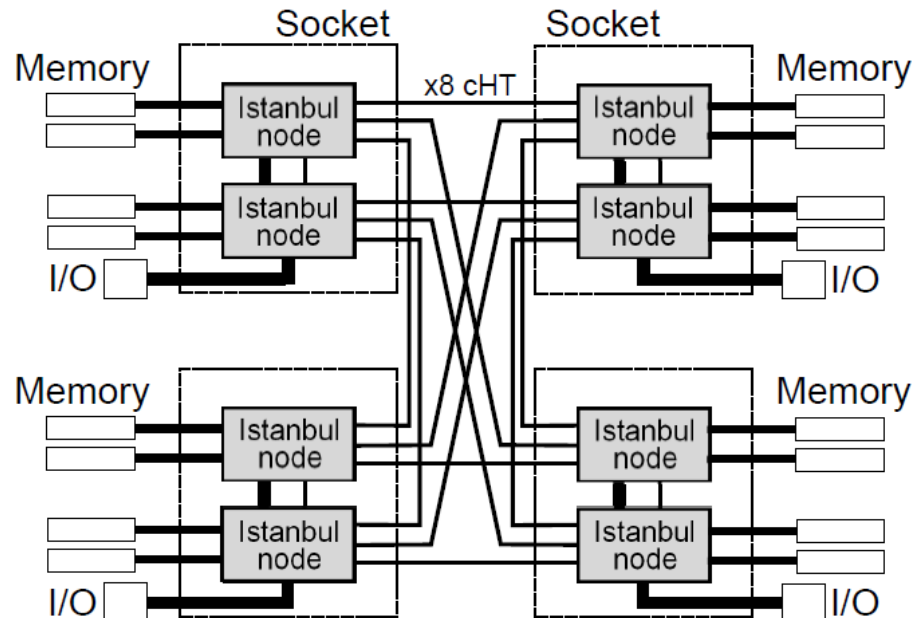


MCM (Magny-Cours)



Jerarquía Cache: Ejemplo 3

- Twelve-core AMD Opteron 6172 (Magny-Cours)



Indice

- Introducción
- Procesadores multicore: Jerarquía cache
- **Coherencia cache**
- Optimización del uso de la jerarquía cache multicore

El Problema de la Coherencia Cache

- Considerar problemas de dependencia de datos.

Real.

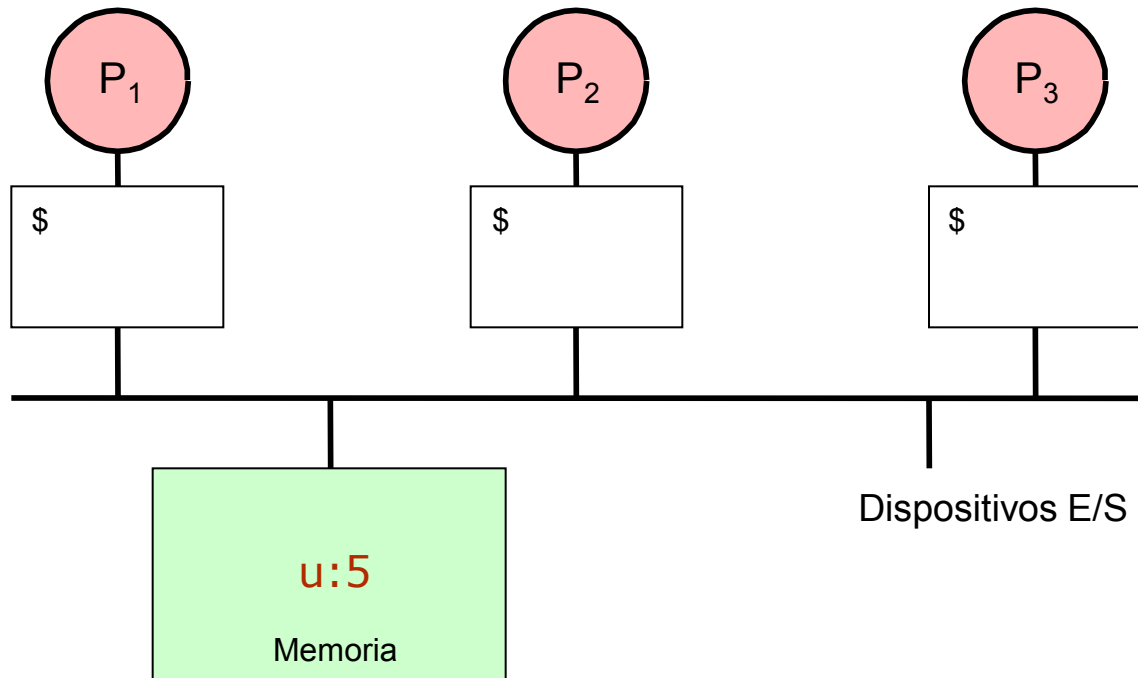
Antidependencia.

De entrada.

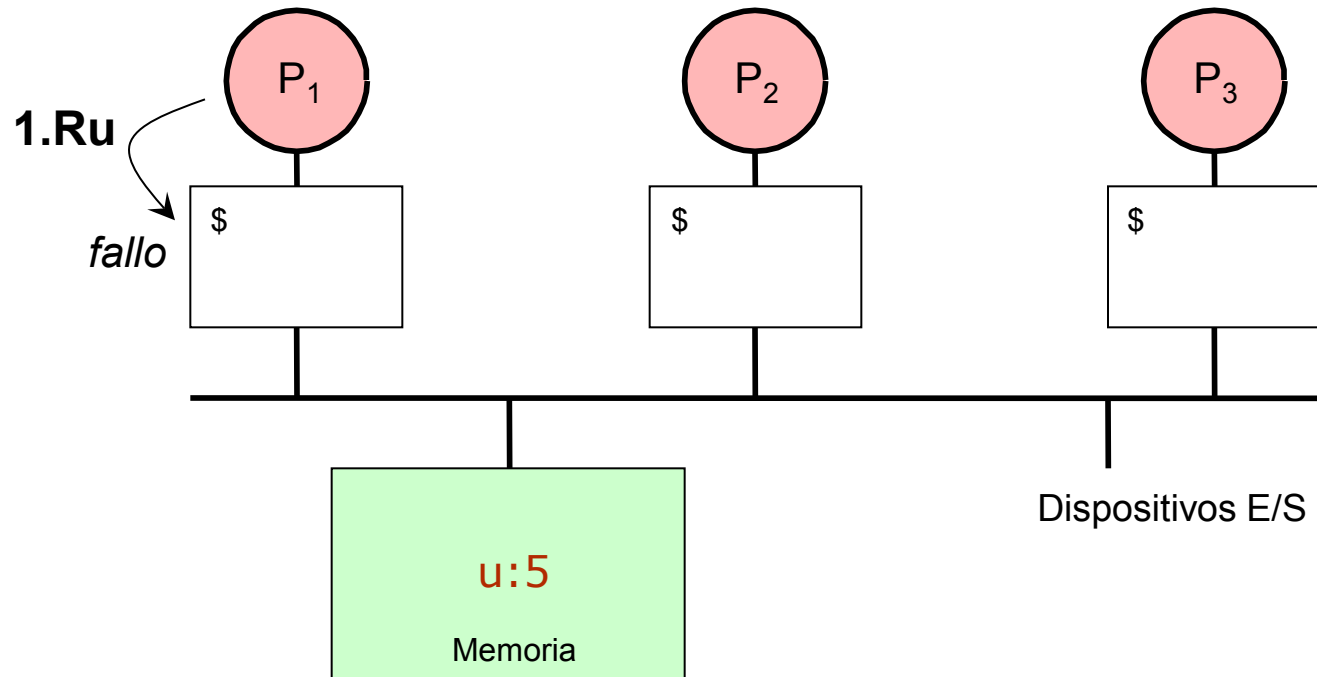
De salida.

Loop carried.

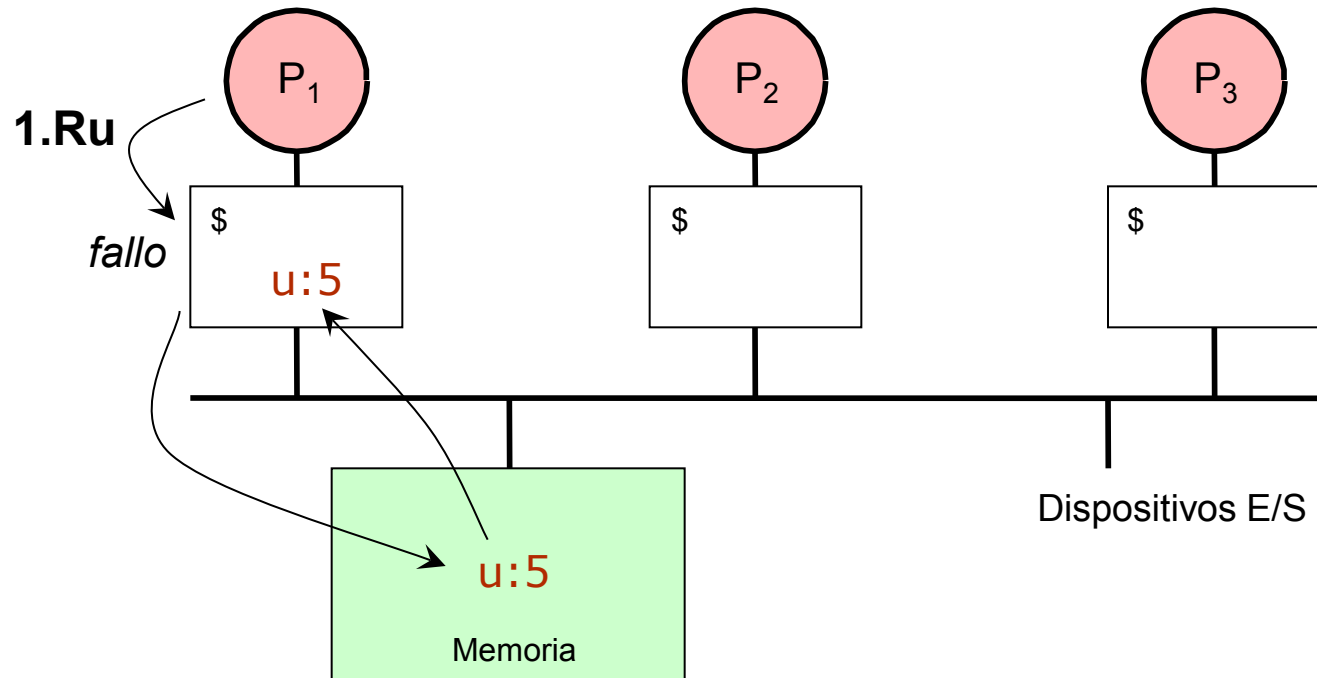
El Problema de la Coherencia Cache



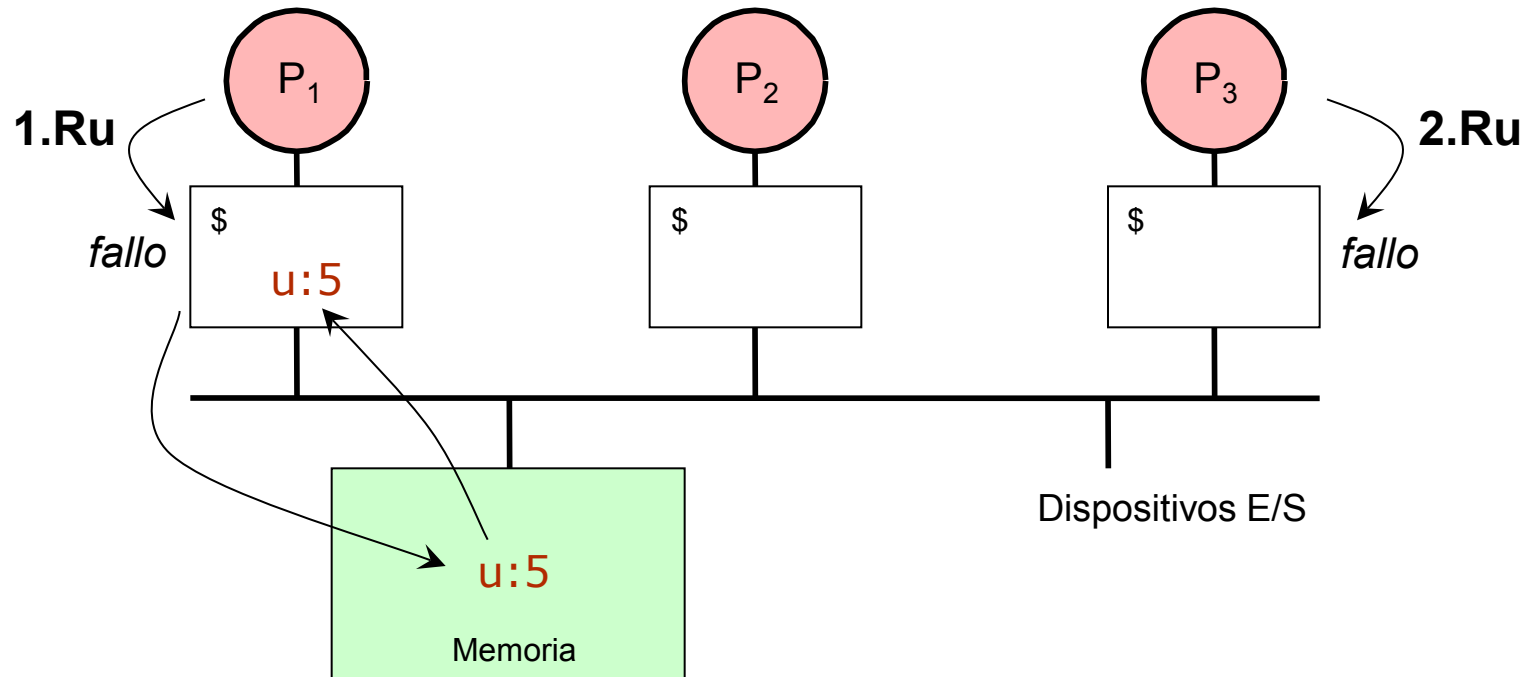
El Problema de la Coherencia Cache



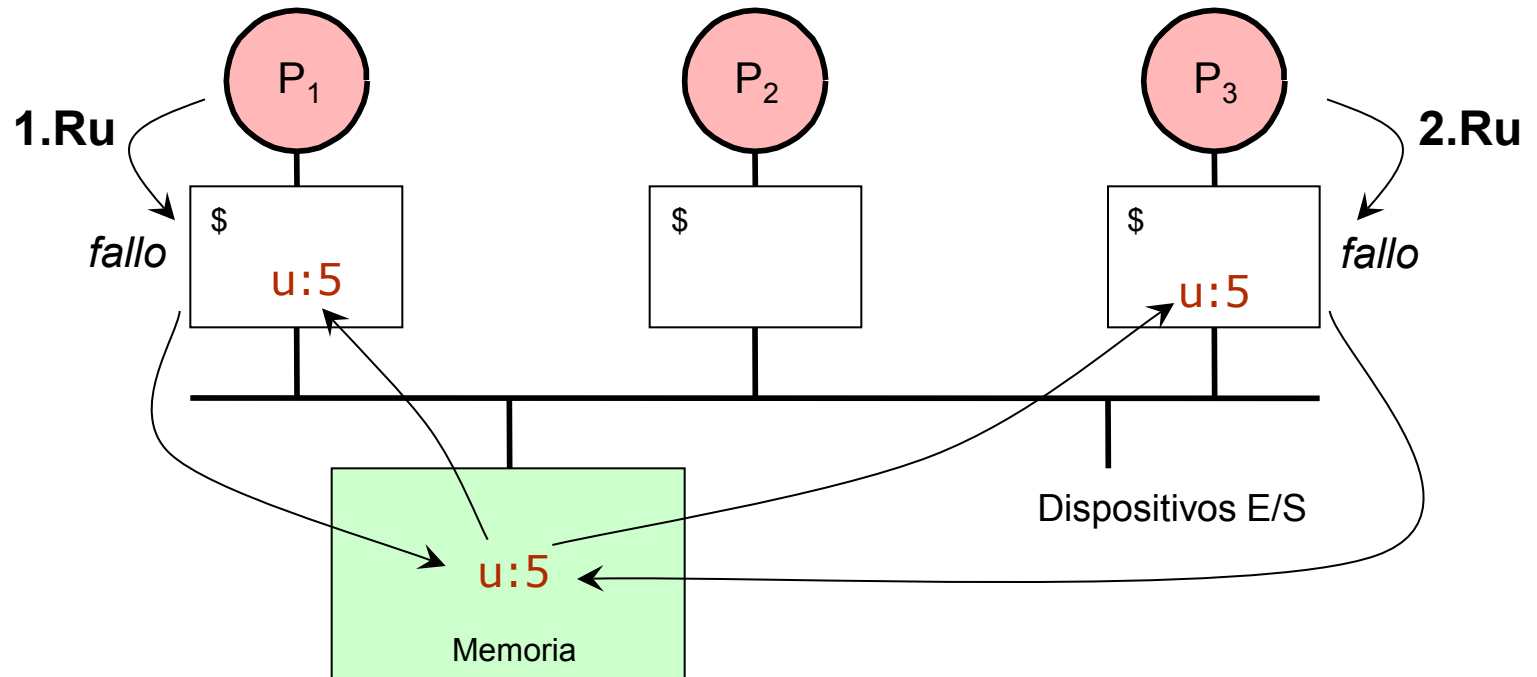
El Problema de la Coherencia Cache



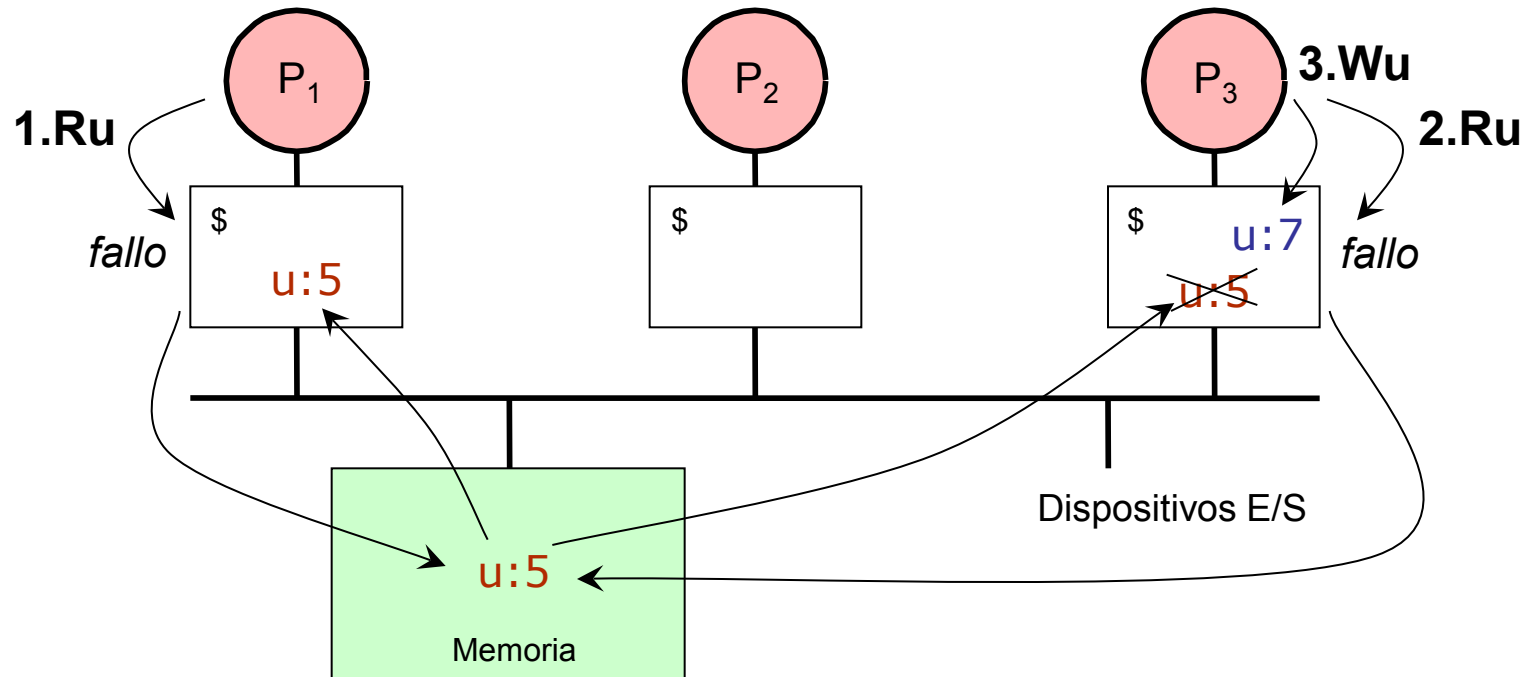
El Problema de la Coherencia Cache



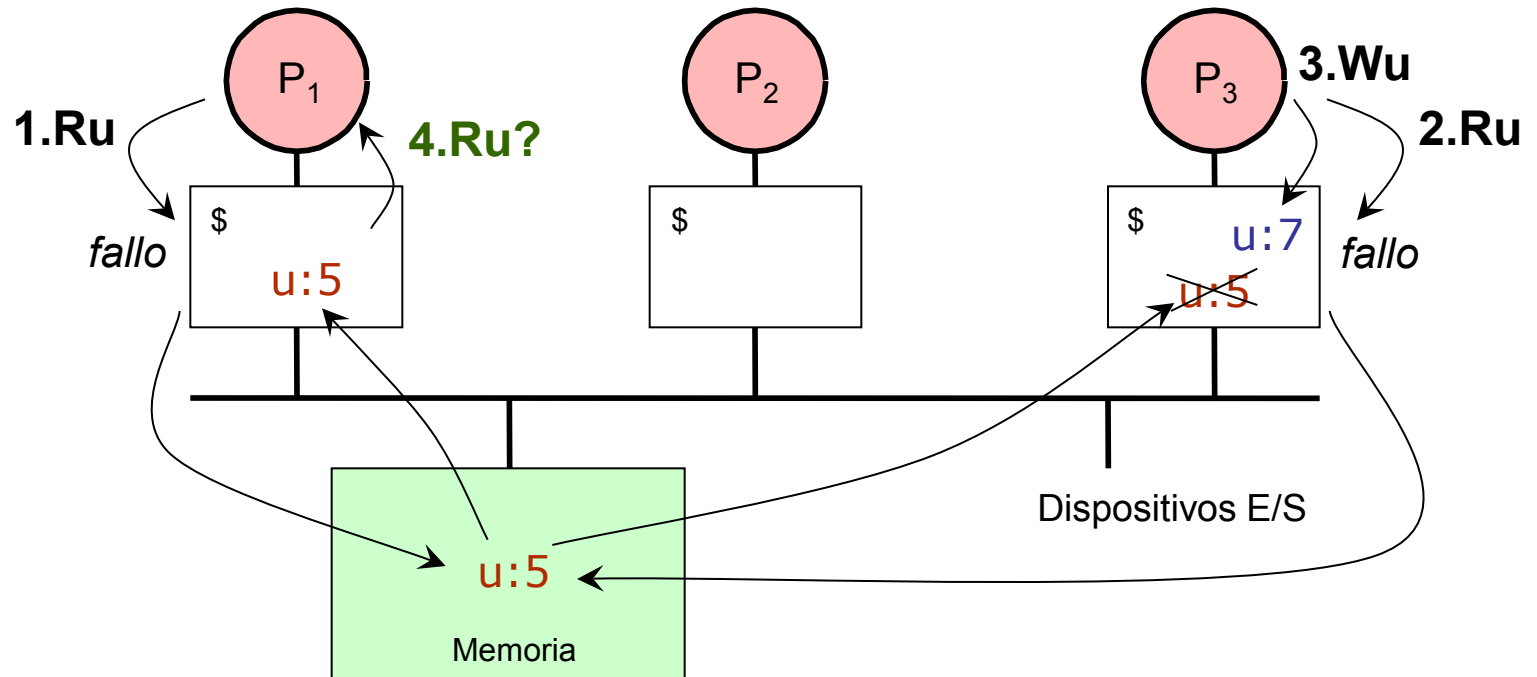
El Problema de la Coherencia Cache



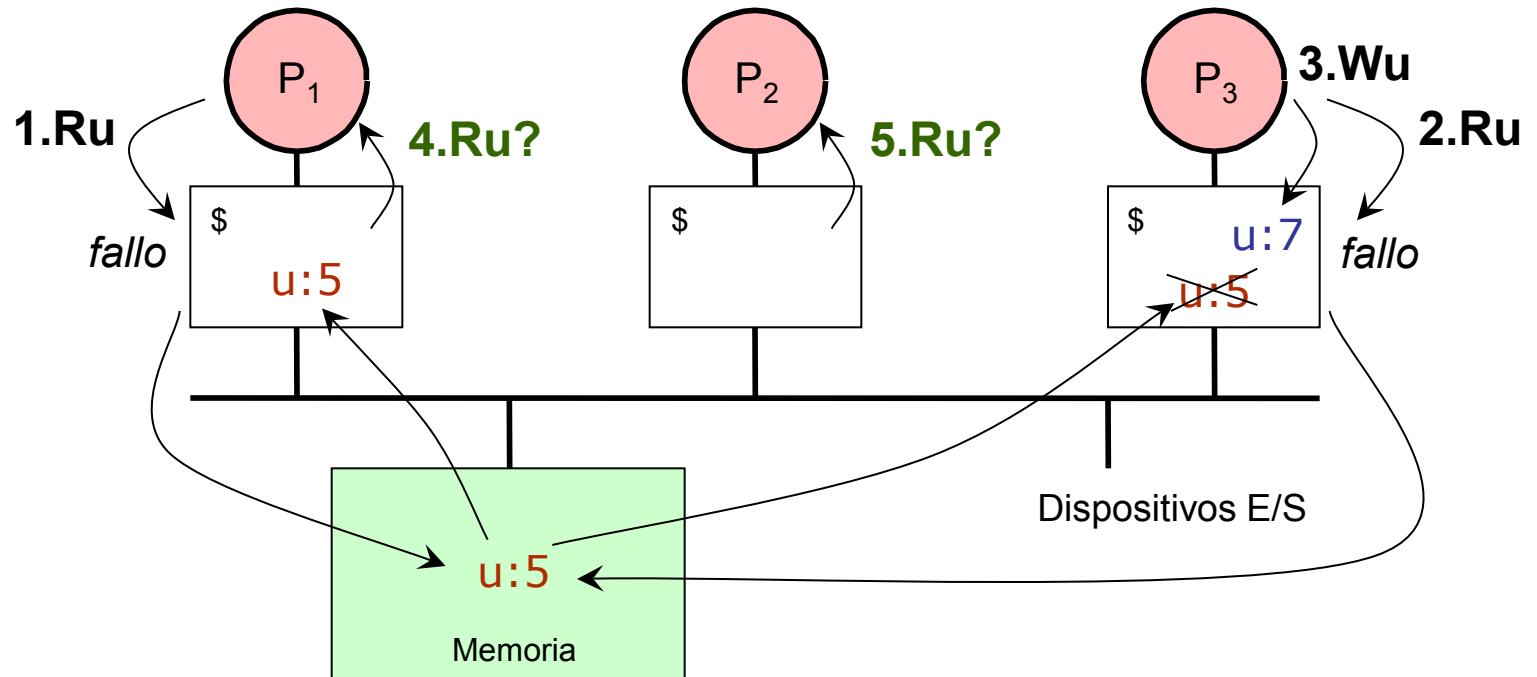
El Problema de la Coherencia Cache



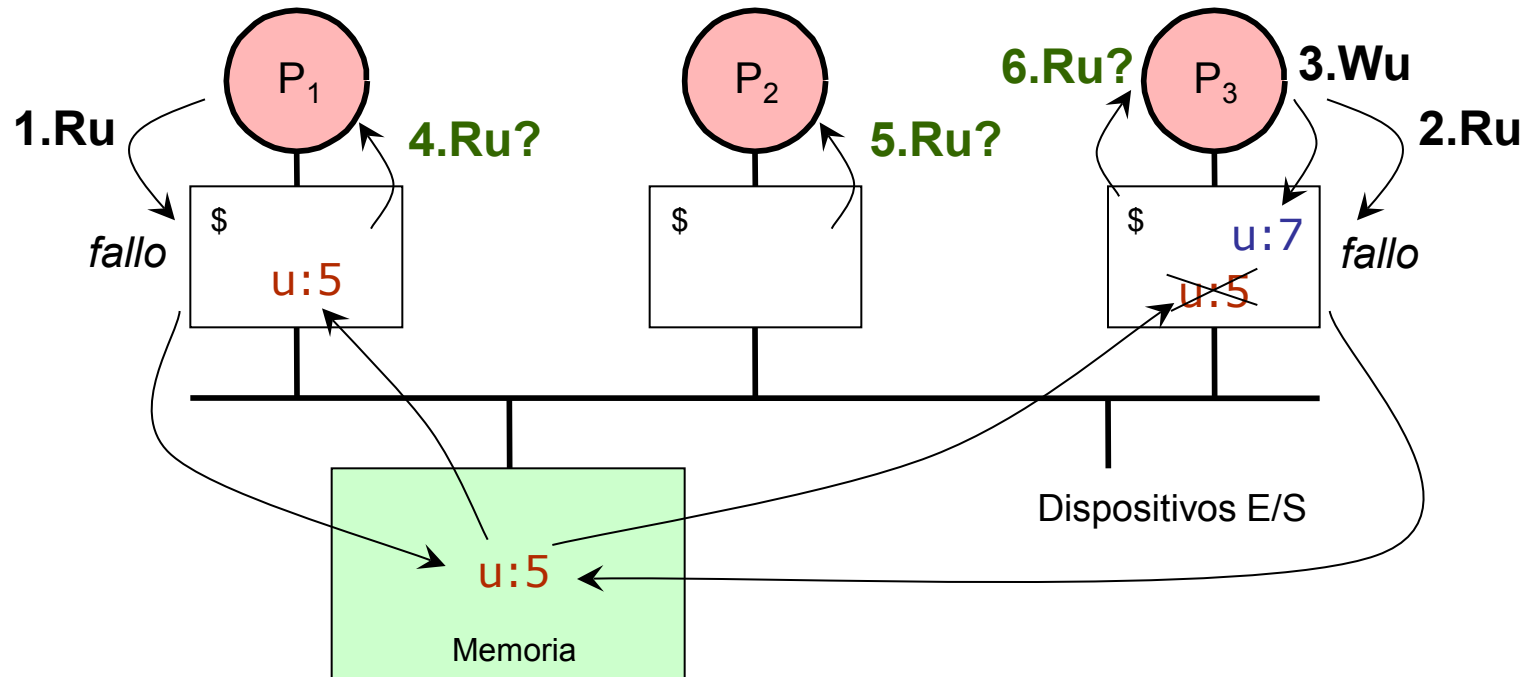
El Problema de la Coherencia Cache



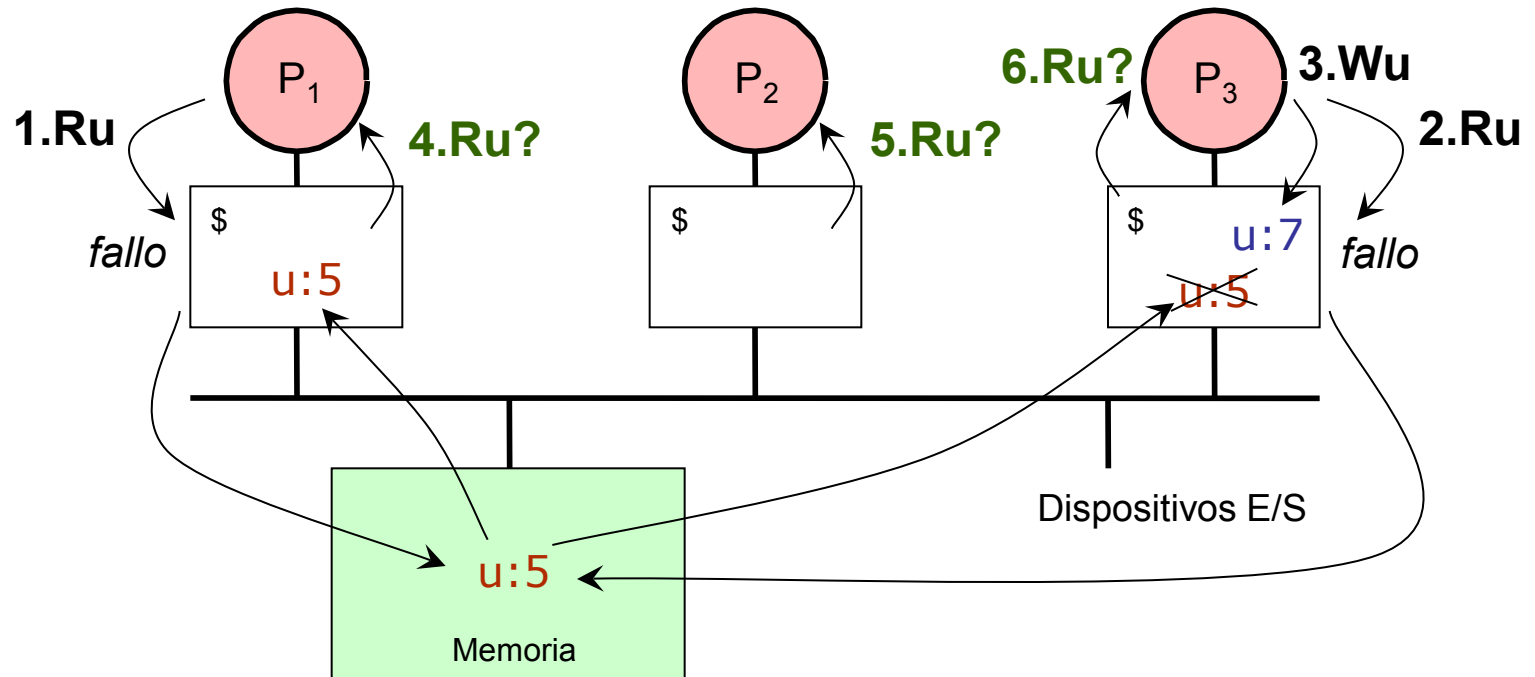
El Problema de la Coherencia Cache



El Problema de la Coherencia Cache



El Problema de la Coherencia Cache



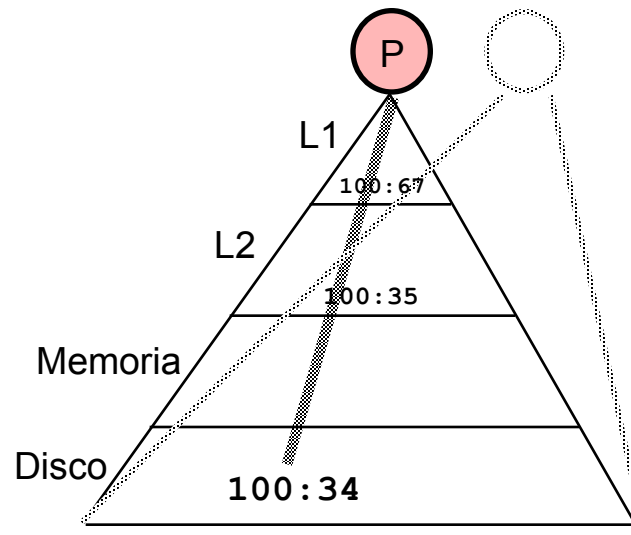
- Los procesadores observan valores diferentes después de la escritura del evento 3
- El comportamiento convencional de las caches *privadas* imposibilita la comunicación entre los procesadores

Caches y Coherencia Cache

- Las caches juegan un papel clave en el acceso a los datos
 - Reducen la latencia media de acceso
 - Reducen el ancho de banda medio en el bus
- Las caches privadas en cada procesador introducen un problema
 - Pueden encontrarse copias de la misma variable en diversas caches
 - Escrituras por un procesador pueden no hacerse visibles al resto de los procesadores (fallo de comunicación)
 - Esto constituye el **problema de la coherencia cache**
- ¿Cómo resolver el problema de coherencia cache?
 - **Organizar la jerarquía de memoria de manera que se elimine el problema**
 - **Mantener la jerarquía de memoria e incorporar mecanismos que detecten y corrijan el problema**

Corregir la Incoherencia Cache

- **Modelo de memoria intuitivo**



- **Comportamiento**

- Leer una posición de memoria debe **devolver el último valor escrito** en esa posición
- Es el modelo natural en uniprocesadores
- En multiprocesadores, el modelo es más complejo

Modelo de Memoria Multiprocesador

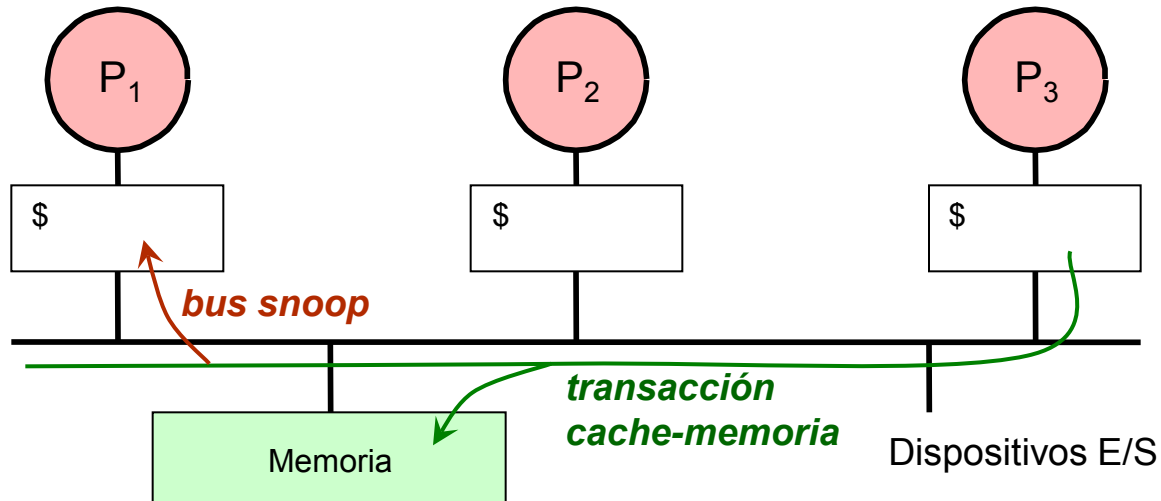
- **Devolver el último valor escrito**
- Implicaciones en un multiprocesador
 - El término “último” no está bien definido
 - El **orden de las operaciones de memoria sobre una posición concreta** dentro de un proceso es igual al caso uniprocesador, pero no es así cuando consideramos procesos concurrentes diferentes
- Se debe definir el significado de orden de operaciones de memoria sobre una posición concreta entre procesos concurrentes
- Debemos imponer un orden **serializado** y **global** a todas las operaciones sobre una posición de memoria concreta

Devolver el Ultimo Valor Escrito

- “Ultimo” está ahora definido por el orden serializado y global impuesto por la memoria
- Implicaciones del orden serializado y global
 - Todas las operaciones de escritura sobre una posición concreta deben hacerse visibles a todos los procesadores que lo requieran (es decir, que lo vayan a leer) y en el mismo orden para todos ellos
 - Es decir, las escrituras deben **propagarse** en un **orden global** a todos los procesadores que lo requieran (esto es, **las escrituras implicadas en una comunicación o sincronización**)
 - Las lecturas también están en ese orden global, pero en la práctica resulta irrelevante el orden relativo entre ellas (específicamente, entre todas las lecturas que aparecen entre dos escrituras consecutivas)
 - Sin embargo, las **lecturas deben leer siempre el último valor escrito**
- El orden global de las operaciones de memoria es hipotético
 - El hardware **sólo** debe imponer una propagación en un orden global de las operaciones de escritura que implican comunicación o sincronización
 - Las operaciones restantes pueden ser concurrentes

Coherencia en Bus Compartido (SMP)

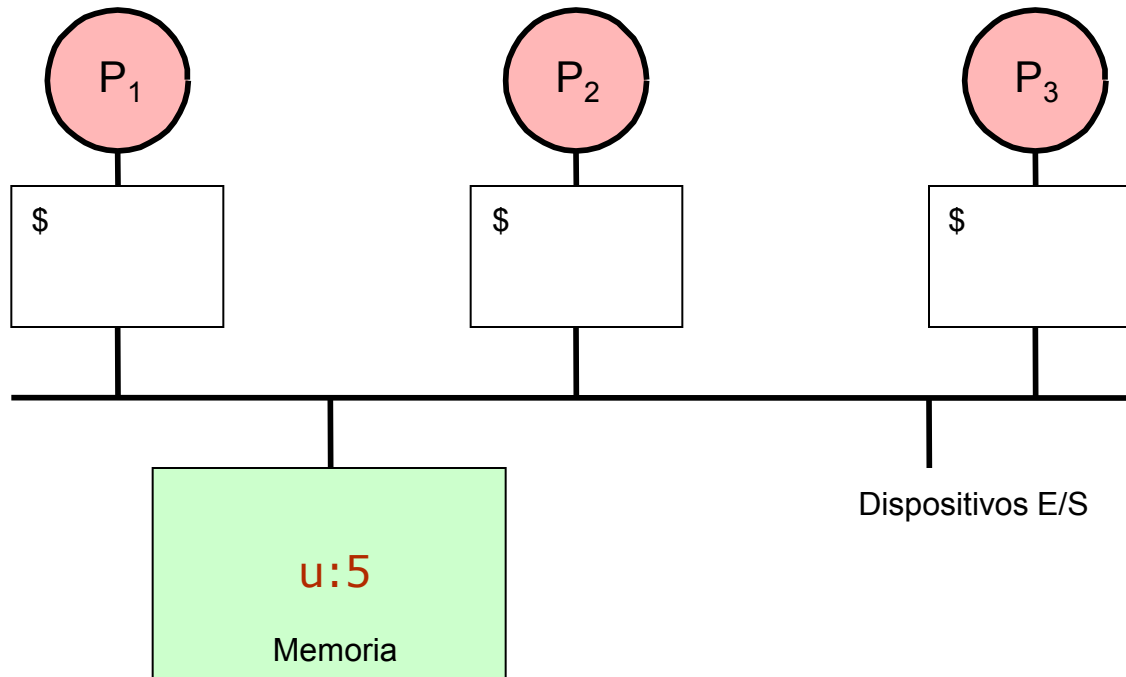
- **Devolver el último valor escrito implica que las escrituras deben propagarse para ser accesibles por cualquier lector**



- El bus es un dispositivo compartido al que acceden todas las caches
- Las caches espían (*snoop*) todas las transacciones de bus
 - Frente a una transacción relevante (afecte a un bloque que posee), realiza una acción concreta que asegure la coherencia
 - » Invalidación, actualización, proveer valor, ...

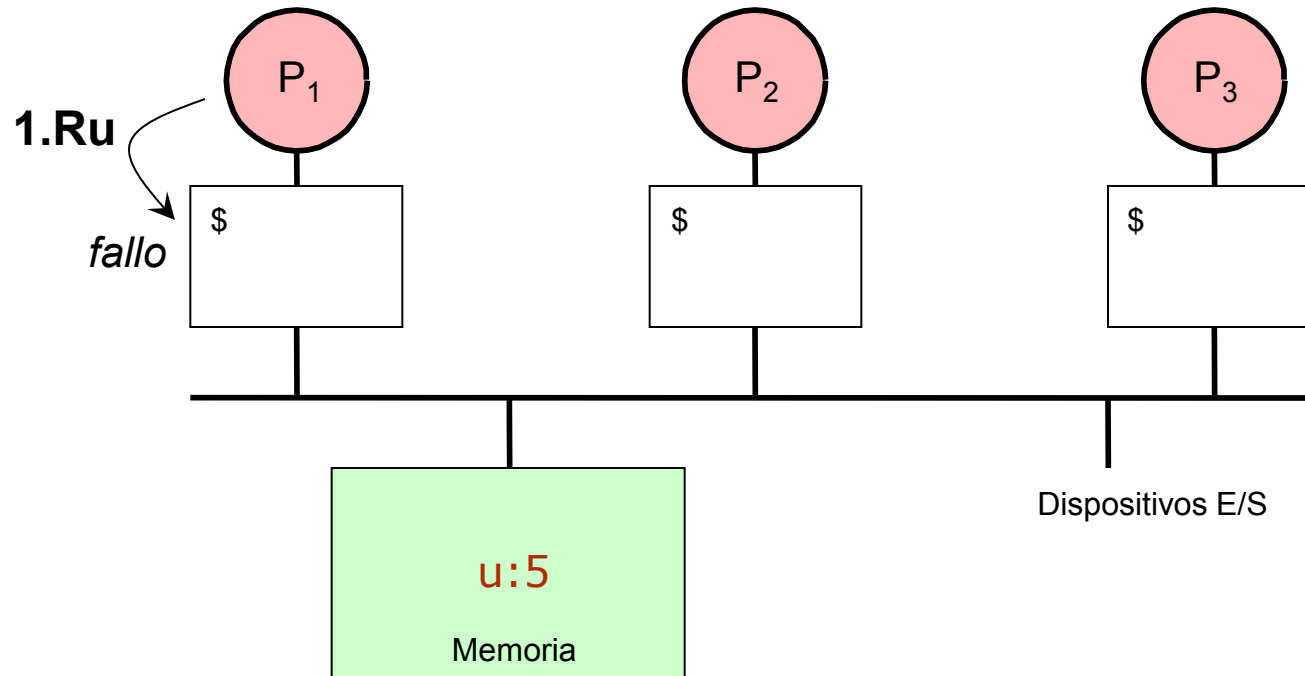
Ejemplo

- Cache de Escritura Directa e Invalidación



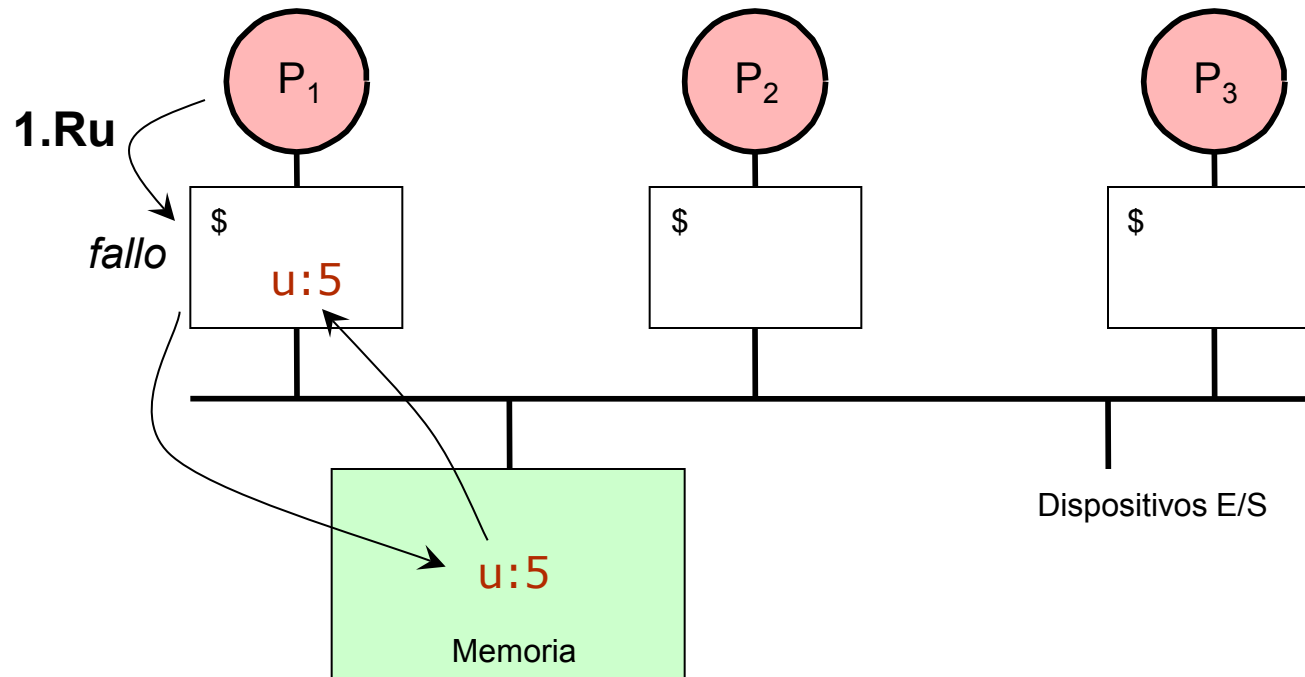
Ejemplo

- Cache de Escritura Directa e Invalidación



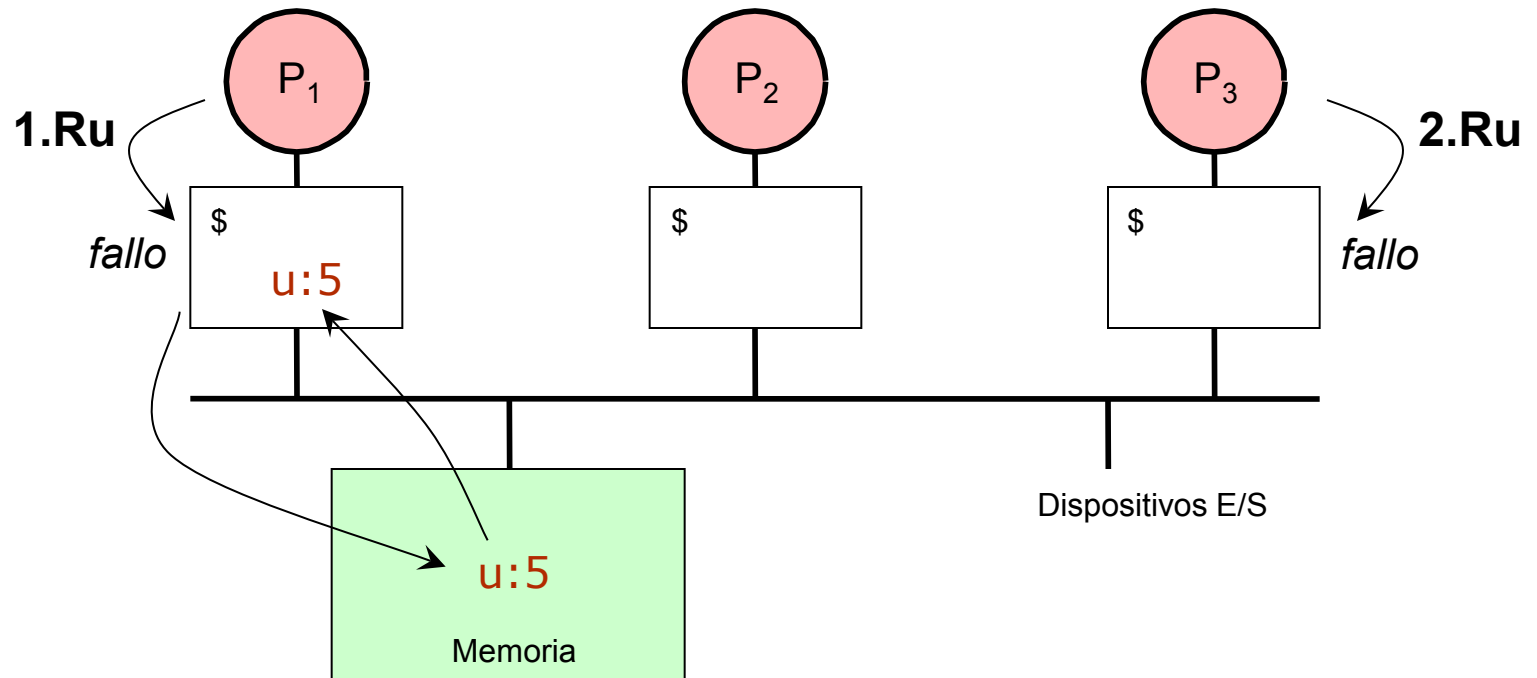
Ejemplo

- Cache de Escritura Directa e Invalidación



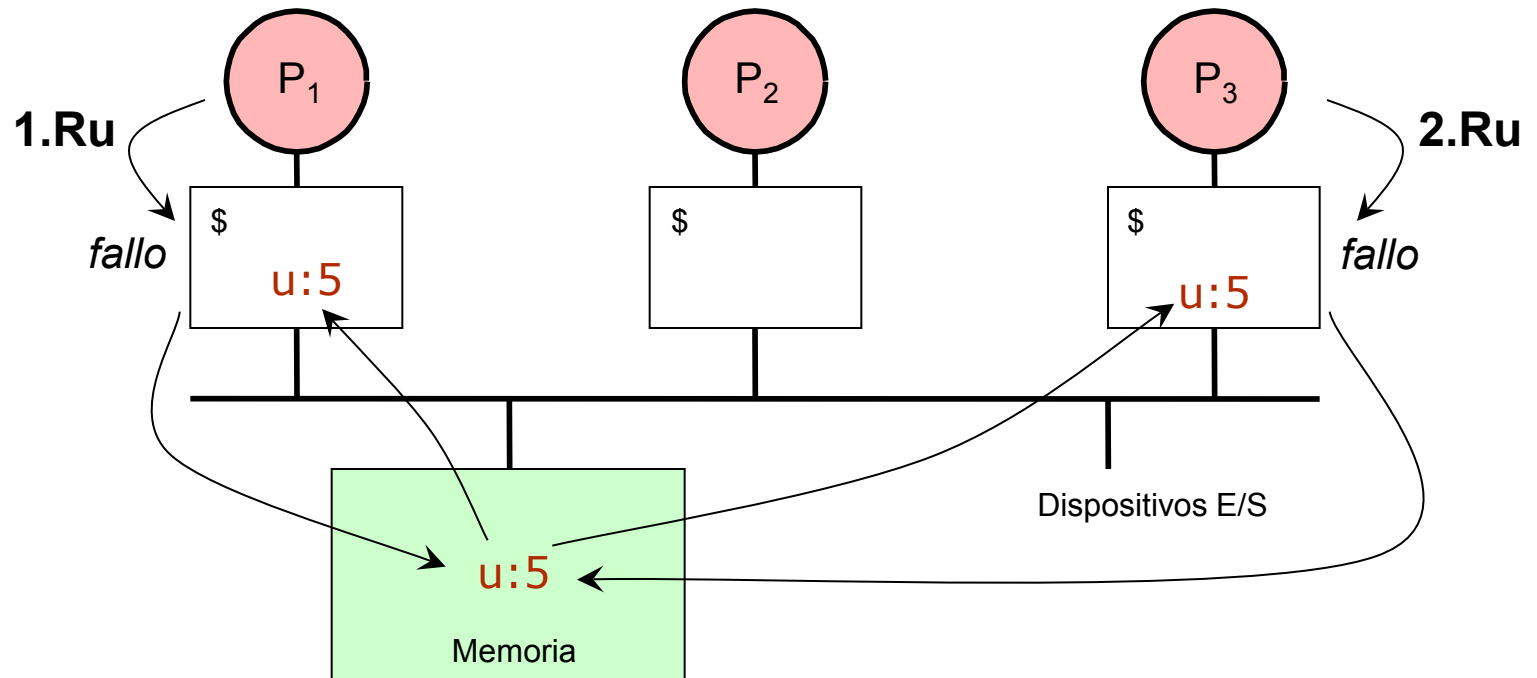
Ejemplo

- Cache de Escritura Directa e Invalidación



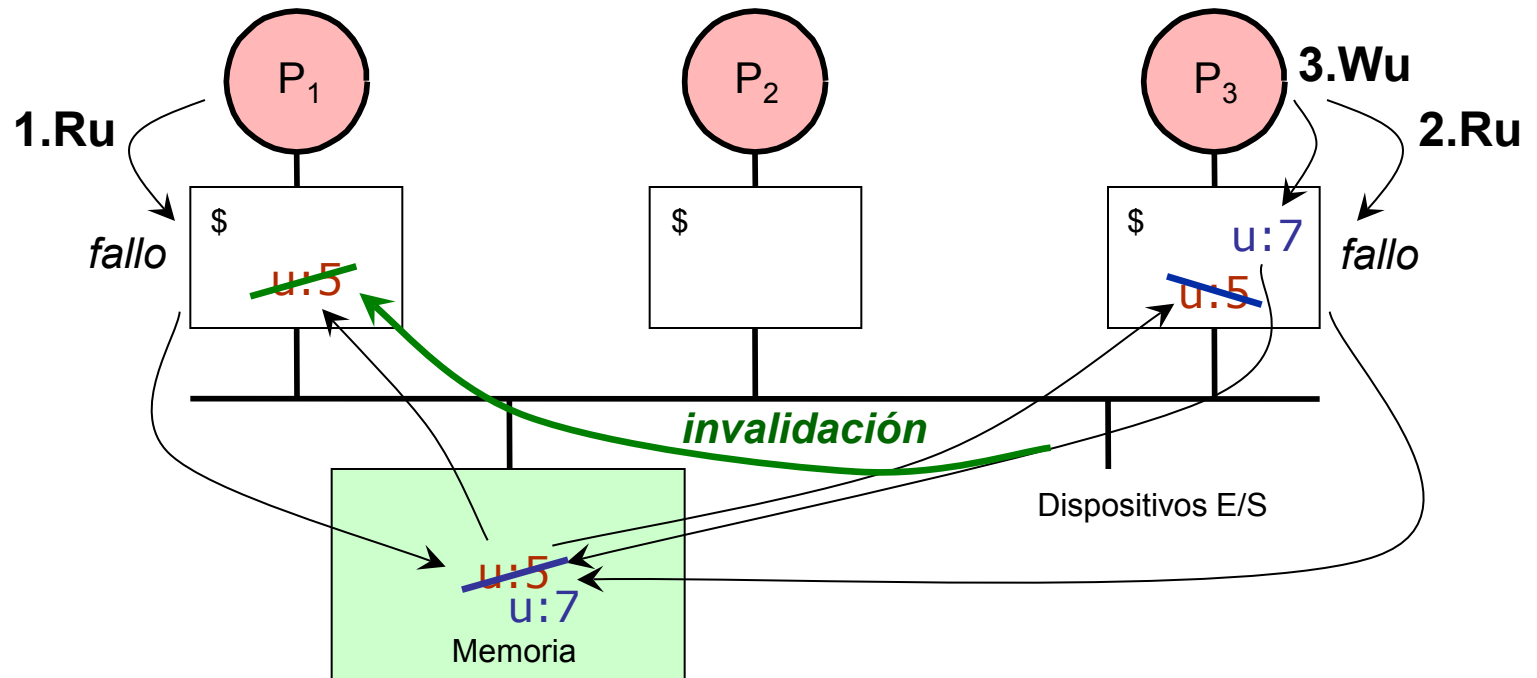
Ejemplo

- Cache de Escritura Directa e Invalidación



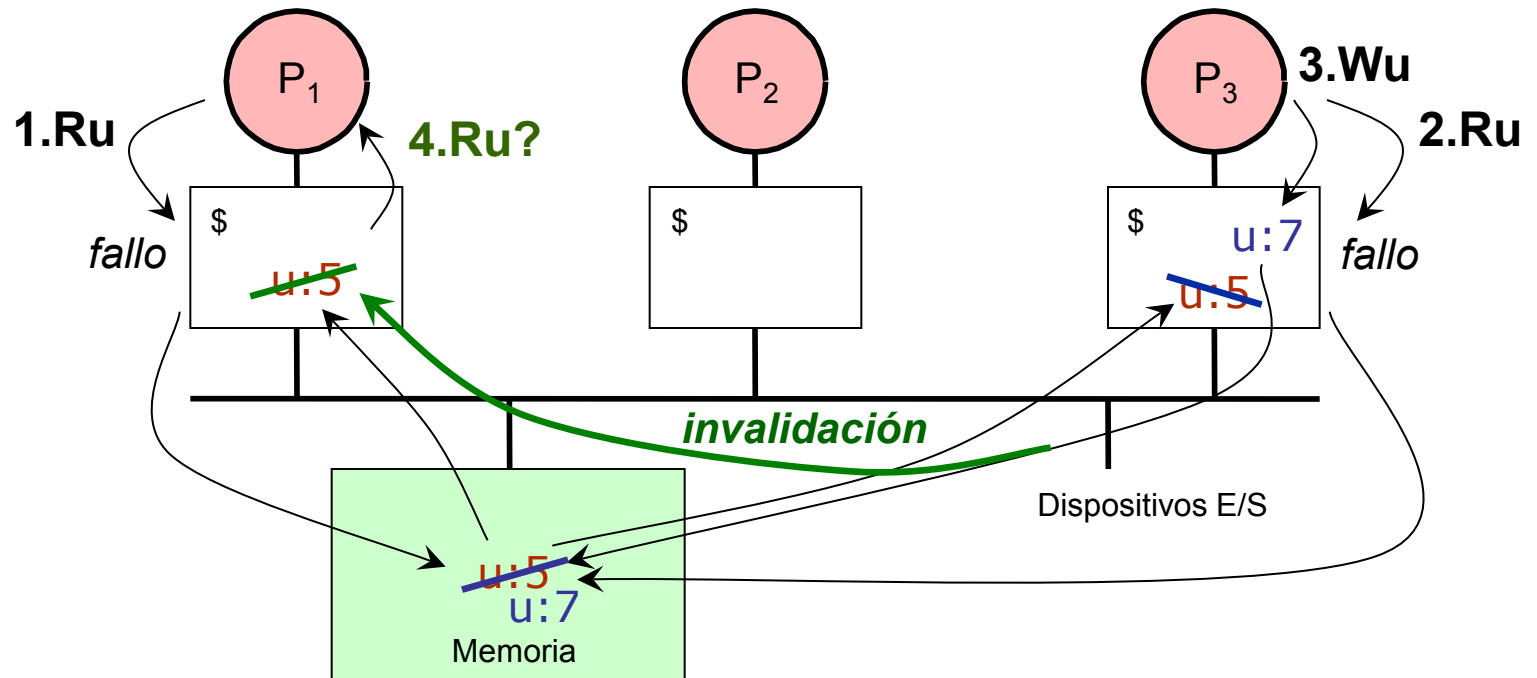
Ejemplo

- Cache de Escritura Directa e Invalidación



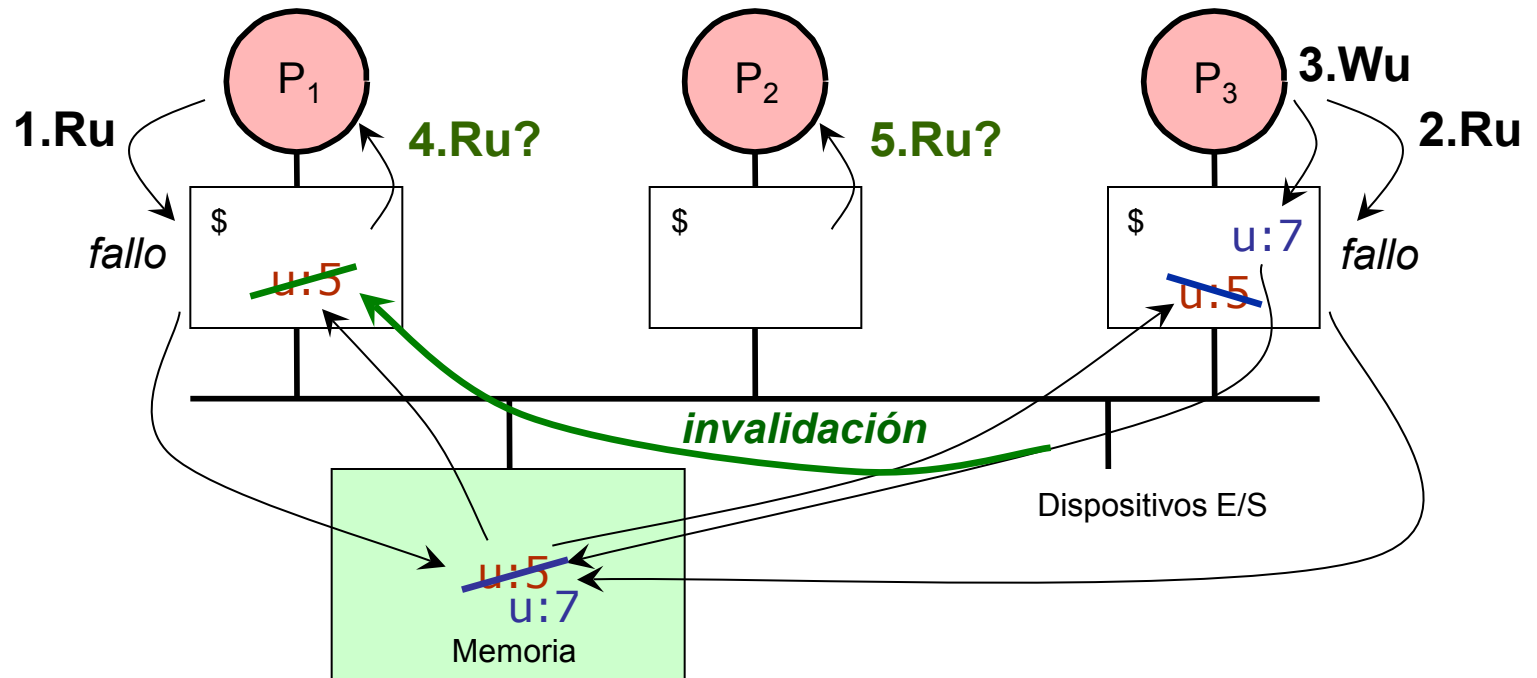
Ejemplo

- Cache de Escritura Directa e Invalidación

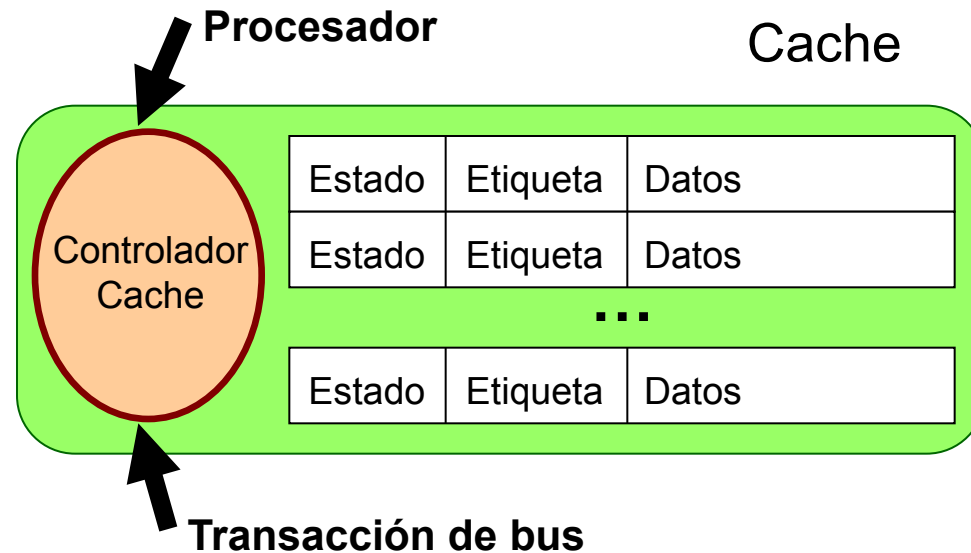


Ejemplo

- Cache de Escritura Directa e Invalidación



Control de Coherencia: Arquitectura

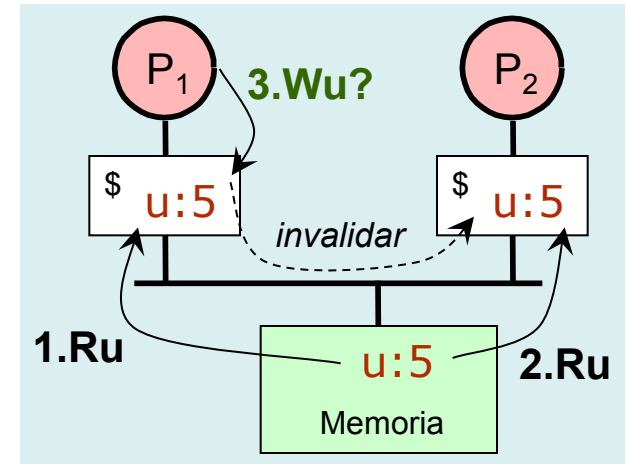


- El controlador cache actualiza el estado de los bloques en respuesta a los eventos de procesador y bus (*snoop*), y genera transacciones de bus
- Protocolo de coherencia
 - Conjunto de estados de los bloques cache
 - Diagrama de transición de estados

Protocolos de Coherencia Cache

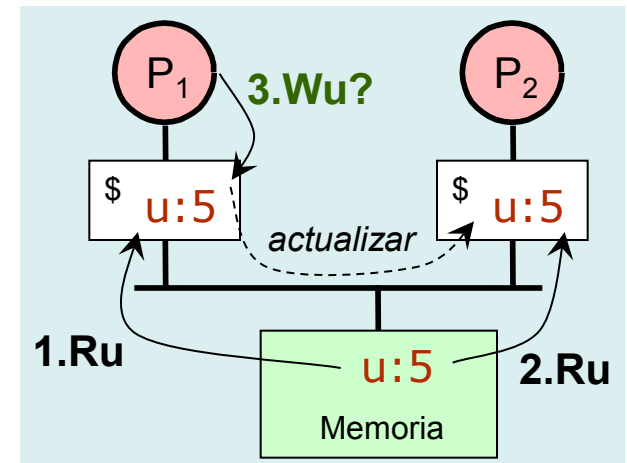
• Invalidar

- Una escritura provoca una situación de exclusividad en el bloque cache
 - » Otra escritura local no provoca tráfico en el bus
 - » Las escrituras se propagan cuando hay un fallo cache en el sistema
 - » La escritura se termina cuando se actualiza la cache local



• Actualizar

- Una escritura actualiza las copias del mismo bloque en otras caches
 - » No se producen fallos cache cuando se accede de nuevo a ese bloque
 - » Una sola transacción de bus propaga una escritura a varias caches
 - » Múltiples escrituras provoca múltiples actualizaciones



Protocolo de Invalidación MESI

- **Cache con post-escritura**

- **Suposiciones**

- Las operaciones de memoria y las transacciones en el bus son **atómicas**

PrRd: Lectura originada en el procesador

PrWr: Escritura originada en el procesador

Reemplazo: Reemplazo del bloque cache

BusRd: Lectura en el bus

BusRd(S): Lectura en el bus (S a 1)

BusRd(!S): Lectura en el bus (S a 0)

BusRdX: Lectura en el bus con exclusividad

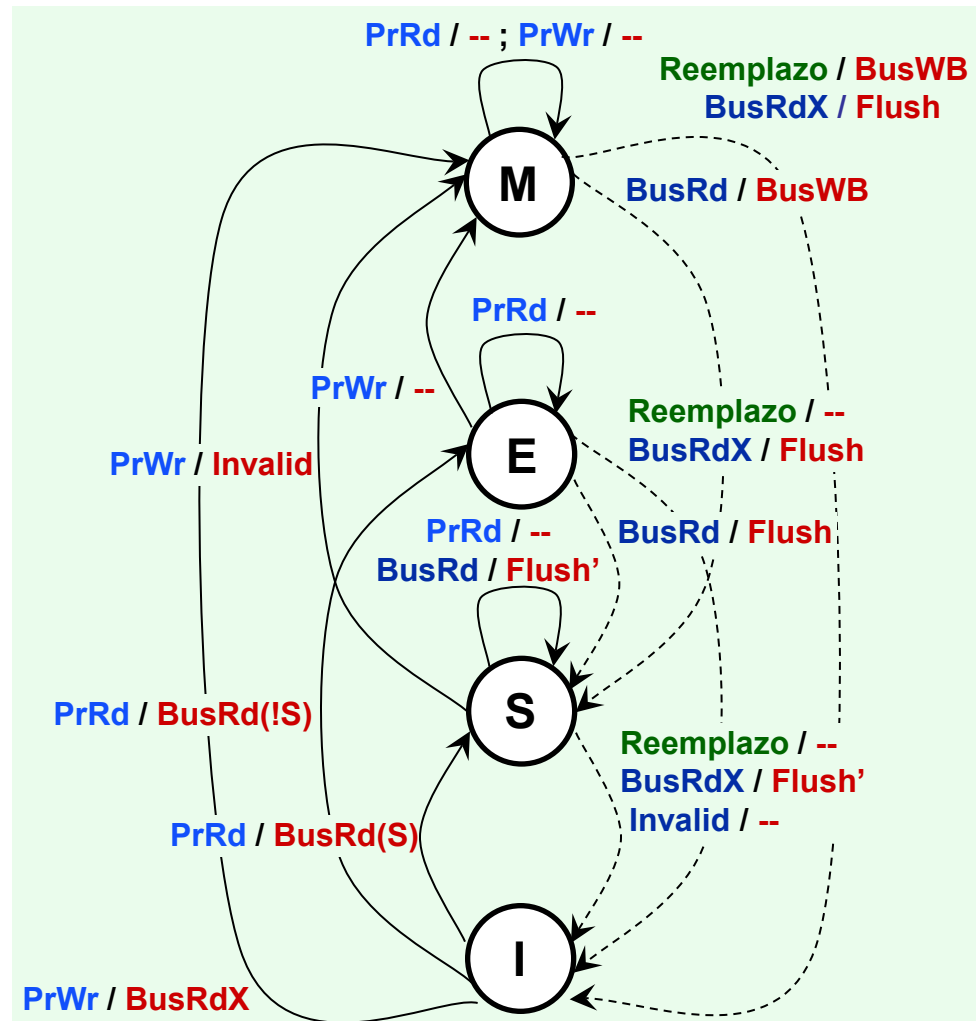
Invalid: Señal de invalidación en el bus

BusWB: Actualización de memoria

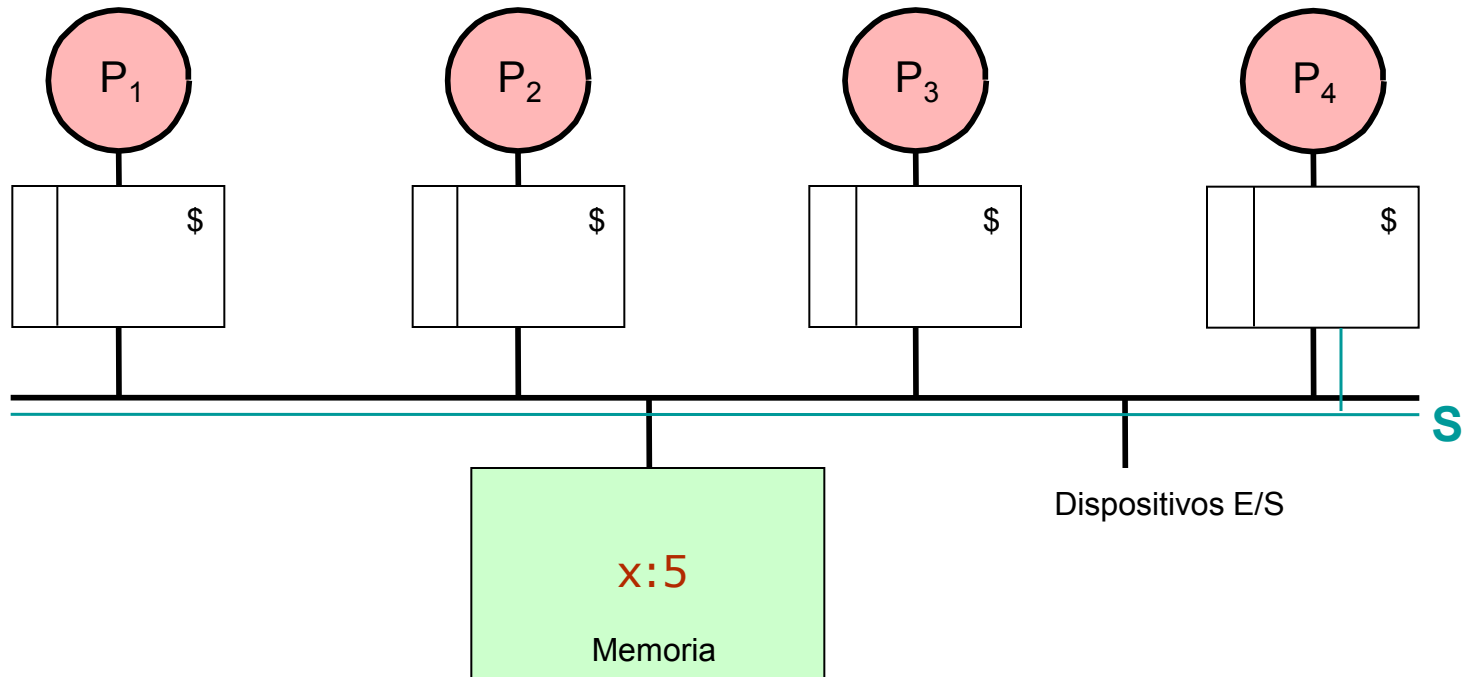
Flush: Pone bloque cache en el bus

Flush': Flush, pero sólo una cache

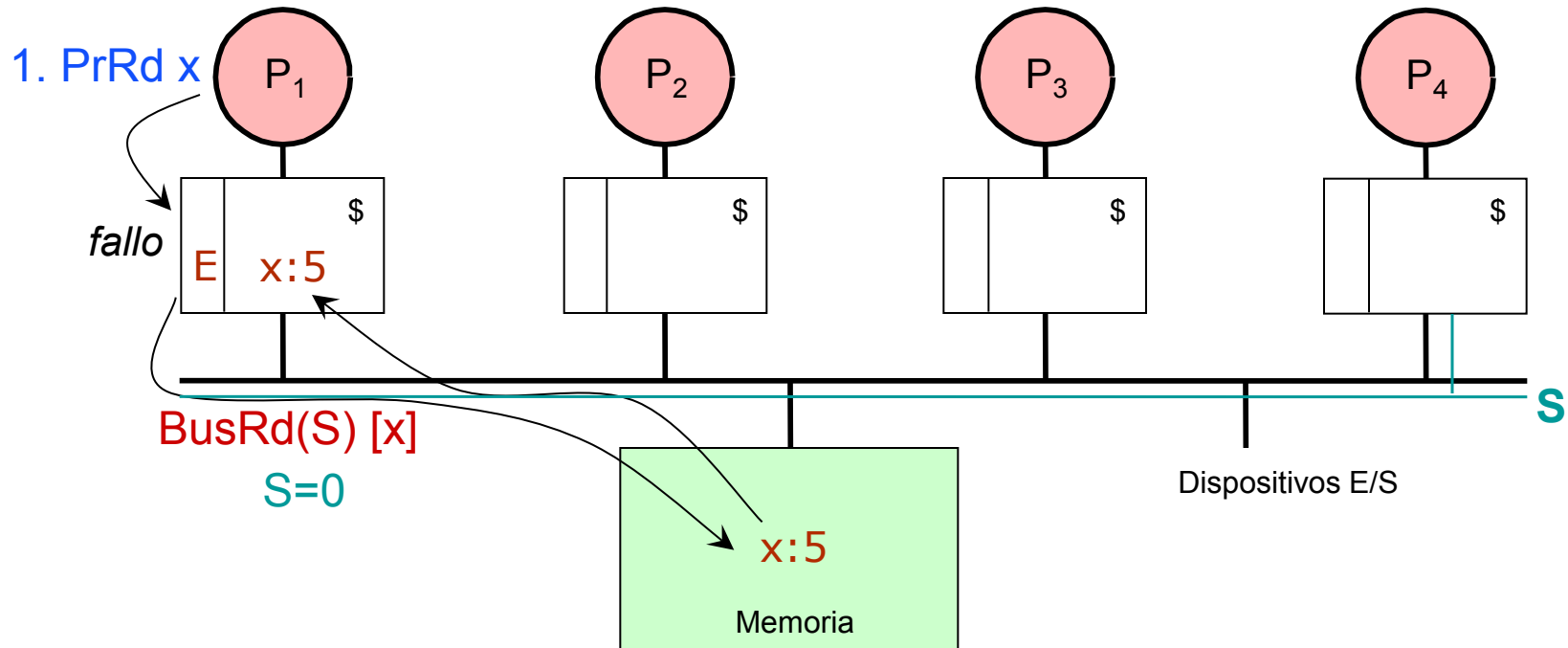
Protocolo MESI



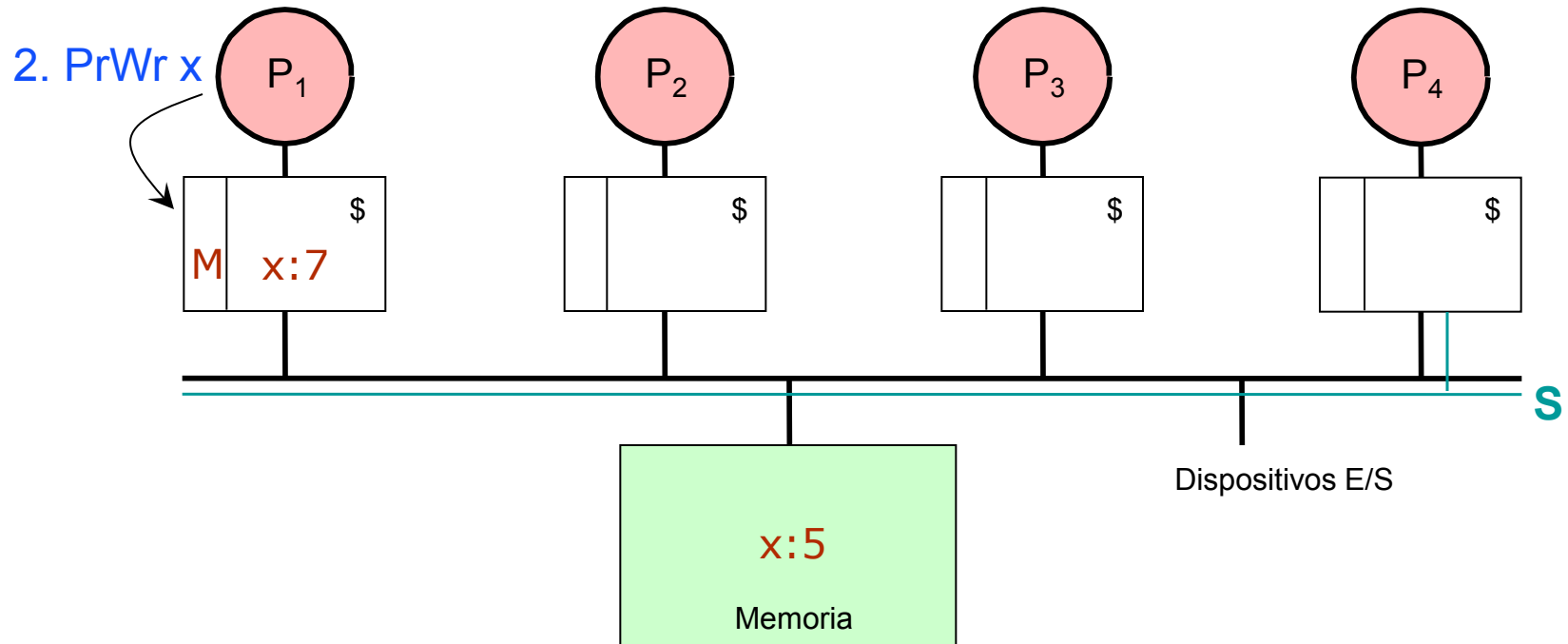
Protocolo MESI: Ejemplo



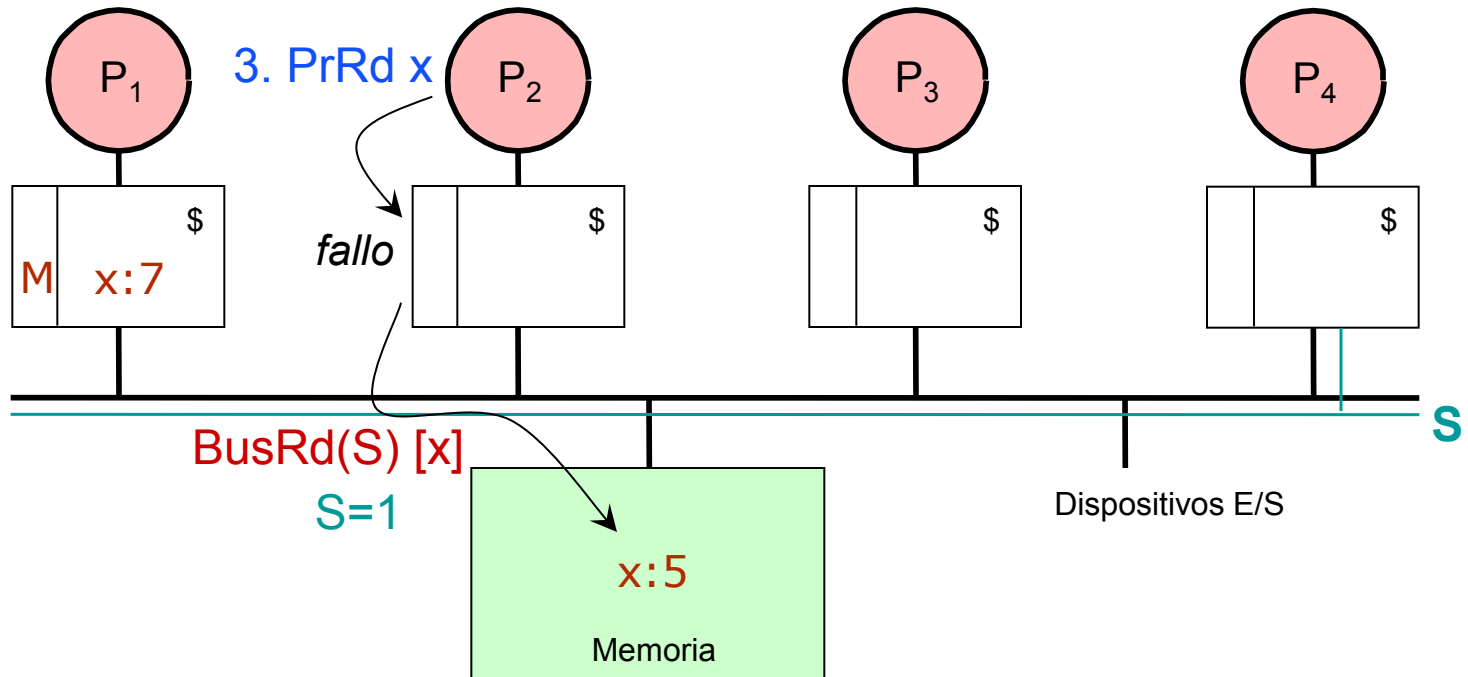
Protocolo MESI: Ejemplo



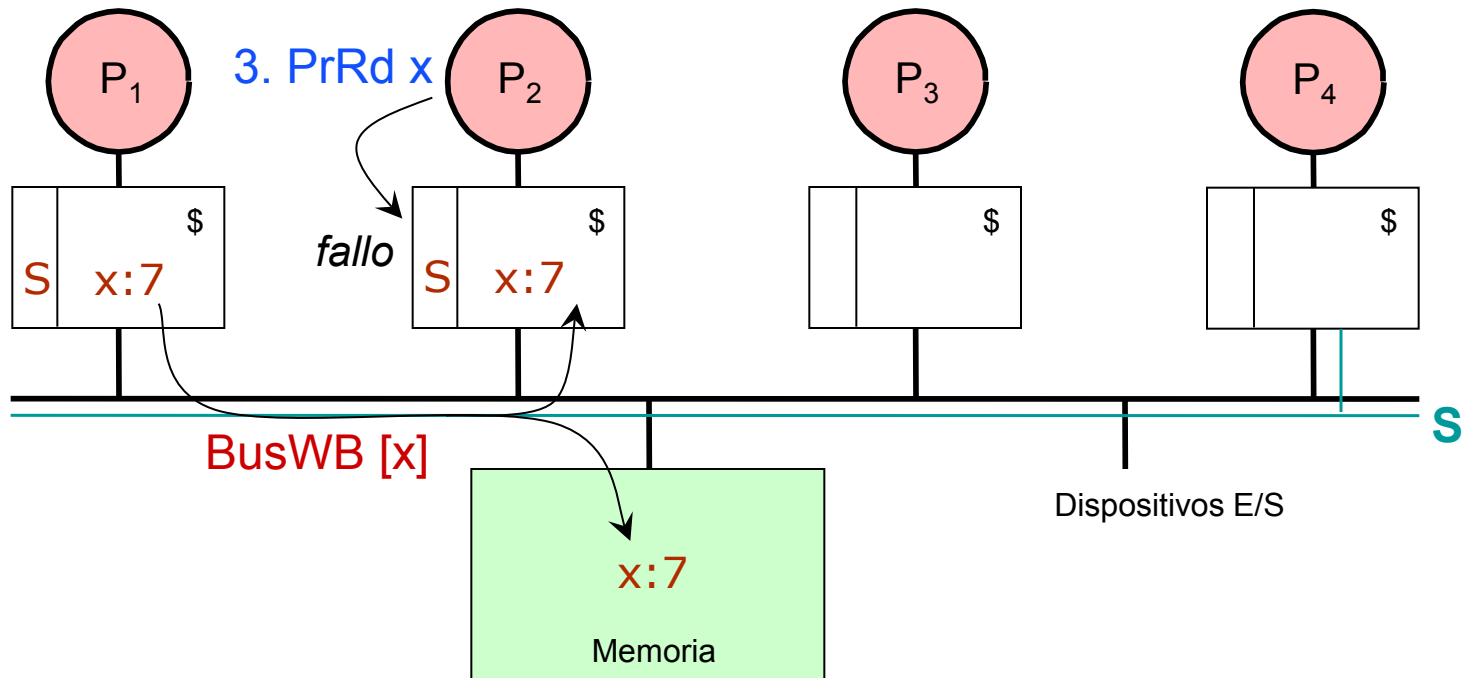
Protocolo MESI: Ejemplo



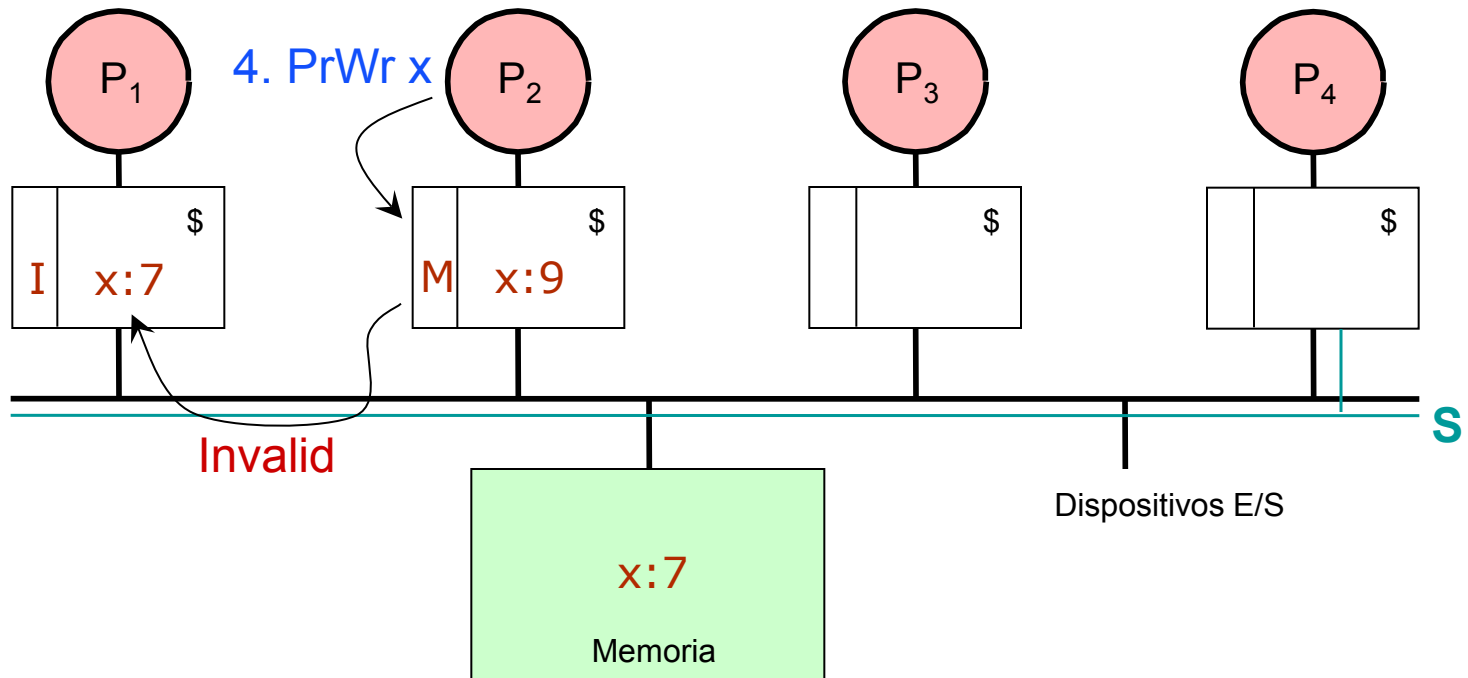
Protocolo MESI: Ejemplo



Protocolo MESI: Ejemplo



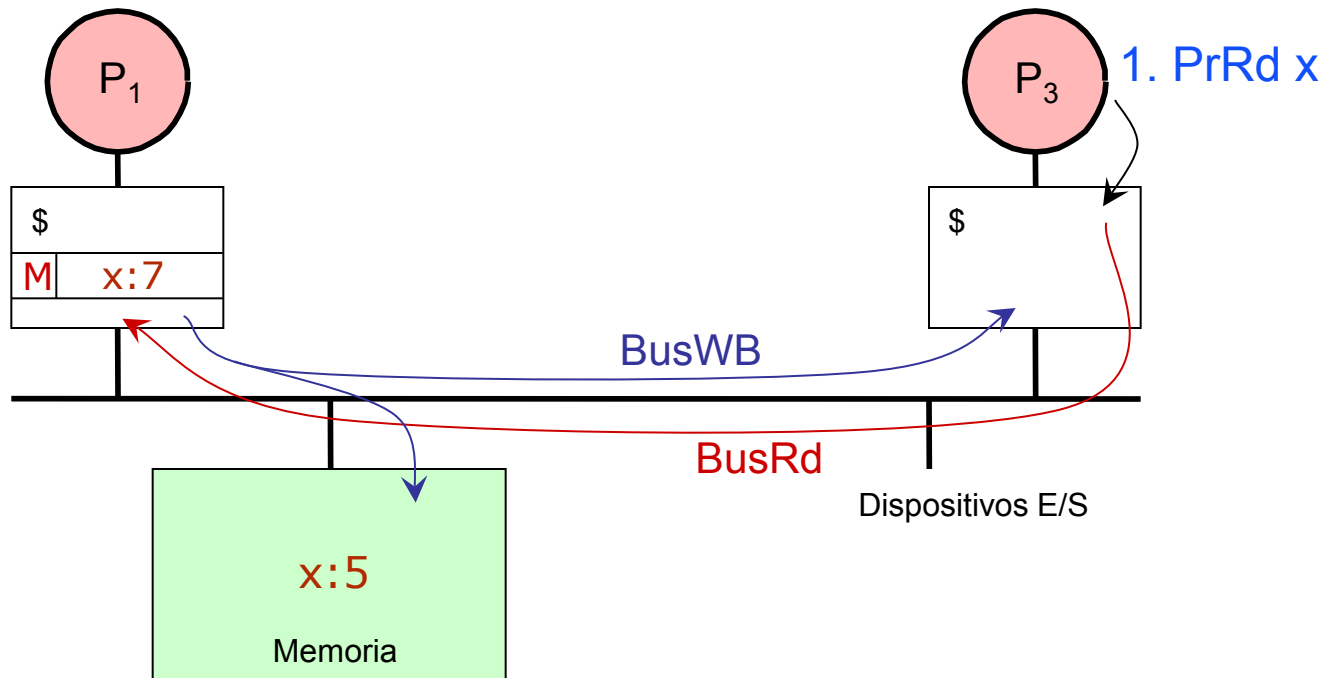
Protocolo MESI: Ejemplo



Protocolo MESI: Ineficiencia

- Actualización de memoria

- La memoria debería actualizarse sólo cuando desaparecen las copias de un bloque en todas las caches del sistema
- Sin embargo, el protocolo MESI provoca una actualización (**BusWB**) cuando algún procesador lee un bloque modificado por otro: Esta actualización es realmente **innecesaria** (sin embargo, hay que hacerlo, pues el estado S implica que la memoria está actualizada)



Protocolo MOESI

- Eliminación de las actualizaciones de memoria innecesarias

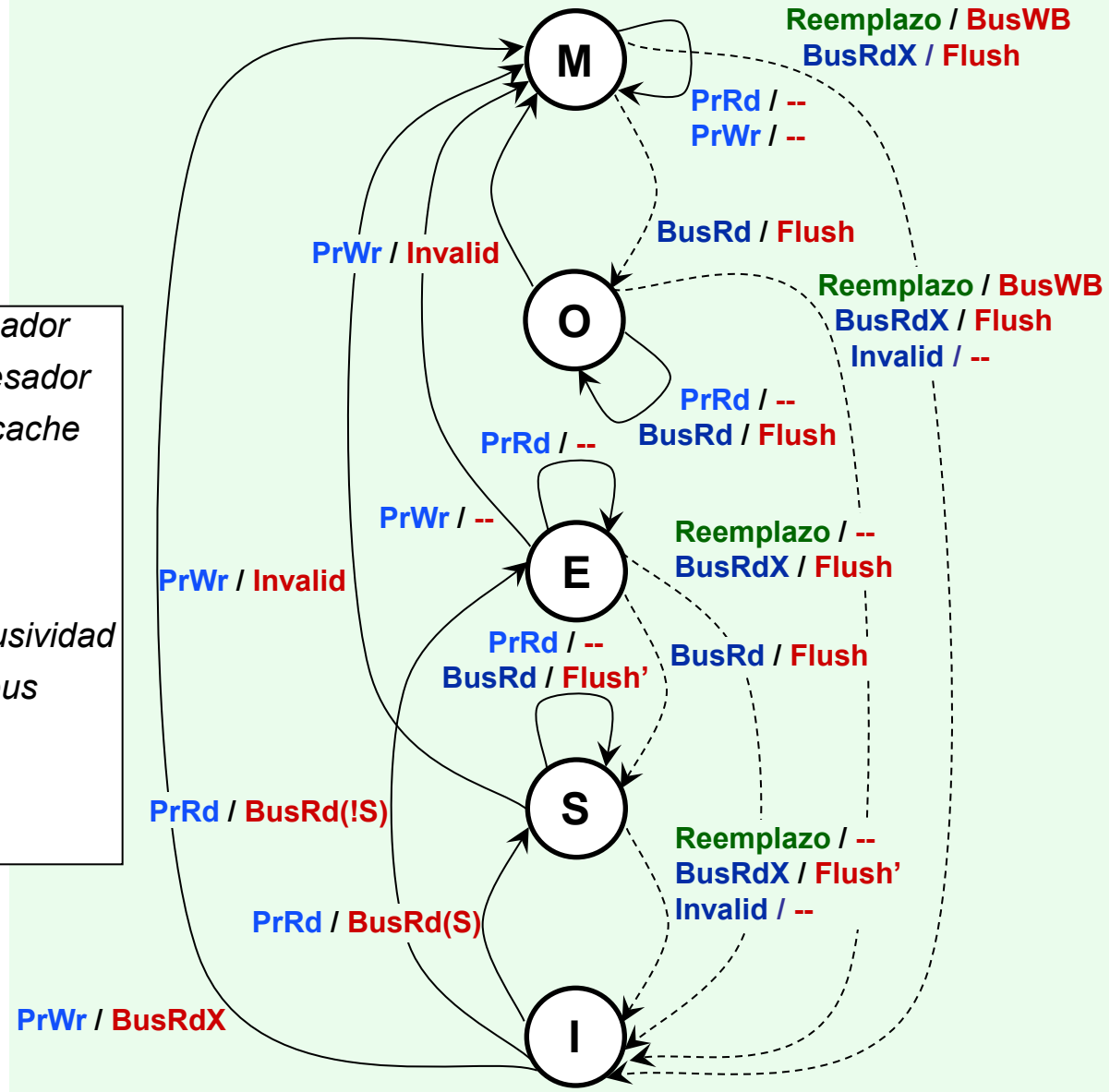
- Definiendo un nuevo estado, O (propietario), es posible eliminar las actualizaciones de memoria innecesarias

- Protocolo MOESI

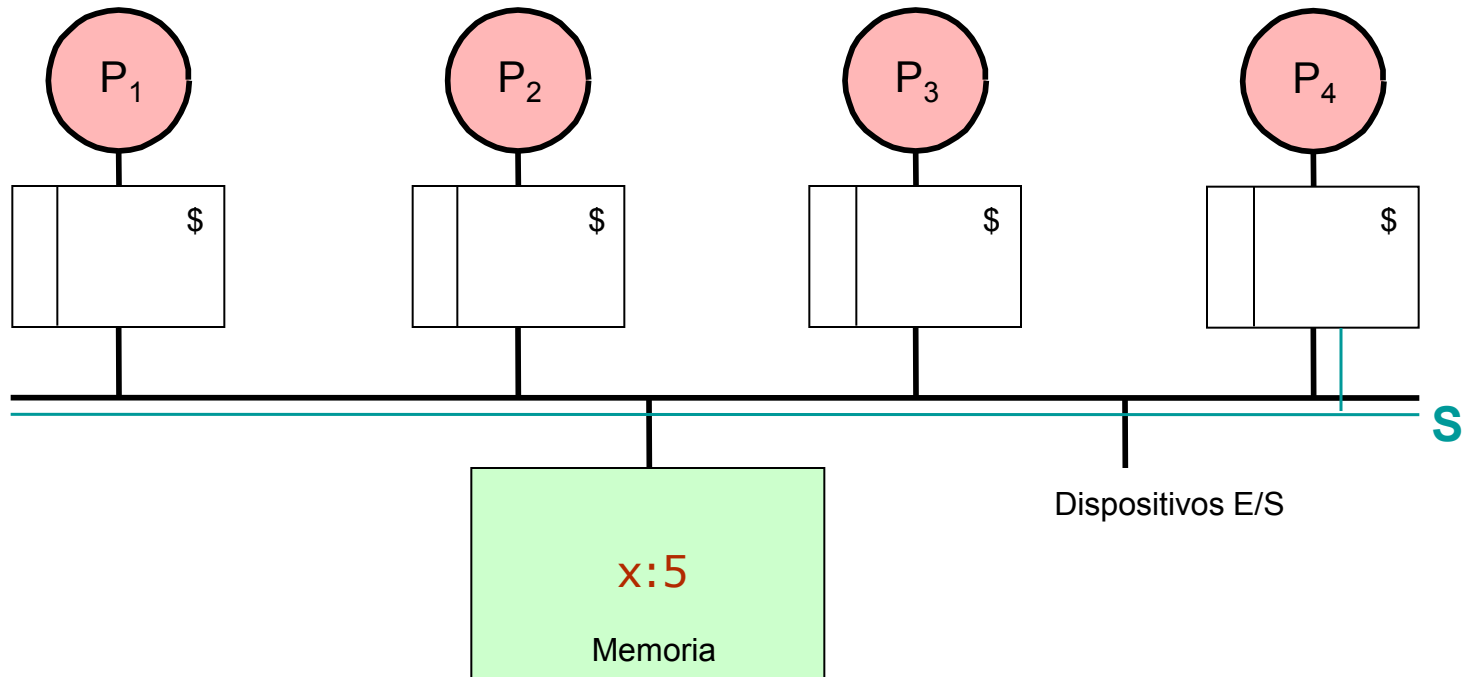
- Cinco estados para cada bloque cache
 - » **I**: estado inválido
 - » **S**: estado compartido (posiblemente, dos o más caches tienen copia y la memoria puede ser coherente o no)
 - » **E**: estado exclusivo no modificado (ninguna otra cache tiene copia y la memoria es coherente)
 - » **O**: estado propietario (posiblemente, dos o más caches tienen copia en estado S y la memoria es incoherente)
 - » **M**: estado exclusivo modificado (ninguna otra cache tiene copia y la memoria es incoherente)

Protocolo MOESI

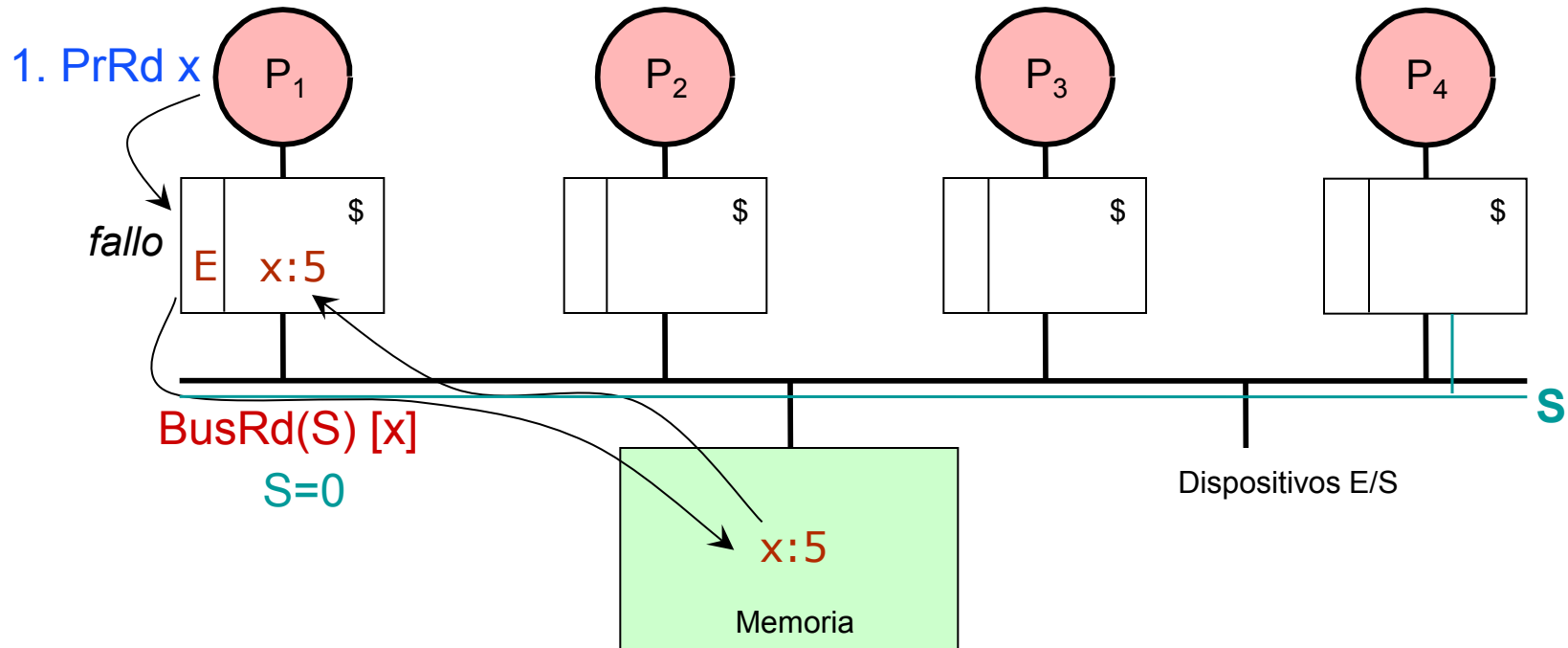
PrRd: Lectura originada en el procesador
PrWr: Escritura originada en el procesador
Reemplazo: Reemplazo del bloque cache
BusRd: Lectura en el bus
BusRd(S): Lectura en el bus (S a 1)
BusRd(!S): Lectura en el bus (S a 0)
BusRdX: Lectura en el bus con exclusividad
Invalid: Señal de invalidación en el bus
BusWB: Actualización de memoria
Flush: Pone bloque cache en el bus
Flush': Flush, pero sólo una cache



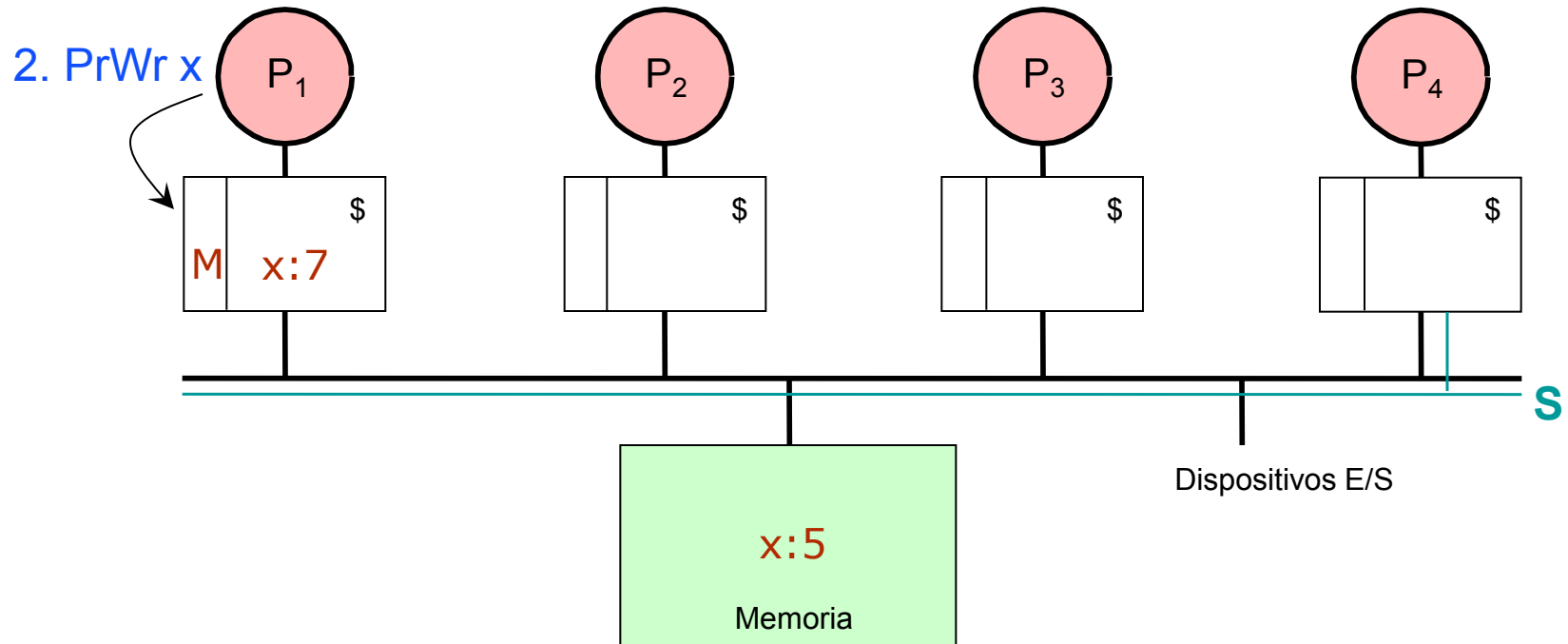
Protocolo MOESI: Ejemplo



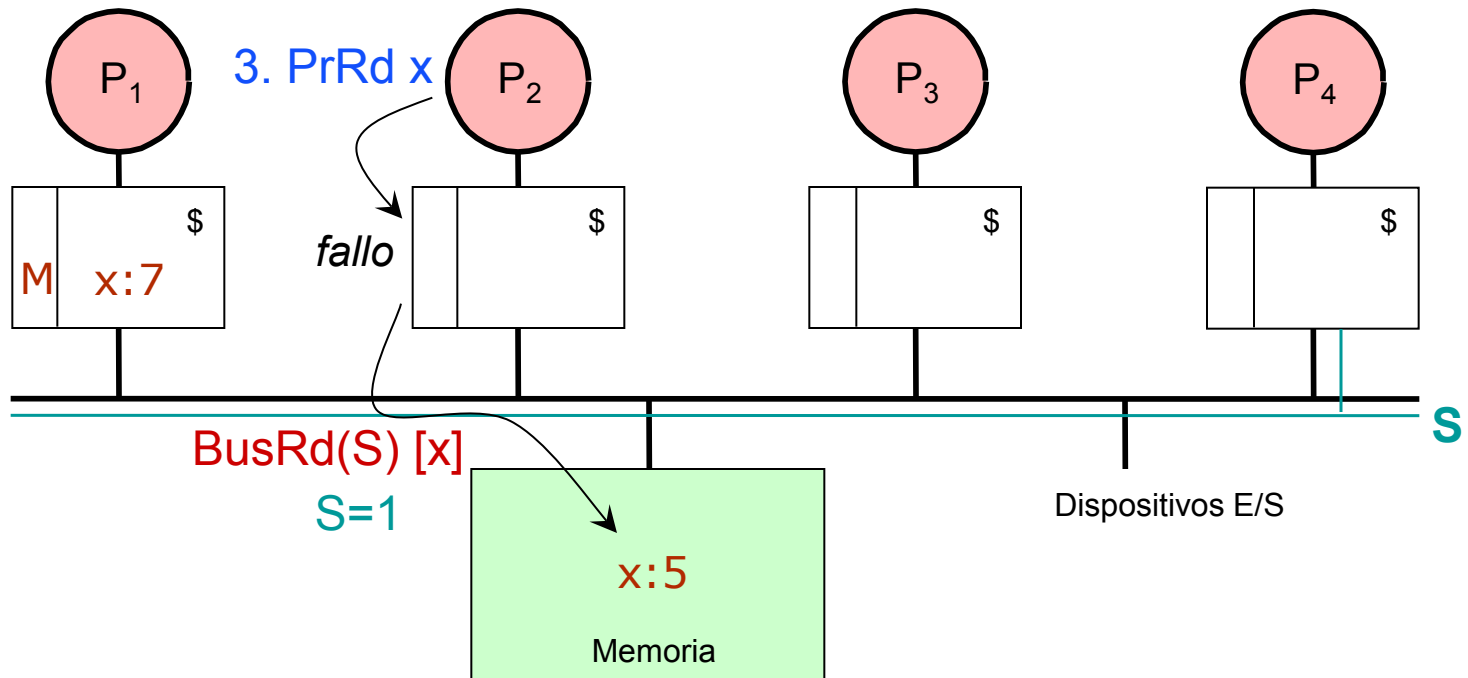
Protocolo MOESI: Ejemplo



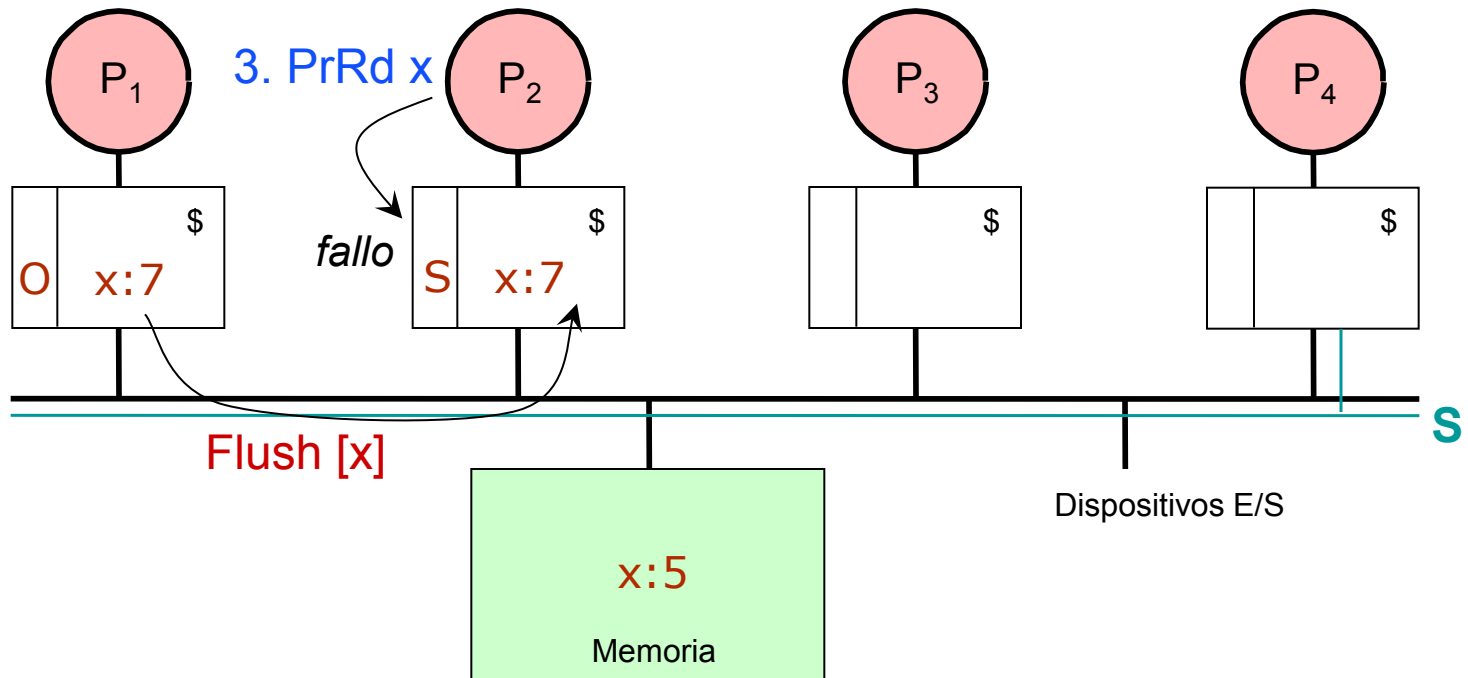
Protocolo MOESI: Ejemplo



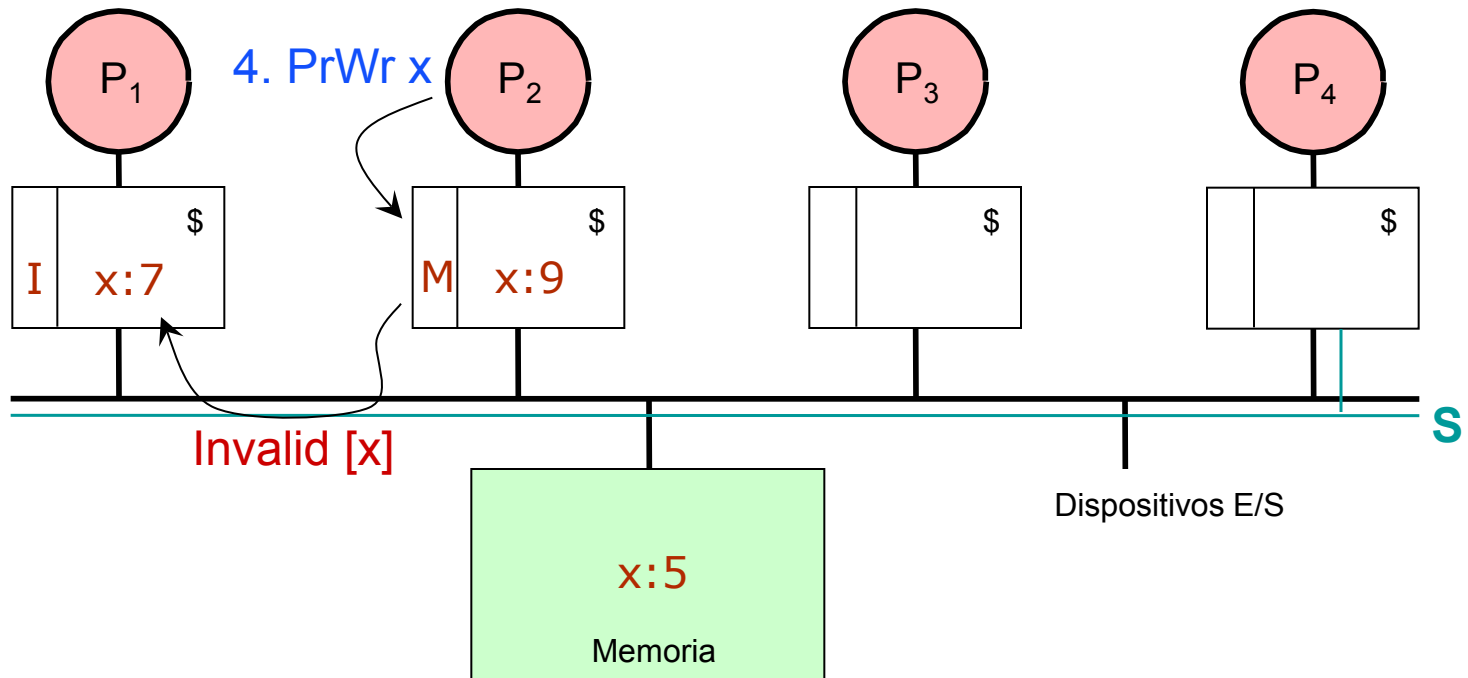
Protocolo MOESI: Ejemplo



Protocolo MOESI: Ejemplo

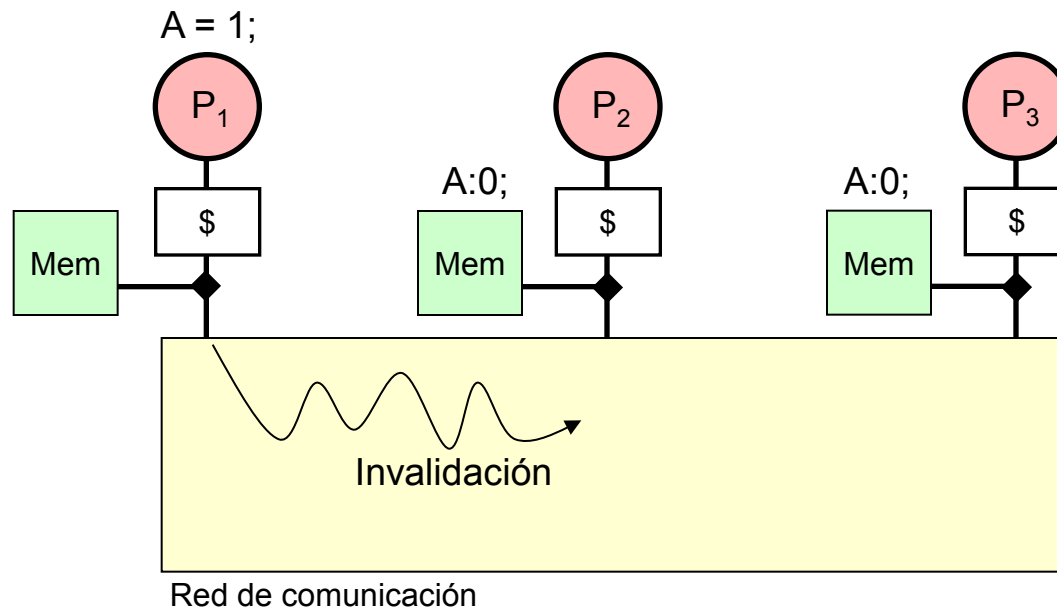


Protocolo MOESI: Ejemplo



Problema con la Coherencia Escalable

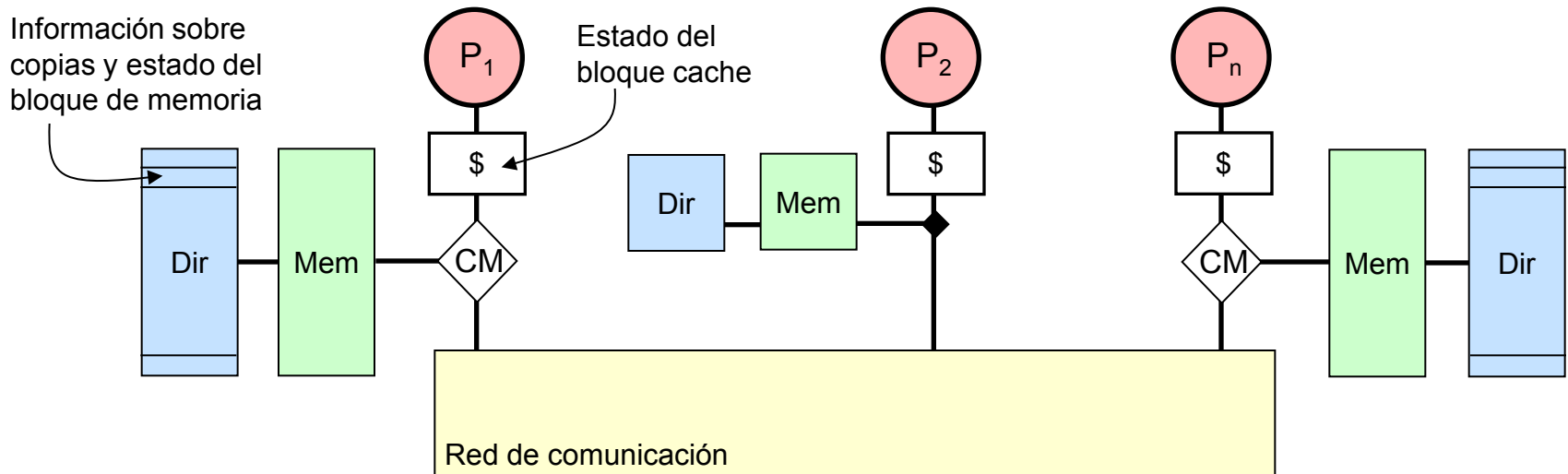
- En general, la coherencia es más compleja de hacer cumplir en un sistema escalable
 - Las transacciones de red no son visibles globalmente (propagación de escrituras)
 - No hay un árbitro de red global que permita serializar las escrituras



Solución Básica Basada en Directorios

- Las transacciones de red no son visibles globalmente y no existe un bus común a todos los nodos, por lo que debemos introducir un nuevo recurso hardware que nos dé información global sobre las copias cache de los bloques de memoria: **Directorio**
- Funcionalidad de un directorio
 - Cada bloque de memoria tiene asociado una entrada en el directorio, que guarda información sobre las copias cache de ese bloque y su estado
 - Las operaciones de lectura y escritura modifican la información de estado mediante transacciones de red
- Ejemplo de operación
 - Cuando se produce un fallo cache, el nodo peticionario envía una transacción de red al módulo de memoria correspondiente
 - Antes de acceder a la memoria, se busca en el directorio información sobre el bloque cache fallado
 - Esta información se utiliza para enviar nuevas transacciones de red a los nodos oportunos

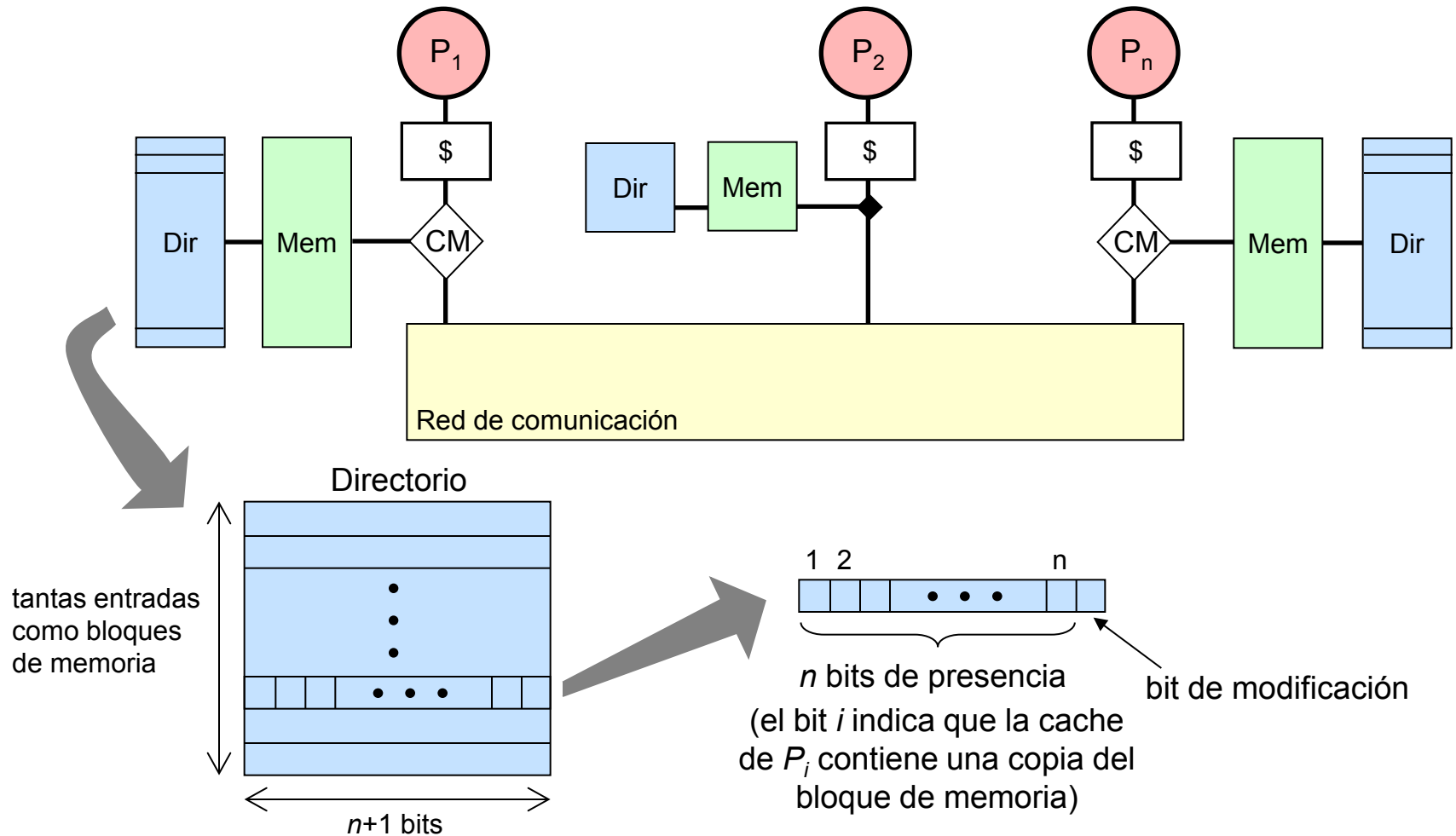
Esquema de Directorios



- **Coherencia cache escalable**

- **Directorio:** Contiene información sobre el estado y ubicación de las las copias cache de cada bloque de memoria
- **Protocolo de coherencia:** Diagrama de estados de los bloques cache junto con acciones, reacciones y transiciones de estado
- El directorio sustituye al mecanismo snoop de un SMP de bus común
- El protocolo de coherencia es similar a un SMP de bus común

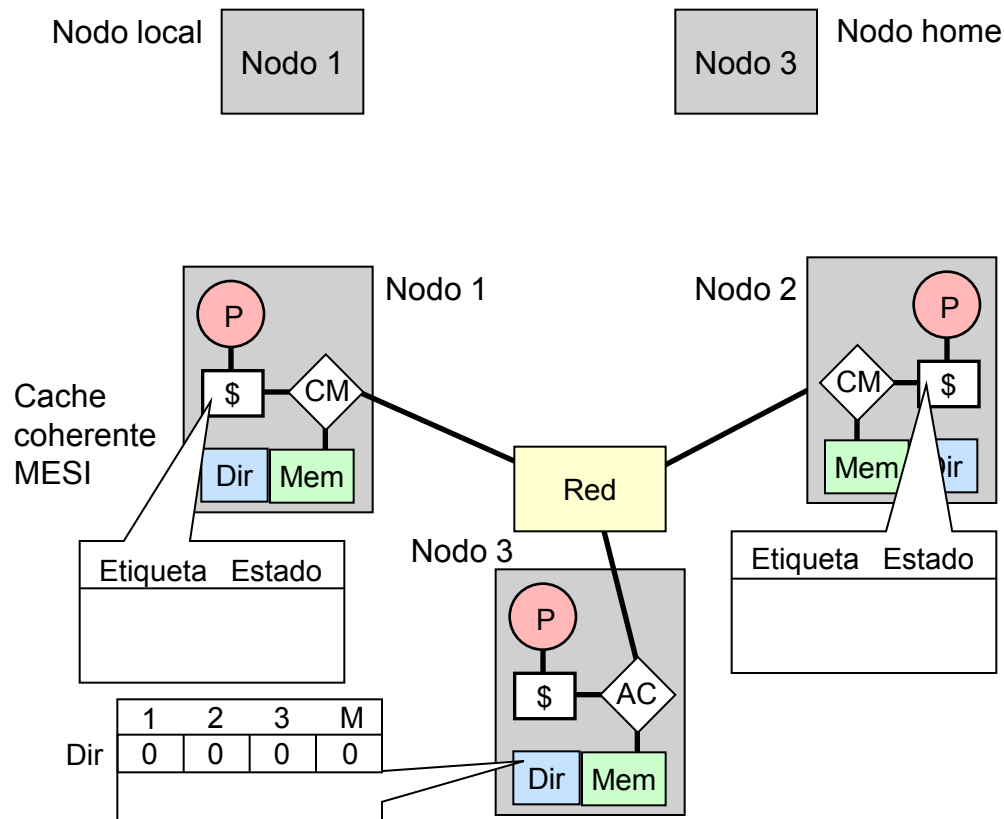
Organización Básica de un Directorio



- Los directorios eliminan la necesidad de broadcasts en la red

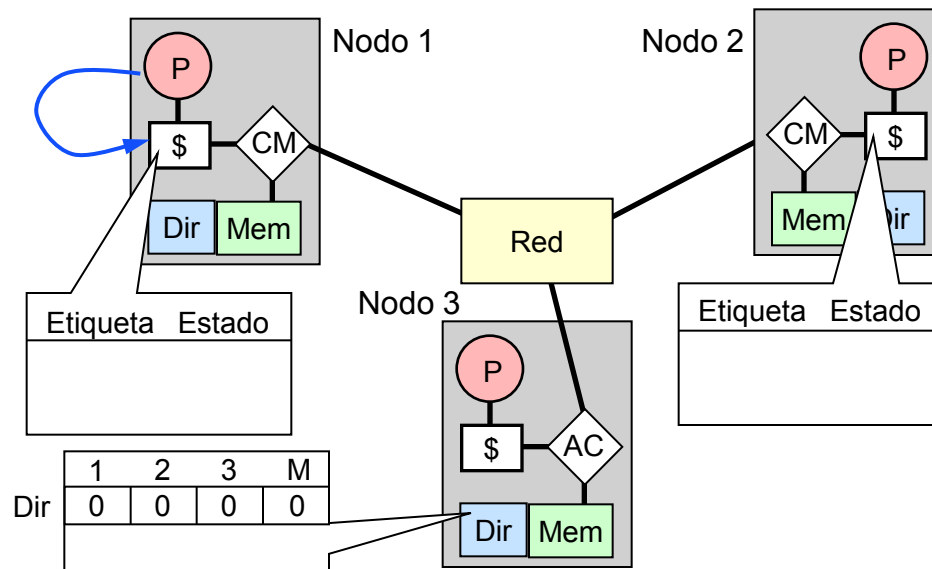
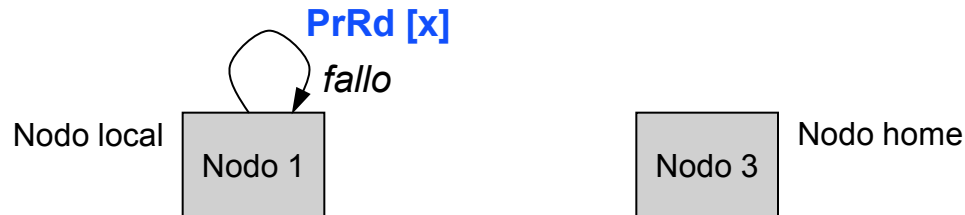
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque no modificado



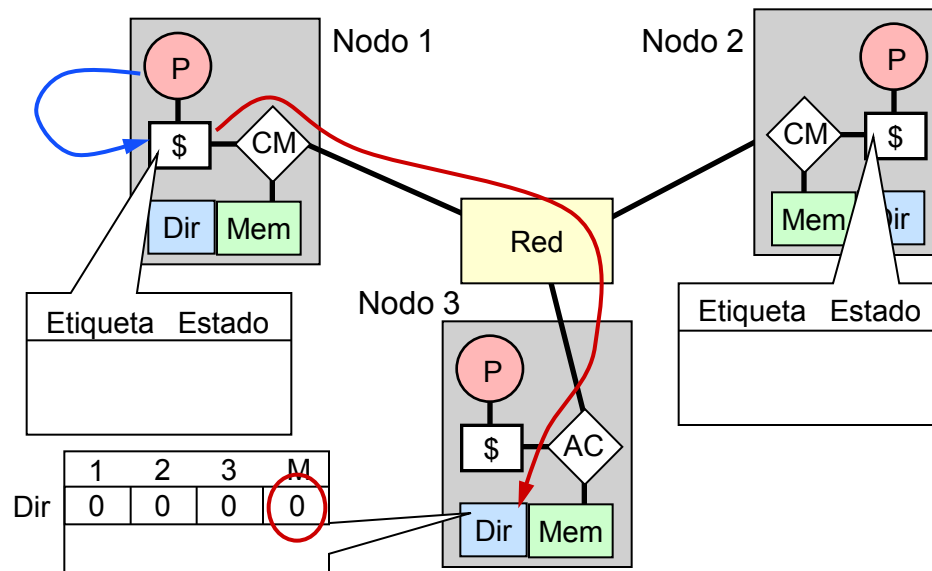
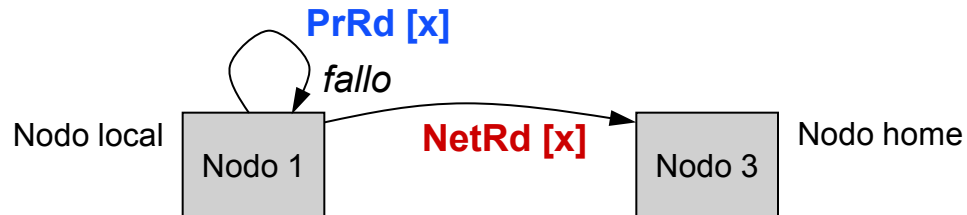
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque no modificado



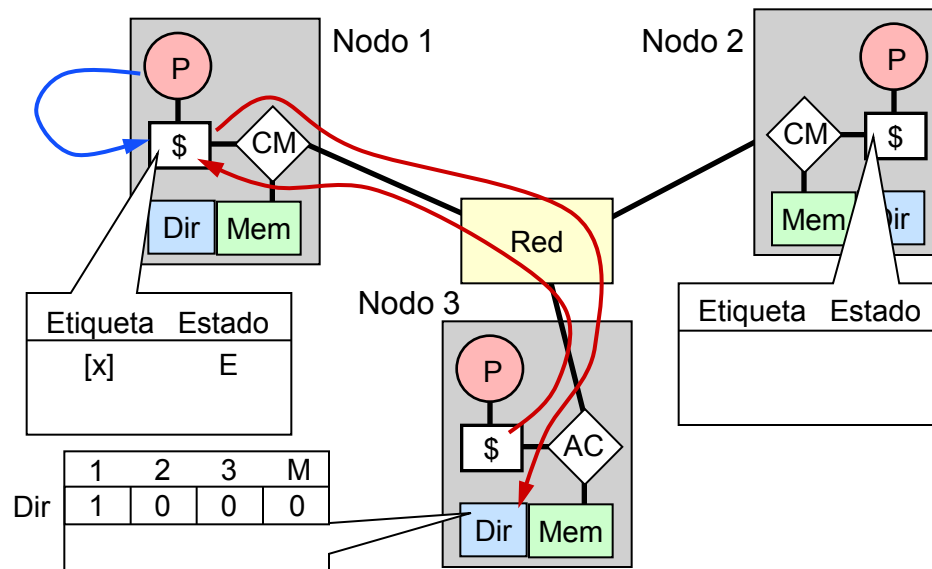
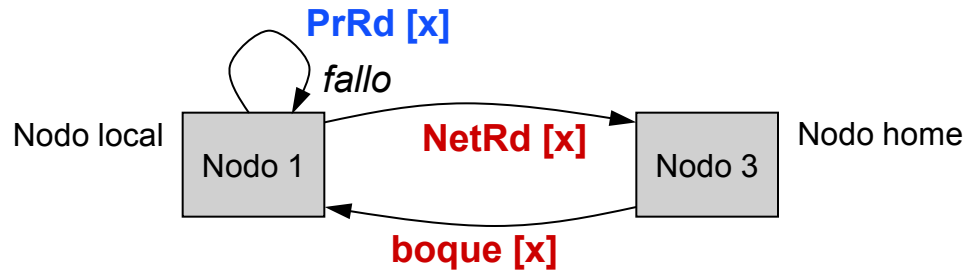
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque no modificado



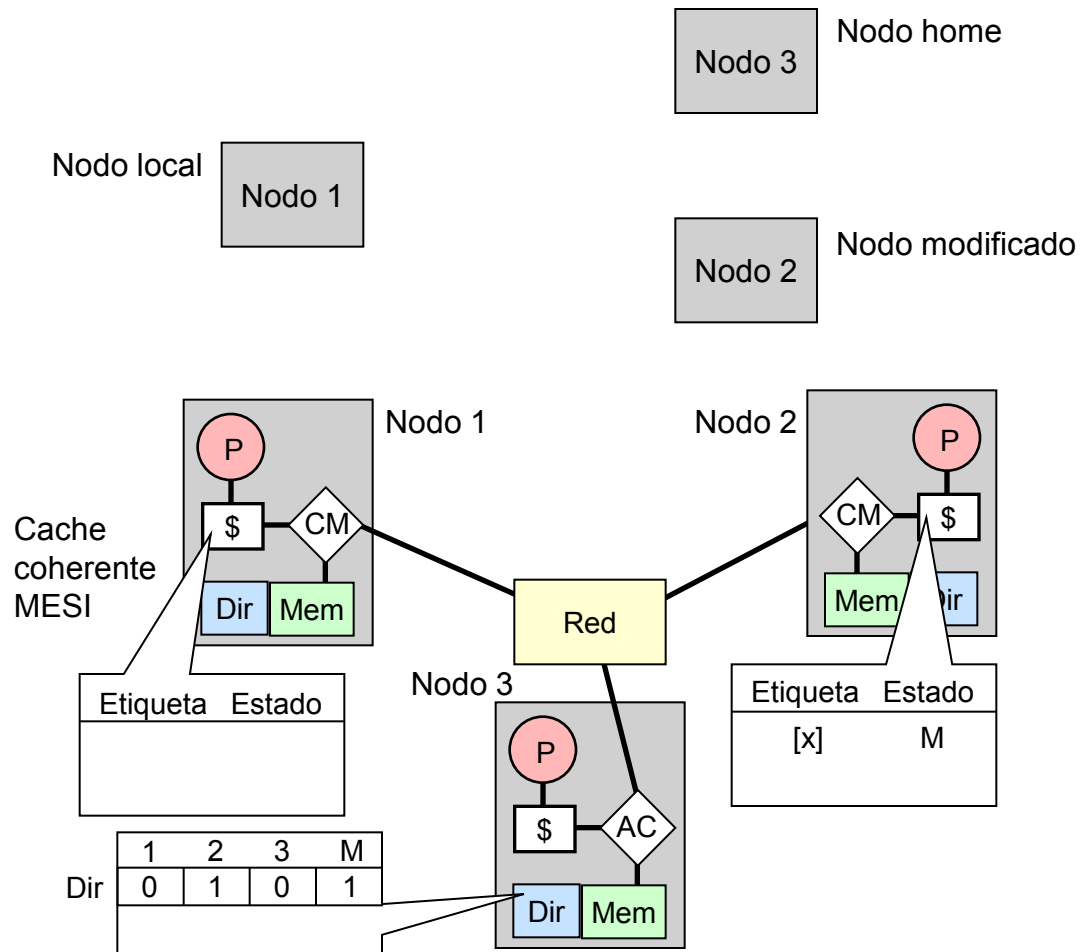
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque no modificado



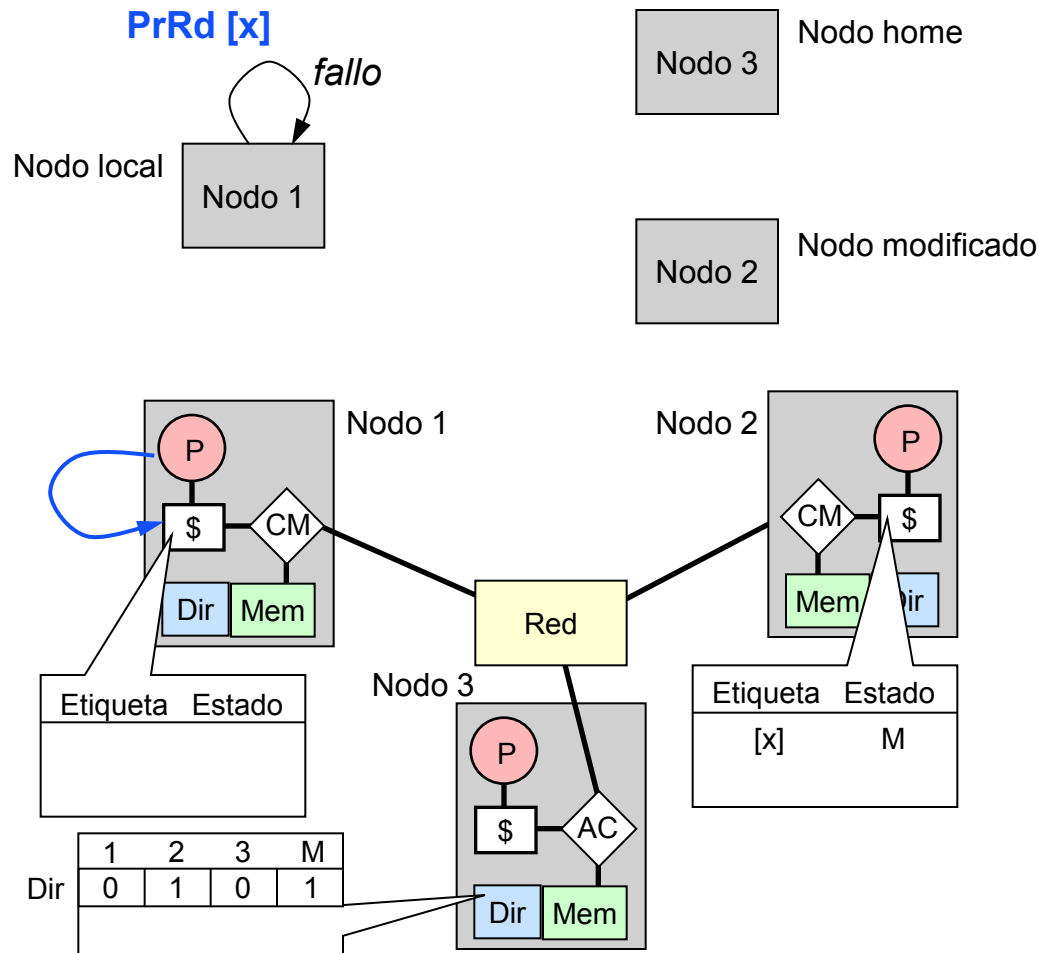
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



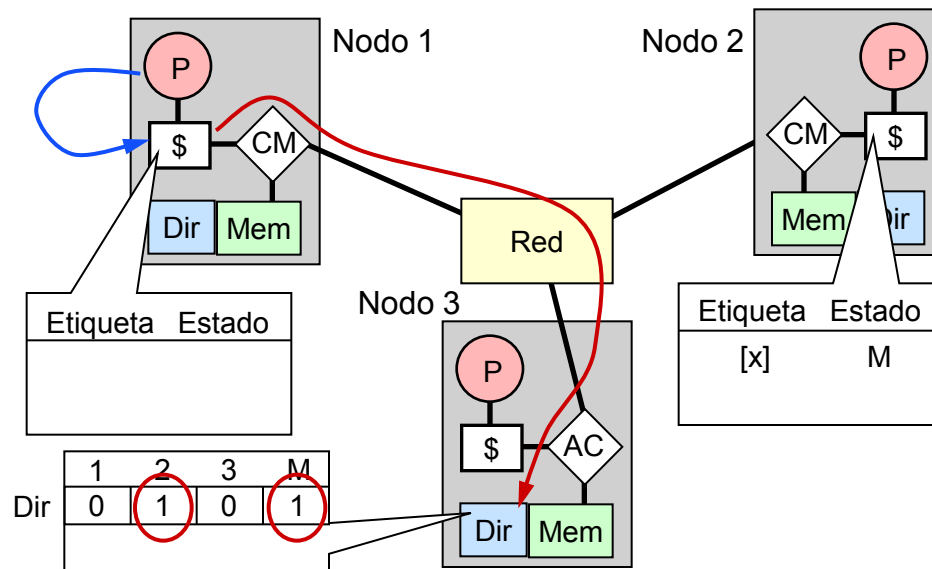
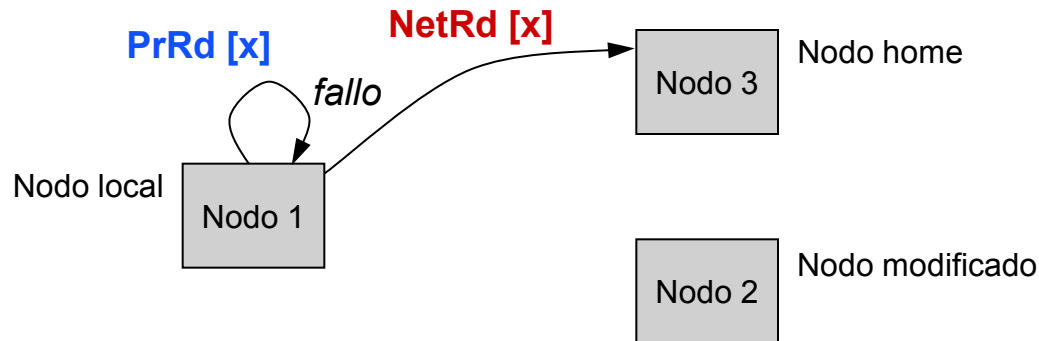
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



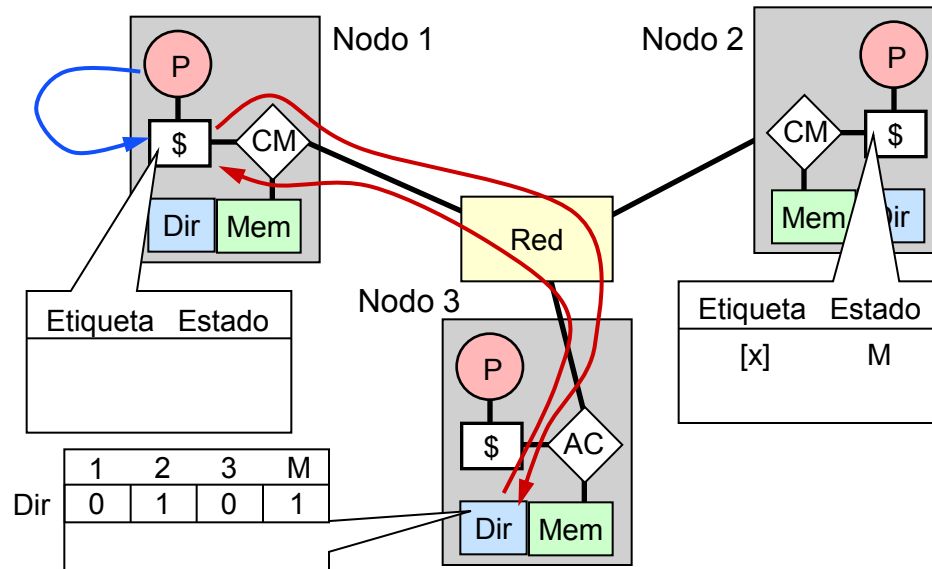
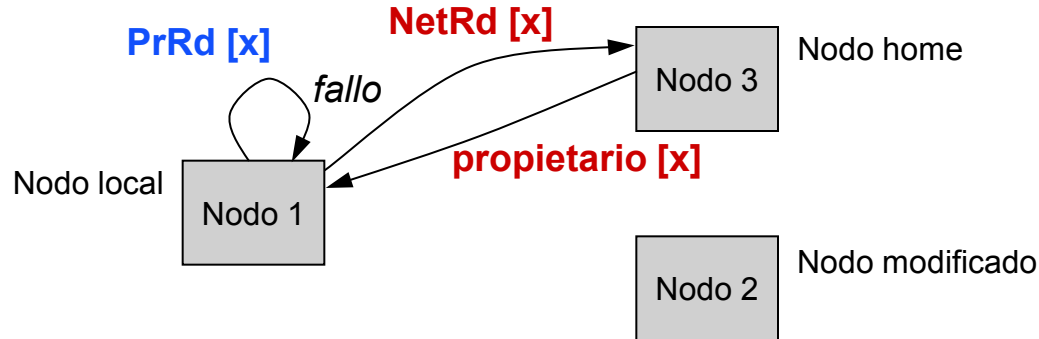
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



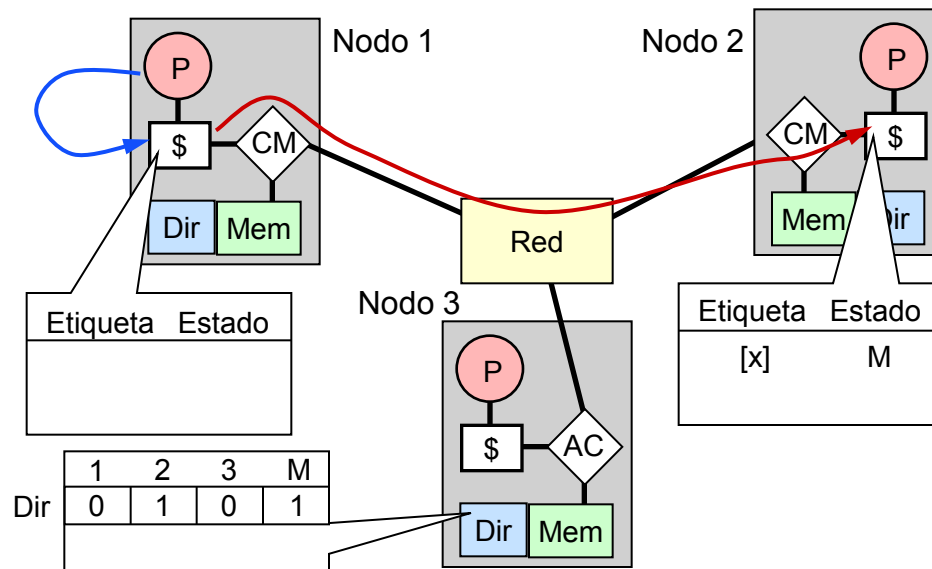
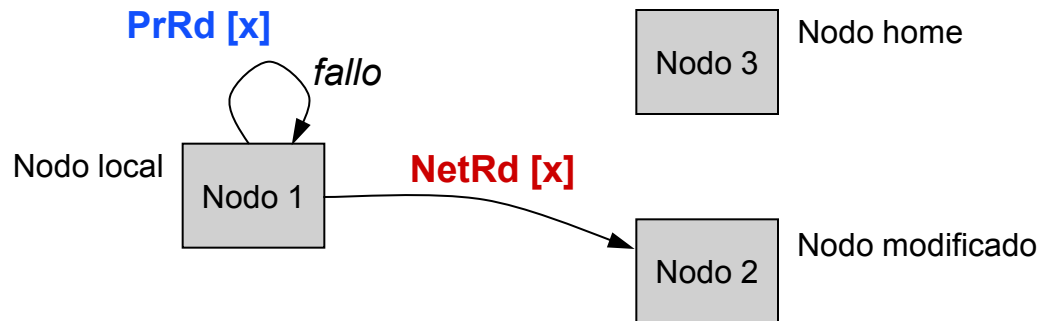
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



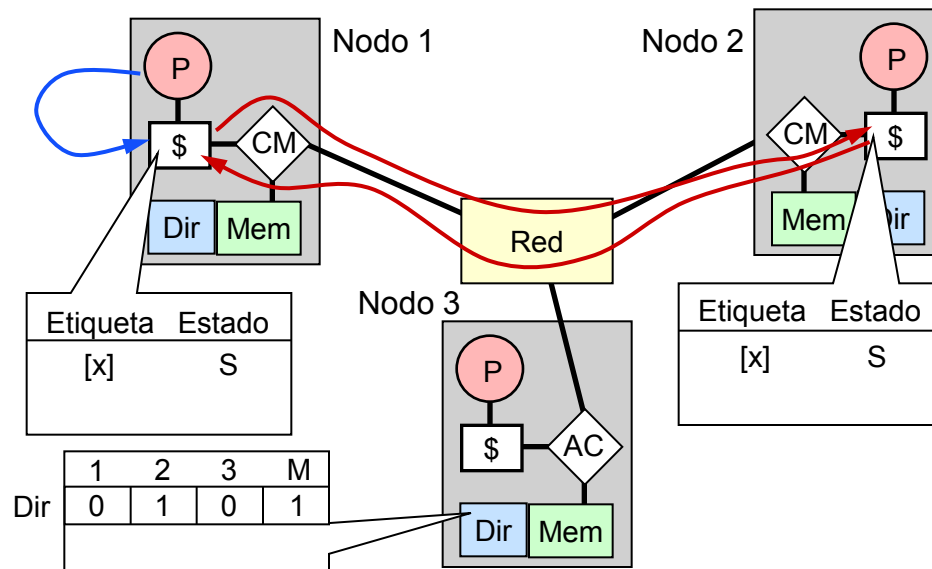
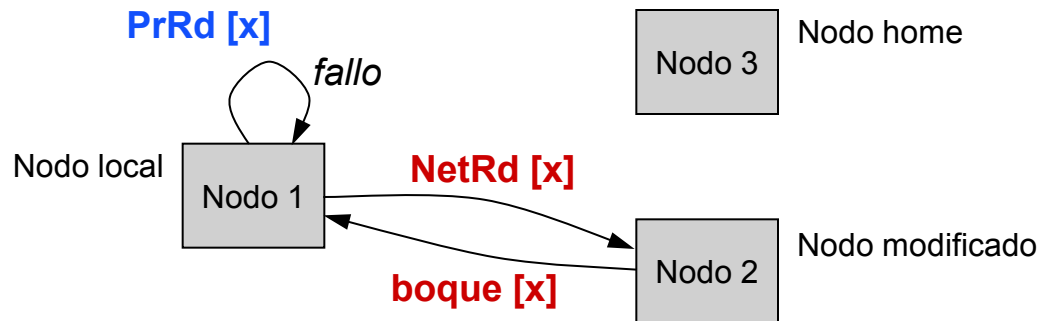
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



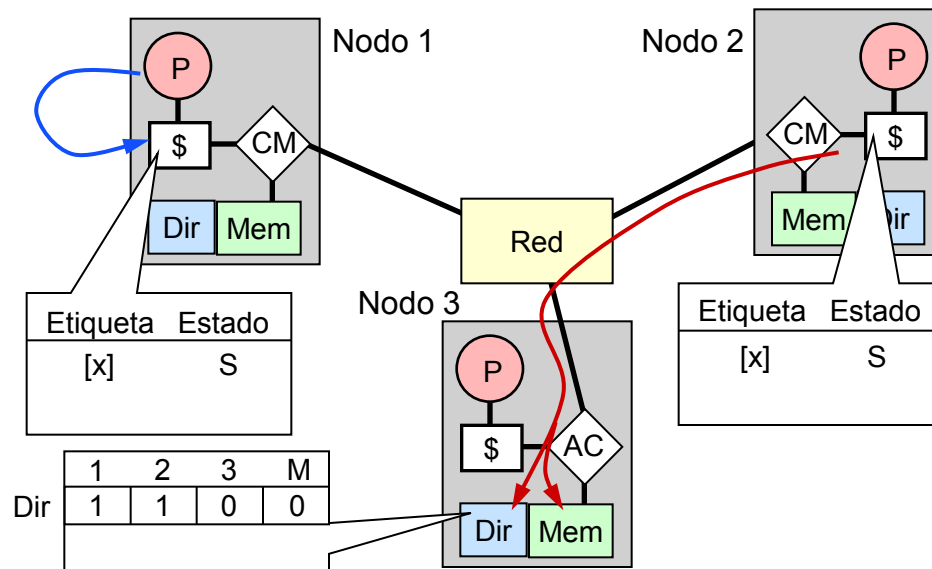
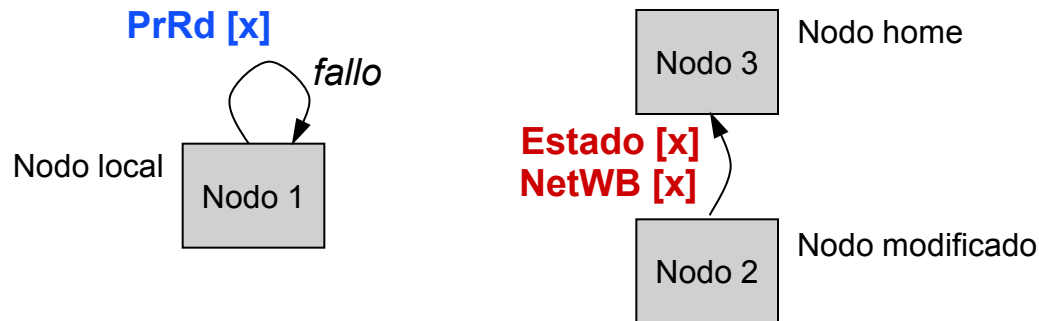
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



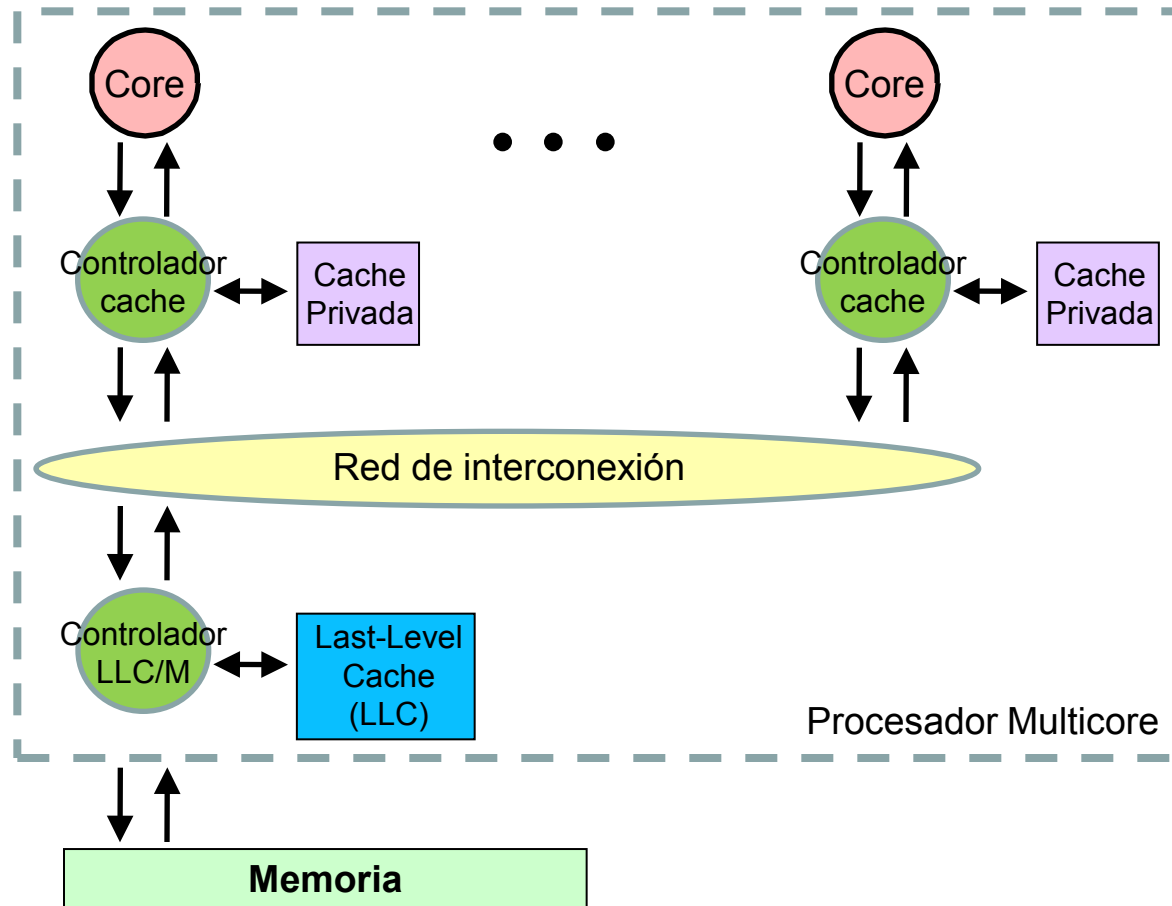
Operaciones de Memoria: Lectura

- Fallo de lectura de un bloque modificado



Procesador Multicore: Coherencia Cache

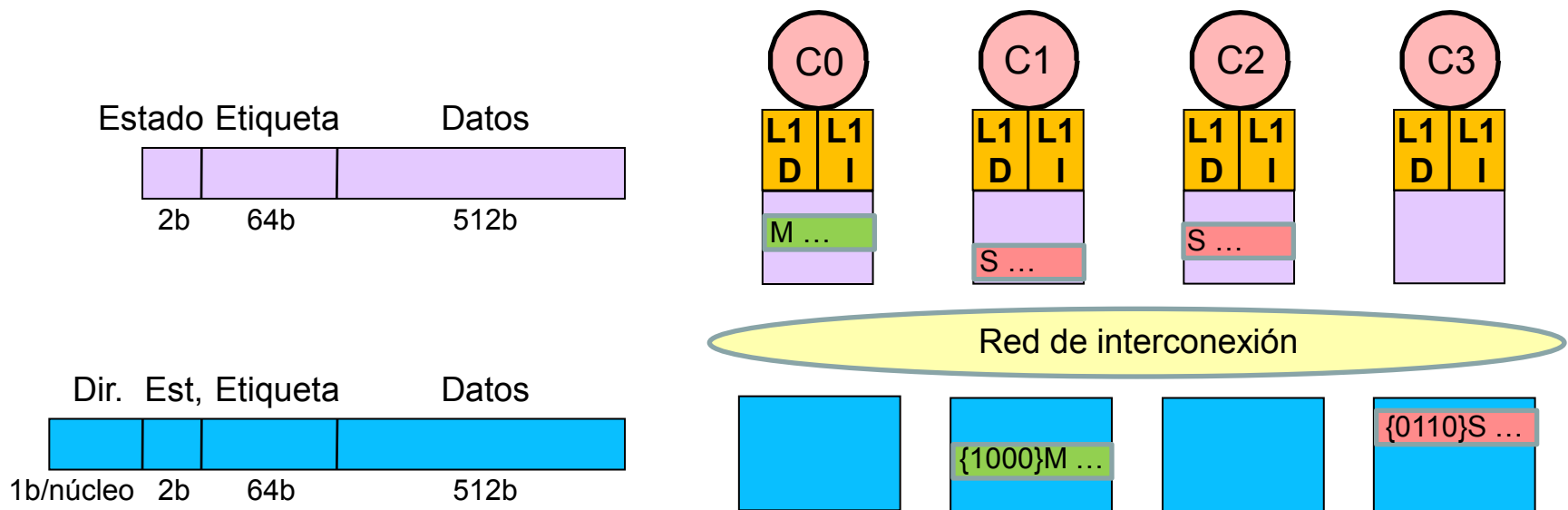
- Sistema básico



Coherencia Cache: Ejemplo

• Ejemplos comerciales

- Quad-core Intel Core i7 (Lynnfield)
 - » L1D y L1I: 32KB
 - » L2: 256KB, privada a cada núcleo
 - » L3: 8MB, compartida entre todos los núcleos, multi-banco, UCA
- Protocolo de coherencia: MESIF
- Tipo de coherencia: Basado en directorios (ampliación del controlador cache)

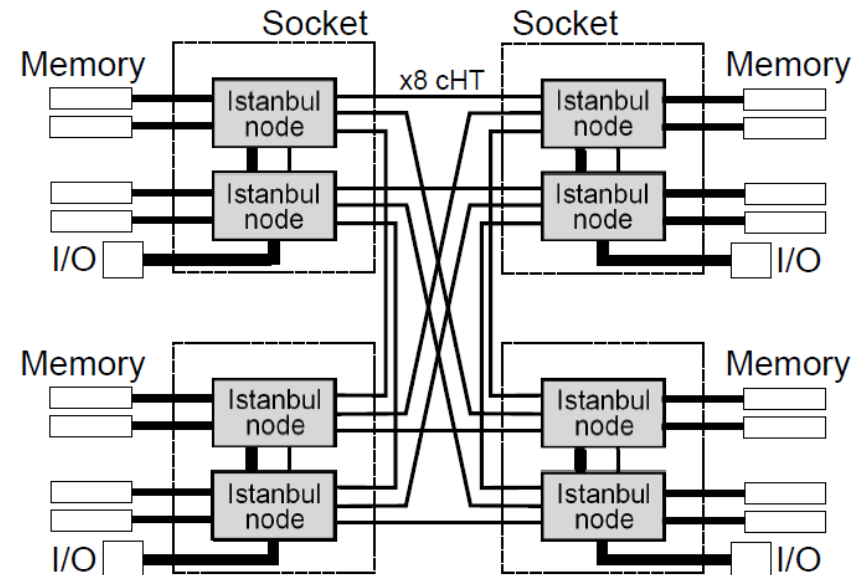
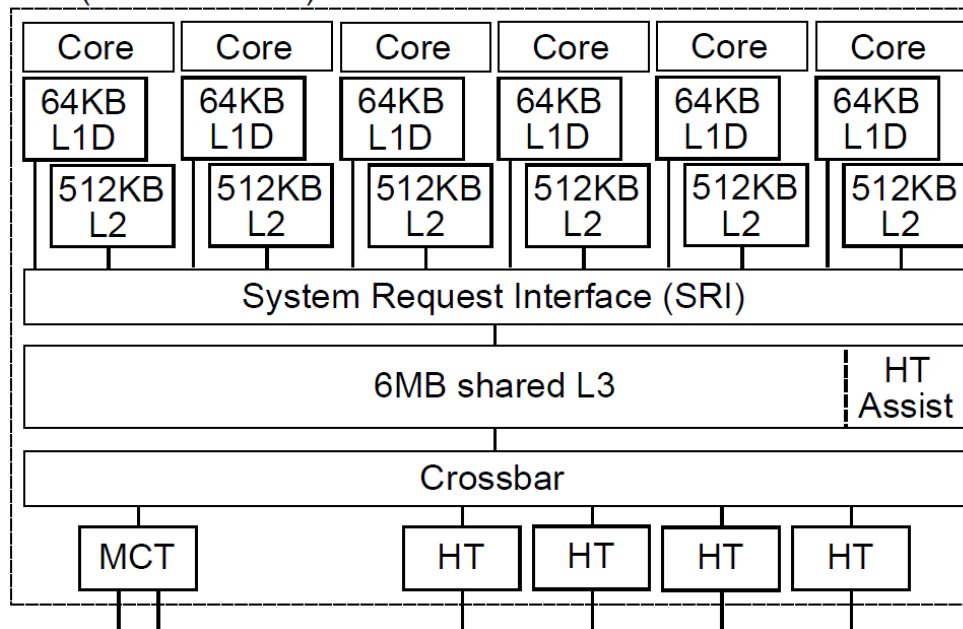


Coherencia Cache: Ejemplo

• Ejemplos comerciales

- Twelve-core AMD Opteron 6172 (Magny-Cours)
- Protocolo de coherencia: MOESI
- HT Assist: Directorio inclusivo con tres estados
 - » Reenviar petición de coherencia a todos los núcleos
 - » Reenviar petición de coherencia a un solo núcleo
 - » Petición de coherencia resuelta en L3

Die (Istanbul node)

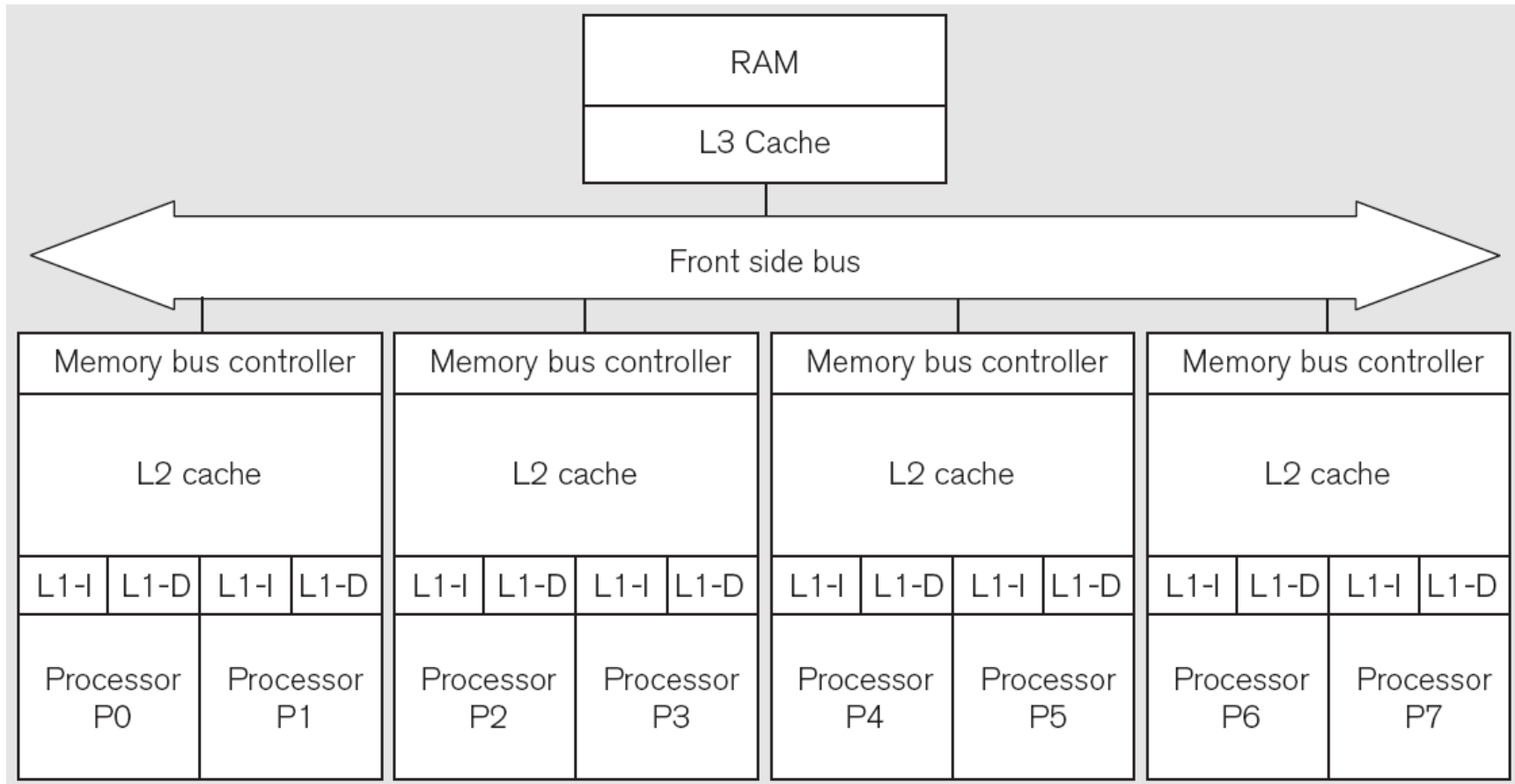


Indice

- Introducción
- Procesadores multicore: Jerarquía cache
- Coherencia cache
- **Optimización del uso de la jerarquía cache multicore**

Ejemplo: Código Multithread Básico

- Procesador multicore

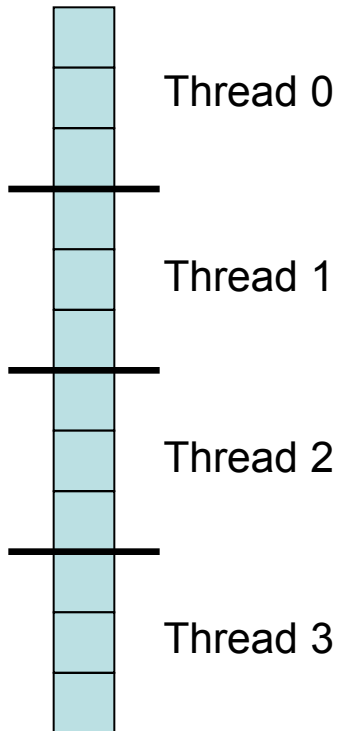


Código Secuencial

- Contar el número de valores '3' en un array

```
int *array;  
int length, count;  
  
int count3s()  
{  
    int i;  
    count = 0;  
    for (i=0; i<length; i++)  
    {  
        if (array[i] == 3)  
            count++;  
    }  
    return count;  
}
```

Primer Código Multithread



```
int t;      // numero de threads
int *array, length, count;

int count3s()
{
    int i; count = 0;
    for (i=0; i<t; i++)    // crear t threads
        thread_create(count3s_thread, i);
    return count;
}

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3) count++;
}
```

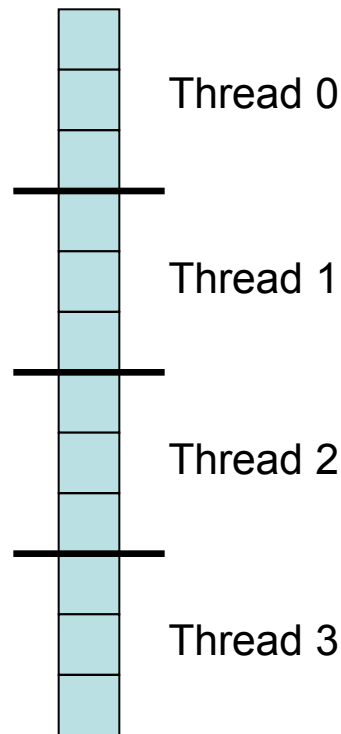
Condición de carrera!



Segundo Código Multithread

- Solución

- Proteger la actualización del contador mediante una sección crítica



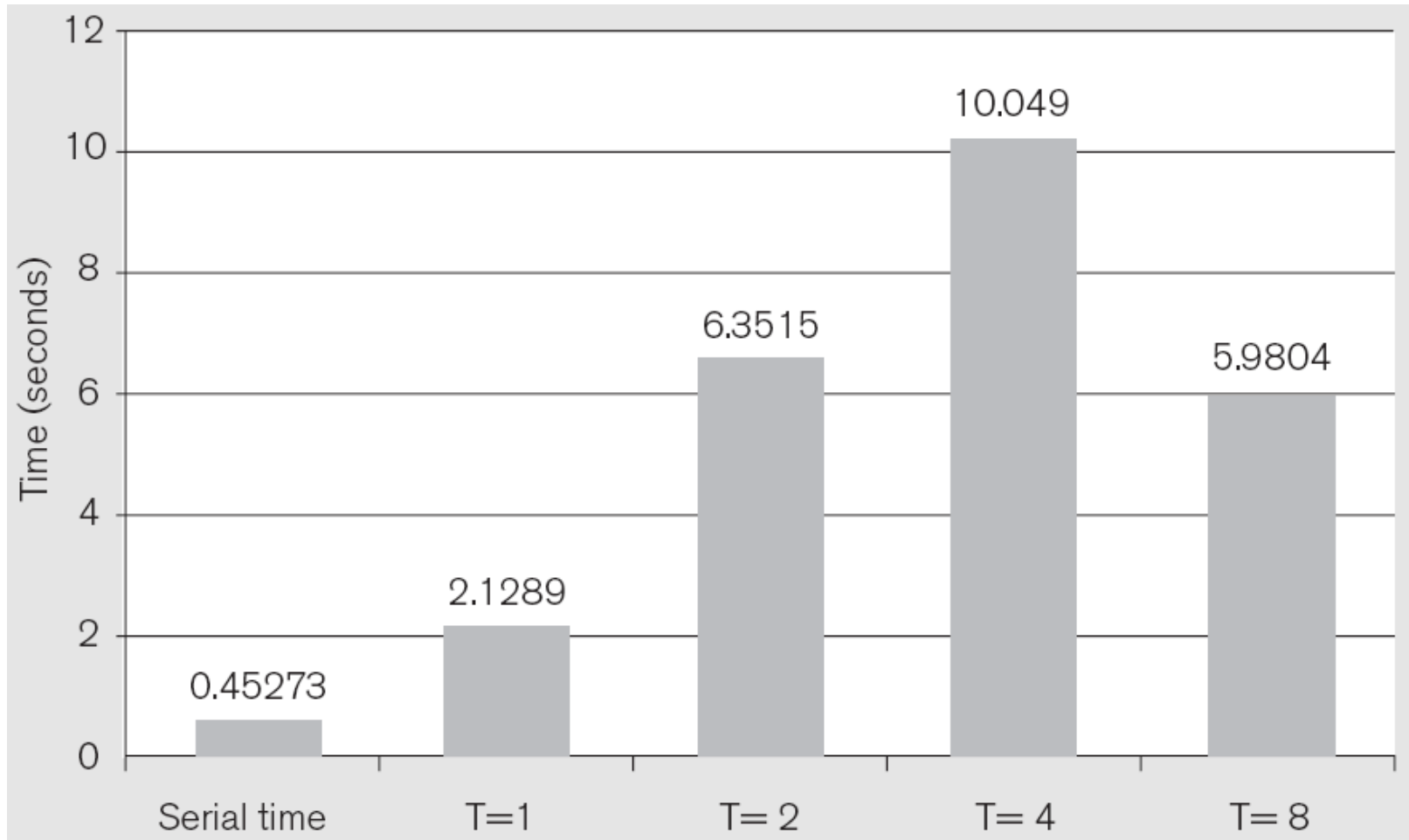
```
mutex m;

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
        {
            mutex_lock(m);
            count++;
            mutex_unlock(m);
        }
}
```

Segundo Código Multithread

- Rendimiento

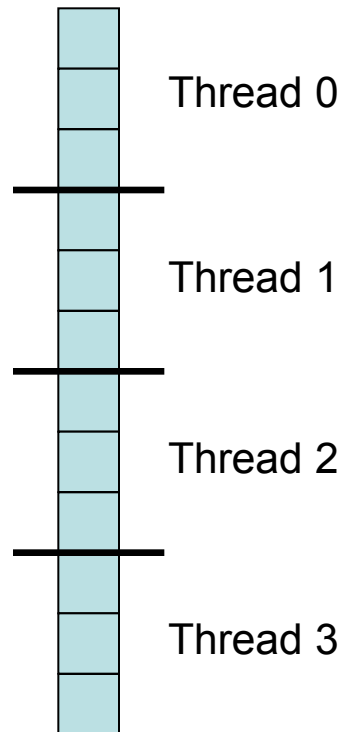
- Bajo rendimiento debido a coste de la sección crítica y a la contención



Tercer Código Multithread

- Solución

- Privatizar el contador y acumular cuentas parciales

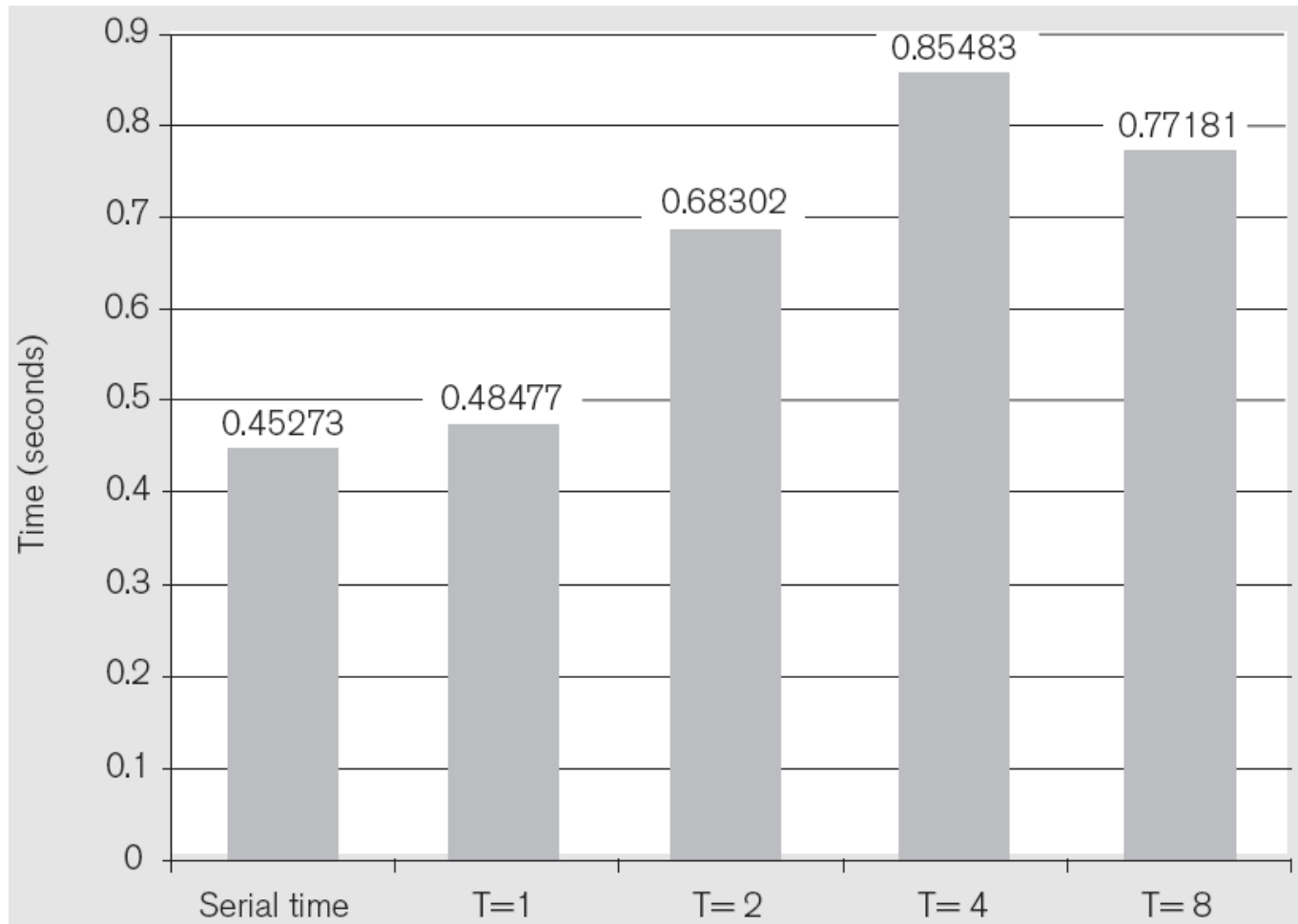


```
mutex m;  
int private_count[MaxThreads];  
  
int count3s_thread(int id)  
{  
    int i; int length_per_thread = length/t;  
    int start = id*length_per_thread;  
    for (i=start; i<start+length_per_thread; i++)  
        if (array[i] == 3)  
            private_count[id]++;  
    mutex_lock(m);  
    count += private_count[id];  
    mutex_unlock(m);  
}
```


Tercer Código Multithread

- Rendimiento

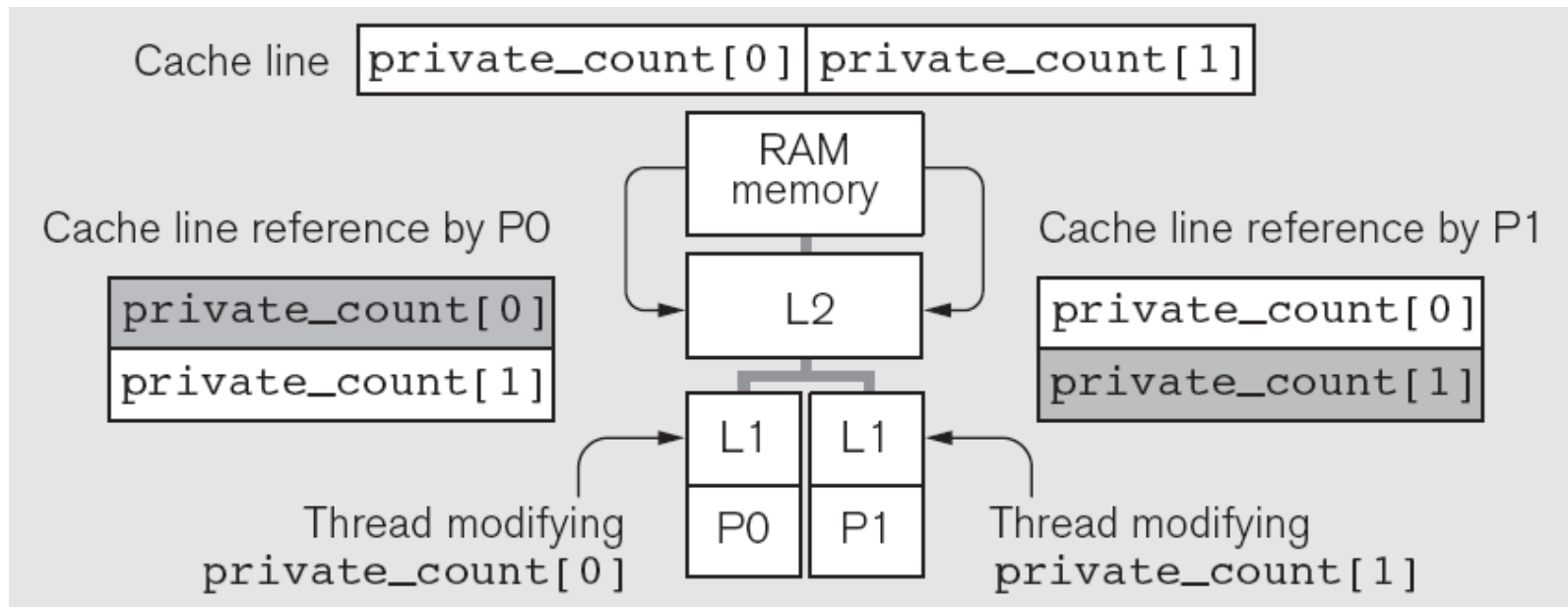
- Bajo rendimiento debido a la compartición falsa



Tercer Código Multithread

- Rendimiento

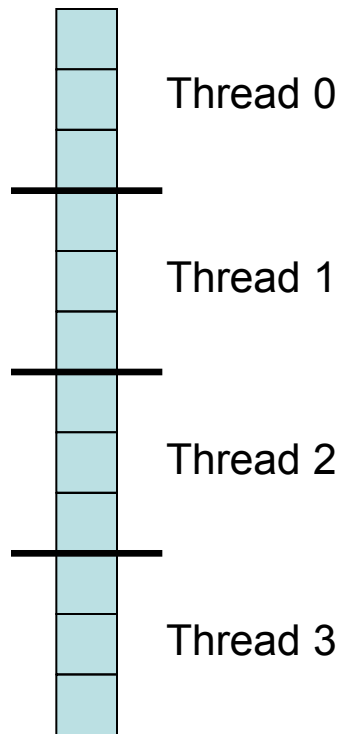
- Bajo rendimiento debido a la compartición falsa



Cuarto Código Multithread

- Solución

- Aplicar padding al contador



```
mutex m;
struct padded_int
{
    int value;
    char padding[60];
} private_count[MaxThreads];

int count3s_thread(int id)
{
    int i; int length_per_thread = length/t;
    int start = id*length_per_thread;
    for (i=start; i<start+length_per_thread; i++)
        if (array[i] == 3)
            private_count[id].value++;
    mutex_lock(m);
    count += private_count[id].value;
    mutex_unlock(m);
}
```

Cuarto Código Multithread

- Rendimiento

- Buen rendimiento, sólo limitado por el ancho de banda de la cache L2 (para T grandes)

